ADTECH众为兴

把工作交给机器 将思想留给自己

ADT-632XE 系列运动控制卡编程手册



深圳众为兴技术股份有限公司

地址：深圳市宝安区福海街道新塘路中粮智造科技园 93-1 号 P1 栋 邮编：518100

电话：0755-26722719 传真：0755-26722718

email：tech@adtechcn.com http://www.adtechcn.com

版权声明

本手册的所有部分，著作财产权归深圳众为兴技术股份有限公司（以下简称众为兴）所有，未经众为兴许可，任何单位或个人不可任意仿制、拷贝、撰抄或转译。本手册无任何形式的担保、立场表达或其它暗示。如由本手册或其所提到的产品的信息，所引起的直接或间接的资料流出，利益损失或事业终止，众为兴及其所属员工不承担任何责任。除此以外，本手册提到的产品及其资料仅供参考，内容如有更新，恕不另行通知。

版权所有，不得翻印。

深圳众为兴技术股份有限公司

说明书基本信息

本说明书由深圳众为兴技术股份有限公司组织编写。

小提示	本手册蓝色带下划线的字样，如 adt_initial 等，均为可链接标识，其链接内容将为您带来更详尽的信息解读。 本手册中所有“控制卡”，”运动控制卡“字样都代指运动控制卡

目录

第一章 指令列表 13

第二章 错误码 19

2.1. 错误码 19

2.2. 错误码解析示例 22

第三章 SDK 调用方法 26

3.1. VC 调用方法 26

3.2. C# 调用方法 27

第四章 基础控制与版本信息 29

4.1. 功能介绍 29

4.1.1. 控制卡初始化 29

4.1.2. 控制卡关闭 29

4.1.3. 控制卡复位 29

4.1.4. 控制卡软件重启 29

4.1.5. 可用控制卡索引获取 30

4.1.6. 控制卡版本信息获取 30

4.2. 例程 4-1 基础控制与版本信息获取 30

第五章 从站配置 32

5.1. 功能介绍 32

5.1.1. 从站配置 32

5.1.2. 从站通信 32

5.1.3. 主从站信息获取 33

5.1.4. 从站资源获取 33

5.2. 例程 5-1 从 RAM/ROM 中加载从站配置，解析配置，获取轴资源信息 33

5.3. 例程 5-2 从本地文件导入从站配置 35

第六章 资源配置 37
6.1. 功能介绍 37
6.1.1. 总线 IO 端口映射 37
6.1.2. 硬件停止信号(EMGN) 37
6.1.3. 硬件限位信号(LMT+/LMT-) 37
6.1.4. 机械原点信号(STOP0) 38
6.1.5. 编程模式 38
6.1.6. 电子齿轮比 39
6.1.7. 脉冲当量 39
6.1.8. 脉冲当量与电子齿轮的计算方法 40
6.1.9. 软件限位 41
6.1.10. 编码器反馈分频比 41
6.1.11. 逻辑变环模式 41
6.1.12. 到位误差与跟随误差 42
6.1.13. 龙门双驱 43
6.2. 例程 6-1 总线 IO 端口映射 44
6.3. 例程 6-2 配置轴基本参数 45
第七章 轴状态监视与控制 46
7.1. 功能介绍 46
7.1.1. 逻辑位置 46
7.1.2. 实际位置 46
7.1.3. 目标位置 46
7.1.4. 伺服使能 46
7.1.5. 轴规划速度 46
7.1.6. 编码器反馈速度 46
7.2. 获取轴状态信息 47
7.2.1. 轴运动状态 47
7.2.2. 停止信息 47
7.3. 例程 7-1 轴状态监控 48
7.4. 例程 7-2 停止信息解释示例 49
第八章 速度规划 50
8.1. 功能介绍 50

8.1.1. 基础速度规划 50
8.1.2. 集成速度设置 50
8.1.3. 速度倍率 51
8.1.4. 速度前瞻 51
8.1.5. 圆弧插补速度约束 52
8.1.6. 拐角加速度平滑等级 53
第九章 点位运动 54
9.1. 功能介绍 54
9.1.1. 相对位置与绝对位置 54
9.1.2. 定量驱动 54
9.1.3. 连续驱动 55
9.1.4. 在线改变目标位置 55
9.1.5. 轴驱动停止 55
9.2. 例程 9-1 定量驱动 55
9.3. 例程 9-2 连续驱动 56
9.4. 例程 9-6 龙门双驱 58
第十章 插补运动 61
10.1. 功能介绍 61
10.2. 创建插补坐标系 61
10.3. 直线插补 62
10.4. 平面圆弧插补 62
10.5. 球面圆弧插补 63
10.6. 缓存插补 63
10.7. 插补暂停与恢复 64
10.8. 例程 10-1 直线插补 64
10.9. 例程 10-2 平面圆弧插补 66
10.10. 例程 10-3 缓存插补 67
第十一章 PT 73
11.1. 功能介绍 73
11.1.1. 点位模式 73
第十二章 回零控制 74
12.1. 功能介绍 74

12.1.1. DSP 回零 74
12.1.2. 总线伺服回零 75
12.2. 例程 12-1 DSP 回零 75
12.3. 例程 12-2 总线伺服回零 77
第十三章 总线 IO 控制 79
13.1. 功能介绍 79
13.1.1. 伺服 IO 控制 79
13.1.2. 总线 IO 模块控制 79
第十四章 缓存事件控制 80
14.1. 功能介绍 80
14.1.1. 缓存输出控制 80
14.1.2. 缓存延时控制 80
14.1.3. 缓存运动暂停等待输入 80
14.1.4. 缓存事件余量 80
14.1.5. 缓存事件清空 81
14.2. 例程 14-1 缓存事件控制 81
第十五章 SDO/PDO 84
15.1. 功能介绍 84
第十六章 用户控制 86
16.1. 功能介绍 86
16.2. 例程 16-1 用户控制 86
第十七章 ADT-9188 扩展板控制 88
17.1. 功能介绍 88
17.1.1. 位置比较 88
17.1.2. 高速 IO 端口 89
17.2. 例程 17-1 2D 位置比较 89
第十八章 标准指令详述 93
18.1. 基础控制类 93
adt_initial 93
adt_close_card 93
adt_reset_card 94
adt_soft_reboot 94

adt_get_card_index 95
adt_get_motion_error 95
18.2. 版本信息类 97
adt_get_lib_ver 97
adt_get_motion_ver 97
adt_get_firmware_ver 98
18.3. 总线从站控制类 99
adt_ecat_download_cfg 99
adt_ecat_load_flash_cfg 99
adt_ecat_stop 100
adt_ecat_start 100
adt_get_ecat_status 101
adt_get_ecat_slave_resource 101
adt_get_ecat_real_slave_num 102
adt_get_ecat_slave_info 102
adt_get_axis_ecat_node 103
adt_get_total_axis 104
18.4. 资源配置类 105
adt_set_axis_io_map 105
adt_get_axis_io_map 105
adt_set_hardlimit_mode 107
adt_get_hardlimit_mode 107
adt_set_emergency_stop 108
adt_get_emergency_stop 109
adt_set_limit_mode 109
adt_get_limit_mode 110
adt_set_stop0_mode 111
adt_get_stop0_mode 111
adt_set_unit_mode 112
adt_get_unit_mode 112
adt_set_pulse_equiv 113
adt_get_pulse_equiv 113

adt_set_bus_axis_gear_ratio 114
adt_get_bus_axis_gear_ratio 114
adt_set_softlimit_mode 115
adt_get_softlimit_mode 116
adt_set_pos_variable_loop 116
adt_get_pos_variable_loop 117
adt_set_encoder_scale 117
adt_set_axis_band 118
adt_get_axis_band 119
adt_set_axis_pos_error 119
adt_set_follow_axis 120
18.5. 轴状态监视与控制 121
adt_set_command_pos 121
adt_get_command_pos 121
adt_set_actual_pos 122
adt_get_actual_pos 122
adt_get_target_pos_unit 123
adt_get_target_pos_pulse 123
adt_set_axis_enable 124
adt_get_axis_enable 125
adt_clear_bus_axis_alarm 125
adt_get_axis_alarm_info 126
adt_get_axis_status 127
adt_get_stopdata 127
adt_get_all_axis_move_status 128
adt_get_all_axis_reach_status 128
adt_get_axis_status_ex 129
adt_clear_axis_status 129
adt_get_speed 130
adt_get_encoder_speed 130
18.6. 速度规划类 132
adt_set_startv 132

adt_get_startv 132
adt_set_maxv 133
adt_get_maxv 133
adt_set_endv 134
adt_get_endv 134
adt_set_acc 135
adt_get_acc 136
adt_set_dec 136
adt_get_dec 137
adt_set_admode 137
adt_get_admode 138
adt_set_axis_move_para_unit 138
adt_set_rate 139
adt_get_rate 140
adt_set_corner_speed_smooth_level 140
adt_set_arc_speed_clamp_unit 141
adt_set_speed_pretreat_mode 141
18.7. 点位运动类 143
adt_pmove_unit 143
adt_multi_pmove_unit 143
adt_pmove_pulse 144
adt_continue_move 144
adt_change_pmove_pos 145
adt_set_axis_stop 145
18.8. 插补运动 147
adt_set_inp_coordinate 147
adt_set_inp_stop 147
adt_get_inp_pause_pos 148
adt_get_inp_fifo_len 148
adt_get_inp_index 149
adt_inp_move_unit 149
adt_inp_move_pulse 150

adt_inp_arc2_unit 151
adt_inp_arc3_unit 152
adt_inp_helix2_unit 153
adt_inp_helix3_unit 153
adt_nurbs_curve 154
18.9. PT 157
adt_set_pt_mode 157
adt_set_pt_data 157
adt_set_pt_table 158
adt_clr_pt_data 158
adt_get_pt_space 159
adt_get_pt_index 159
adt_pt_start 160
adt_set_pt_loop 161
adt_get_pt_loop 161
18.10. 回零控制类 163
adt_set_home_mode 163
adt_set_home_speed 164
adt_set_home_process 164
adt_get_home_status 165
adt_set_servo_home_para 165
adt_set_servo_home 166
adt_get_servo_home_status 167
adt_stop_servo_home 167
18.11. 总线 IO 控制类 168
adt_read_servo_inbit 168
adt_read_servo_inport 168
adt_write_servo_outbit 169
adt_read_servo_outbit 169
adt_read_busio_inbit 170
adt_read_busio_inport 170
adt_write_busio_outbit 171

adt_write_busio_outport 172
adt_read_busio_outbit 172
adt_read_busio_outport 173
18.12. 缓存事件控制类 174
adt_get_fifo_event_len 174
adt_get_fifo_event_index 174
adt_set_fifo_adjust_time 175
adt_set_fifo_adjust_pos 175
adt_set_fifo_outbit 176
adt_set_fifo_multi_io 176
adt_set_fifo_delay 177
adt_set_fifo_wait_inbit 178
adt_clear_fifo_event 178
18.13. 从站仿真控制类 180
adt_set_axis_resource 180
adt_get_axis_resource 180
adt_set_axis_bound 180
adt_get_axis_bound 181
adt_reset_axis_bound 182
adt_set_virtual_io_num 182
adt_get_virtual_io_num 182
adt_write_busio_inbit 183
adt_write_busio_inport 183
18.14. SDO/PDO 类 185
adt_ecat_sdo_read 185
adt_ecat_sdo_write 185
adt_ecat_pdo_read 186
adt_ecat_pdo_write 187
18.15. 17.14 用户控制类 188
adt_set_user_password 188
adt_check_password 188
18.16. ADT-9188 控制 189

adt_write_outbit 189
adt_read_outbit 189
adt_read_inport 190
adt_read_inbit 190
adt_set_input_filter 190
adt_get_compare_len 191
adt_get_compare_status 191
adt_clear_compare_point 192
adt_reset_compare 192
adt_set_compare_xd_mode 193
adt_get_compare_xd_mode 194
adt_set_compare_xd_area 194
adt_add_compare_xd_point 195
adt_set_encoder_mode 196
adt_get_endoder_pos 196
adt_set_endoder_pos 197
18.17. 辅助功能 198
adt_get_bit 198
adt_set_bit 198
adt_clr_bit 198
adt_cpl_bit 199
第十九章 附录 200
19.1. 例程索引 200
19.2. 例程附加接口与定义 201
19.3. EtherCAT CIA 对象组常用 6000h 分配 203

第一章 指令列表

基础控制类	
adt_initial	控制卡初始化
adt_close_card	控制卡关闭
adt_reset_card	控制卡复位
adt_soft_reboot	控制卡软件重启
adt_get_card_index	获取系统可用控制卡索引
adt_get_motion_error	获取当前控制卡运动库异常
版本信息类	
adt_get_lib_ver	获取当前控制卡 LIB 库版本号
adt_get_motion_ver	获取当前控制卡 MOTION 库版本号
adt_get_firmware_ver	获取当前控制卡固件版本号
总线从站控制类	
adt_ecat_download_cfg	下发总线启动配置文件
adt_ecat_load_flash_cfg	加载控制卡存储的总线配置参数
adt_ecat_stop	终止总线通信
adt_ecat_start	启动总线通讯
adt_get_ecat_status	获取 Ethercat 总线运行状态
adt_get_ecat_slave_resource	获取 Ethercat 从站资源信息
adt_get_ecat_slave_info	获取 Ethercat 从站节点信息
adt_get_axis_ecat_node	获取轴号 Ethercat 从站节点映射
adt_get_total_axis	获取当前控制卡可用实体轴数
资源配置类	
adt_set_axis_io_map	自定义控制卡可用轴硬件信号映射
adt_get_axis_io_map	获取控制卡可用轴硬件信号映射
adt_set_hardlimit_mode	设置硬件信号输入模式
adt_get_hardlimit_mode	获取硬件信号输入模式
adt_set_emergency_stop	设置当前控制卡硬件停止信号模式
adt_get_emergency_stop	读取当前控制卡硬件停止信号模式
adt_set_limit_mode	设置控制卡当前轴硬件限位信号模式

adt_get_limit_mode	读取控制卡当前轴硬件限位信号模式
adt_set_stop0_mode	设置控制卡当前轴机械原点信号(STOP0)模式
adt_get_stop0_mode	读取控制卡当前轴机械原点信号(STOP0)模式
adt_set_unit_mode	设置控制卡当前轴编程模式
adt_get_unit_mode	获取控制卡当前轴编程模式
adt_set_pulse_equiv	设置控制卡当前轴脉冲当量
adt_get_pulse_equiv	获取控制卡当前轴脉冲当量
adt_set_bus_axis_gear_ratio	设置总线轴的电子齿轮比
adt_get_bus_axis_gear_ratio	获取总线轴的电子齿轮比
adt_set_softlimit_mode	设置控制卡当前轴软件限位模式
adt_get_softlimit_mode	获取控制卡当前轴软件限位模式
adt_set_pos_variable_loop	设置控制卡当前轴逻辑位置可变环功能
adt_get_pos_variable_loop	获取控制卡当前轴逻辑位置可变环功能
adt_set_encoder_scale	设置编码器反馈分频比
adt_set_axis_band	设置轴到位误差带
adt_set_axis_band	获取轴到位误差带
adt_set_axis_pos_error	设置轴最大跟随误差限制
adt_set_follow_axis	设置同步跟随轴(开环龙门双驱)
轴状态监视与控制	
adt_set_command_pos	设置指定轴逻辑位置
adt_get_command_pos	获取指定轴逻辑位置
adt_set_actual_pos	设置指定轴实际位置
adt_get_actual_pos	获取指定轴实际位置
adt_get_target_pos_unit	获取指定基于脉冲当量编程模式轴目标位置
adt_get_target_pos_pulse	获取指定基于脉冲编程模式轴目标位置
adt_set_axis_enable	使能伺服电机
adt_get_axis_enable	读取伺服电机使能状态
adt_clear_bus_axis_alarm	伺服报警清除
adt_get_axis_alarm_info	获取总线轴的报警状态
adt_get_axis_status	获取轴状态信息
adt_get_stopdata	获取轴异常停止信息

adt_get_alll_axis_move_status	获取所有轴运动规划状态
adt_get_alll_axis_reach_status	获取所有轴到位状态
adt_get _axis _status_ex	获取轴状态信息
adt_clear_axis _status	清除轴状态信息
adt_get_speed	获取当前轴的驱动速度
adt_get_encoder_speed	获取当前轴编码器反馈速度
速度规划类	
adt_set_startv	设置控制卡当前轴驱动起始速度
adt_get_startv	获取控制卡当前轴驱动起始速度
adt_set_maxv	设置控制卡当前轴驱动最大速度
adt_get_maxv	获取控制卡当前轴驱动最大速度
adt_set_acc	设置控制卡当前轴驱动加速度
adt_get_acc	获取控制卡当前轴驱动加速度
adt_set_dec	设置控制卡当前轴驱动减速度
adt_get_dec	获取控制卡当前轴驱动减速度
adt_set_admode	设置控制卡当前轴驱动加减速模式
adt_get_admode	获取控制卡当前轴驱动加减速模式
adt_set_axis_move_para_unit	集成设置轴运动参数
adt_set_rate	设置控制卡当前轴驱动速度倍率
adt_get_rate	获取控制卡当前轴驱动速度倍率
adt_set_corner_speed_smooth_level	设置控制卡当前轴拐角加速度平滑等级
adt_set_arc_speed_clamp_unit	设置插补轴拐角圆弧速度约束
adt_set_speed_pretreat_mode	设置插补轴速度前瞻功能模式
点位运动	
adt_pmove_unit	基于脉冲当量编程模式的单轴定量驱动
adt_pmove_pulse	基于脉冲编程模式的单轴定量驱动
adt_continue_move	单轴持续驱动
adt_change_pmove_pos	运动中改变点位运动目标位置
adt_set_axis_stop	设置任意轴停止
插补运动	
adt_set_inp_coordinate	创建插补坐标系

adt_set_inp_stop	设置插补组停止
adt_get_inp_pause_pos	获取插补坐标系各轴暂停位置
adt_get_inp_fifo_len	获取插补缓存余量
adt_get_inp_index	获取当前插补运动的索引信息
adt_inp_move_unit	基于脉冲当量编程模式的可缓存式多轴直线插补
adt_inp_move_pulse	基于脉冲编程模式的可缓存式多轴直线插补
adt_inp_arc2_unit	基于脉冲当量编程模式的可缓存式平面圆弧插补
adt_inp_arc3_unit	基于脉冲当量编程模式的可缓存式球面圆弧插补
adt_inp_helix2_unit	任意两轴圆弧插补，二轴跟随
adt_inp_helix3_unit	任意三轴圆弧插补，二轴跟随
adt_nurbs_curve	2D/3D 样条(NURBS)曲线插补驱动
PT	
adt_set_pt_mode	设置 PT FIFO 模式
adt_set_pt_data	发送 PT 数据
adt_set_pt_table	发送 PT 数据列表
adt_clr_pt_data	清空 PT 缓存
adt_get_pt_space	获取 PT 缓冲区剩余段数
adt_get_pt_index	获取当前执行 PT 数据段索引
adt_pt_start	启动 PT 运动
adt_set_pt_loop	设置循环运行(只有 PT 静态模式有效)
adt_get_pt_loop	获取循环运行执行次数
回零控制类	
adt_set_home_mode	DSP 回零模式设置
adt_set_home_speed	DSP 回零速度设置
adt_set_home_process	DSP 驱动回零
adt_get_home_status	DSP 回零状态查询
adt_set_servo_home_para	伺服回零参数设置
adt_set_servo_home	伺服回零启动
adt_get_servo_home_status	伺服回零状态获取
adt_stop_servo_home	伺服回零停止
总线 IO 控制类	

adt_read_servo_inbit	读取总线伺服输入端口状态
adt_read_servo_inport	按组读取总线伺服输入端口状态
adt_write_servo_outbit	写总线伺服 IO 输出点
adt_read_servo_outbit	读总线伺服 IO 输出点
adt_read_busio_inbit	按节点号读取总线 IO 输入点状态
adt_read_busio_inport	按组读取总线 IO 输入点状态
adt_write_busio_outbit	按节点号操作总线 IO 板输出点
adt_write_busio_outport	按组操作总线 IO 板输出点
adt_read_busio_outbit	按节点号读取总线 IO 板输出点
adt_read_busio_outport	按组读取总线 IO 板输出点
缓存事件控制类	
adt_get_fifo_event_len	获取缓存事件的剩余段数
adt_get_fifo_event_index	获取最新触发缓存事件的索引号
adt_set_fifo_adjust_time	设置提前滞后触发时间
adt_set_fifo_adjust_pos	设置提前滞后触发位置
adt_set_fifo_outbit	设置缓存输出事件(基于脉冲当量编程)
adt_set_fifo_multi_io	设置缓存输出事件,同时设置多个输出点电平
adt_set_fifo_delay	设置缓存运动暂停事件
adt_set_fifo_wait_inbit	设置插补缓存运动暂停等待输入信号
adt_clear_fifo_event	清除未执行完毕的缓存事件
从站仿真控制类	
adt_set_axis_resource	设置规划轴资源数
adt_get_axis_resource	获取规划轴资源数
adt_set_axis_bound	设置规划轴资源绑定输出通道
adt_get_axis_bound	获取规划轴资源绑定输出通道
adt_reset_axis_bound	重置规划轴资源绑定输出通道
adt_set_virtual_io_num	配置虚拟 IO 资源通道
adt_get_virtual_io_num	获取虚拟 IO 资源通道
adt_write_busio_inbit	按节点号控制总线 IO 输入点状态
adt_write_busio_inport	按组控制总线 IO 输入点状态
SDO/PDO	

adt_ecat_sdo_read	Ethercat SDO 数据读操作
adt_ecat_sdo_write	Ethercat SDO 数据写操作
adt_ecat_pdo_read	Ethercat 过程数据读
adt_ecat_pdo_write	Ethercat 过程数据写
用户控制类	
adt_set_user_password	修改/设置用户密码
adt_check_password	校验用户密码
ADT-9188 扩展板控制	
adt_write_outbit	写 9188 本地输出点
adt_read_outbit	读 9188 本地输出点
adt_read_inport	按位读取 9188 本地输入点状态
adt_read_inbit	按组读取 9188 本地输入点状态
adt_set_input_filter	设置输入信号滤波时间
adt_get_compare_len	获取位置比较器余量
adt_get_compare_status	获取比较触发状态
adt_clear_compare_point	清除位置比较器的所有比较点,复位比较器
adt_reset_compare	关闭位置比较器并清空状态
adt_set_compare_xd_mode	设置位置比较器模式
adt_get_compare_xd_mode	获取位置比较器模式
adt_set_compare_xd_area	设定 2D/3D 比较区域范围
adt_add_compare_xd_point	添加位置比较位置点
adt_set_encoder_mode	设置编码器工作方式
adt_get_encoder_pos	获取编码器位置
adt_set_encoder_pos	设置编码器位置
辅助功能	
adt_get_bit	读取指定位的值
adt_set_bit	指定位写 1
adt_clr_bit	指定位写 0
adt_cpl_bit	指定位值取反

第二章 错误码

错误码是库函数使用执行情况的真实反馈，库函数的每个接口都会在执行结束后返回执行结果，用户需依据返回值判定接口调用是否成功，并执行相应的控制与处理措施。

2.1. 错误码

错误类型	返回值	含义
LIB 库错误	0	成功
	1	参数错误
	2	信号量创建错误
	3	圆弧轴选位错误
	4	圆弧不存在
	5	运动库响应超时
	6	圆弧上的点平行
	7	PCI 返回数据错误
	8	通信数据量异常
	9	Winlo 初始化失败
	10	PCI 桥存在故障
	11	创建互斥量失败
	12	打开互斥量失败
	13	使用多张卡时，卡号重复
	14	没有识别到控制卡，卡安装有误或者驱动安装失败
	15	通信 CRC 校验失败
程序更新类错误	20	PCI 驱动重启失败，需要关机重启
	21	程序更新失败
	22	超时
	23	DSP 程序校验不通过
	24	固件程序版本和 dll 不匹配,或者固件异常,无法识别,需要送厂升级,或使用老版本 dll
圆弧过渡错误码	30	非相连曲线段(圆弧过渡处理)
	31	过渡精度或者参数阈值要求过高(圆弧过渡处理)

	32	存在一条圆弧段时输入曲线不共面(圆弧过渡处理)
	33	输入曲线夹角超出设定阈值范围(圆弧过渡处理)
	34	输入曲线长度太小(圆弧过渡处理)
	35	过渡后，原线段的长度小于 min_len(圆弧过渡处理)
	36	过渡后，圆半径过大或者圆半径小于 min_R(圆弧过渡处理)
	37	过渡后，过渡圆弧长小于 min_len(圆弧过渡处理)
	38	求解过渡圆过程中无解(圆弧过渡处理)
	39	存在两条圆弧段时输入曲线不共面(圆弧过渡处理)
SDO 回零类错误	40	总线 SDO 回零正在进行
	41	总线 SDO 回零超时或回零偏差过大
NURBS 插补类错误	50	样条数据过多或过少(>1000 或<2)
	51	样条插补模式错误
	52	样条数据大小有误
	53	样条导矢模过小
	54	样条步长过小
	55	节点小于 0
EtherCAT 错误	60	ECAT RunTime 响应错误
	61	ECAT 配置文件路径错误
	62	ECAT 配置文件读取错误
	63	ECAT 加载 FLASH 中配置文件失败
	64	ECAT 下载配置文件失败
	65	ECAT 主站启动超时
	66	ECAT 主站启动过程中强制终止
	67	ECAT 伺服操作模式切换失败
EtherCAT 总线 邮箱操作错误码	4096	读写超时
	4097	命令无效或不存在
	4098	对象字典不支持访问
	4099	对象字典不存在
	4100	子索引不存在
	4101	内存不足或溢出

	4102	硬件错误访问失败
	4103	设备参数不兼容
	4104	该地址不可读，只能写操作
	4105	该地址不可写，只能读操作
	4112	对象数据长度不匹配
	4113	超出参数的取值范围
	4114	由于设备原因，导致数据保存失败
	4115	数据对象不能映射到 PDO
	4116	分段传输时翻转位无变化
	4117	SDO 操作未知错误
MOTION 库错误	**101**	参数错误
	102	通信数据量异常
	103	对应轴处于限位停止状态
	104	运动冲突
	105	当前状态不允许修改参数
	106	输入数据模式错误
	107	加减速模式设置错误
	108	S 型速度规划时计算错误
	109	外部紧急停止信号生效中
	110	底层运动目标位置数据异常
	111	空指令或无效指令
	112	FPGA 读写错误
	113	插补运动指令缓冲区已满
	114	点位运动指令缓冲区已满
	115	恢复倍率时存在位置偏差
	116	当前固件不支持
	117	轴类型错误
	118	总线通信故障
	119	轴报警
	120	邮箱命令通信错误
	121	数据发送 CRC 校验错误

总线伺服回零类错误	**-1**	回零命令错误
	-2	回零过程出错或者回零超时
	-3	回零伺服报警
	-4	回零异常终止
	-5	回零未启动

2.2. 错误码解析示例

该辅助算子旨在将标准算子或扩展算子执行时的非正常返回解析为可读字符串错误信息，即错误码解析。用户参考之可自主设计错误码解析函数。

```
void DecodeErrCode(char* msg, int len, int retn, const char* itf)
{
    if(NULL==msg || 0>=len)
        return ;
    ZeroMemory(msg, len);
    sprintf(msg, "算子: %s\n 错误码: %d\n 错误描述: ", itf, retn);
    switch(retn)
    {
    case 0:strcat(msg, "执行成功!");break;
    case 1:strcat(msg, "参数错误!");break;
    case 2:strcat(msg, "信号量创建错误!");break;
    case 3:strcat(msg, "圆弧轴选位错误!");break;
    case 4:strcat(msg, "圆弧不存在!");break;
    case 5:strcat(msg, "运动库响应超时!");break;
    case 6:strcat(msg, "圆弧上的点平行!");break;
    case 7:strcat(msg, "PCI 返回数据错误!");break;
    case 8:strcat(msg, "通讯数据量异常!");break;
    case 9:strcat(msg, "WinIo 初始化失败!");break;
    case 10:strcat(msg, "PCI 桥存在故障!");break;
    case 11:strcat(msg, "创建互斥量失败!");break;
    case 12:strcat(msg, "打开互斥量失败!");break;
    case 13:strcat(msg, "使用多张卡时，卡号重复!");break;
    case 14:strcat(msg, "没有识别到控制卡，卡安装有误或者驱动安装失败!");break;
    case 15:strcat(msg, "通信 CRC 校验失败!");break;
```

```
    case 20:strcat(msg, "PCI 驱动重启失败，需要关机重启!");break;
    case 21:strcat(msg, "程序更新失败!");break;
    case 22:strcat(msg, "超时!");break;
    case 23:strcat(msg, "DSP 程序校验不通过!");break;
    case 24:strcat(msg, "固件程序版本和 dll 不匹配,或者固件异常,无法识别,需要送厂升级,或使用老版本 dll!");break;
    case 30:strcat(msg, "非相连曲线段(圆弧过渡处理)!");break;
    case 31:strcat(msg, "过渡精度或者参数阈值要求过高(圆弧过渡处理)!");break;
    case 32:strcat(msg, "存在一条圆弧段时输入曲线不共面(圆弧过渡处理)!");break;
    case 33:strcat(msg, "输入曲线夹角超出设定阈值范围(圆弧过渡处理)!");break;
    case 34:strcat(msg, "输入曲线长度太小(圆弧过渡处理)!");break;
    case 35:strcat(msg, "过渡后，原线段的长度小于 min_len(圆弧过渡处理)!");break;
    case 36:strcat(msg, "过渡后，圆半径过大或者圆半径小于 min_R(圆弧过渡处理)!");break;
    case 37:strcat(msg, "过渡后，过渡圆弧长小于 min_len(圆弧过渡处理)!");break;
    case 38:strcat(msg, "求解过渡圆过程中无解(圆弧过渡处理)!");break;
    case 39:strcat(msg, "存在两条圆弧段时输入曲线不共面(圆弧过渡处理)!");break;
    //伺服回零错误码
    case 40:strcat(msg, "总线 SDO 回零进行中!");break;
    case 41:strcat(msg, "总线 SDO 回零超时或偏差过大!");break;
    case 50:strcat(msg, "样条数据过多或过少(>1000 或<2)!");break;
    case 51:strcat(msg, "样条插补模式错误!");break;
    case 52:strcat(msg, "样条数据大小有误!");break;
    case 53:strcat(msg, "样条导矢模过小!");break;
    case 54:strcat(msg, "样条步长过小!");break;
    case 55:strcat(msg, "节点数小于!");break;
    //ETHERCAT 错误
    case 60:strcat(msg, "ECAT RunTime 响应错误!");break;
    case 61:strcat(msg, "ECAT 配置文件路径错误!");break;
    case 62:strcat(msg, "ECAT 配置文件读取错误!");break;
    case 63:strcat(msg, "ECAT 加载 FLASH 中配置文件失败!");break;
    case 64:strcat(msg, "ECAT 下载配置文件失败!");break;
```

```
case 65:strcat(msg, "ECAT 主站启动超时!");break;
case 66:strcat(msg, "ECAT 主站启动过程中强制终止!");break;
case 67:strcat(msg, "ECAT 伺服操作模式切换失败!");break;
case 101:strcat(msg, "参数错误!");break;
case 102:strcat(msg, "通信数据量异常!");break;
case 103:strcat(msg, "对应轴处于限位停止状态!");break;
case 104:strcat(msg, "运动冲突!");break;
case 105:strcat(msg, "当前状态不允许修改参数!");break;
case 106:strcat(msg, "输入数据模式错误!");break;
case 107:strcat(msg, "加减速模式设置错误!");break;
case 108:strcat(msg, "S 型速度规划时计算错误!");break;
case 109:strcat(msg, "外部紧急停止信号生效中!");break;
case 110:strcat(msg, "底层运动目标位置数据异常!");break;
case 111:strcat(msg, "空指令或无效指令!");break;
case 112:strcat(msg, "FPGA 读写错误!");break;
case 113:strcat(msg, "插补运动指令缓冲区已满!");break;
case 114:strcat(msg, "点位运动指令缓冲区已满!");break;
case 115:strcat(msg, "恢复倍率时存在位置偏差!");break;
case 116:strcat(msg, "当前固件不支持!");break;
case 117:strcat(msg, "轴类型错误!");break;
case 118:strcat(msg, "总线通信故障!");break;
case 119:strcat(msg, "轴报警!");break;
case 120:strcat(msg, "邮箱命令通信错误!");break;
case 121:strcat(msg, "数据发送 CRC 校验错误!");break;
//ECAT 总线 邮箱操作错误码
case 0x1000:strcat(msg, "读写超时");break;
case 0x1001:strcat(msg, "命令无效或不存在");break;
case 0x1002:strcat(msg, "对象字典不支持访问");break;
case 0x1003:strcat(msg, "对象字典不存在");break;
case 0x1004:strcat(msg, "子索引不存在");break;
case 0x1005:strcat(msg, "内存不足或溢出");break;
case 0x1006:strcat(msg, "硬件错误访问失败");break;
```

```
    case 0x1007:strcat(msg, "设备参数不兼容");break;
    case 0x1008:strcat(msg, "该地址不可读，只能写操作");break;
    case 0x1009:strcat(msg, "该地址不可写，只能读操作");break;
    case 0x1010:strcat(msg, "对象数据长度不匹配");break;
    case 0x1011:strcat(msg, "超出参数的取值范围");break;
    case 0x1012:strcat(msg, "由于设备原因，导致数据保存失败");break;
    case 0x1013:strcat(msg, "数据对象不能映射到 PDO");break;
    case 0x1014:strcat(msg, "分段传输时翻转位无变化");break;
    case 0x1015:strcat(msg, "SDO 操作未知错误");break;

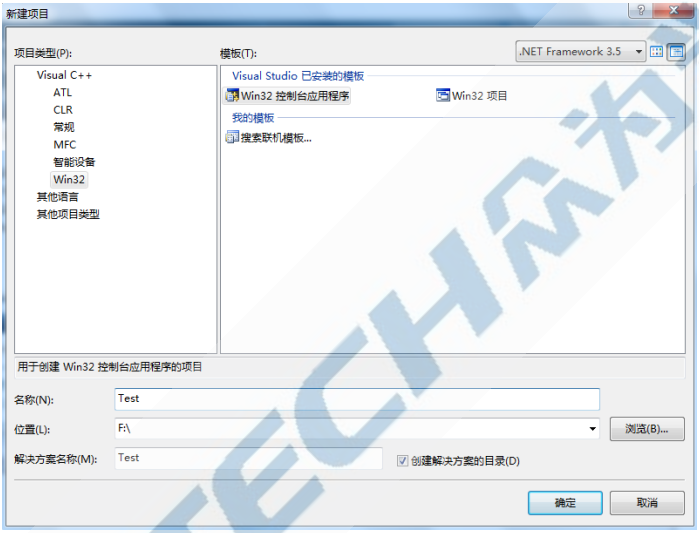
    //归零类错误
    case -1:strcat(msg, "回零命令错误!");break;
    case -2:strcat(msg, "回零过程出错或者回零超时!");break;
    case -3:strcat(msg, "回零伺服报警!");break;
    case -4:strcat(msg, "回零异常终止!");break;
    case -5:strcat(msg, "回零未启动!");break;
    default:strcat(msg, "内部错误!");break;
    }
    strcat(msg, "\n");
}
```

第三章 SDK 调用方法

3.1. VC 调用方法

开发平台以 WIN7 x86 操作系统下 VS2008 为例，其余操作系统下的其他平台操作类似，不再赘述。

1. 文件-->新建-->项目，以 Win32 控制台应用程序为例，其他项目类型类似，不再赘述



2. 将开发包内 adt_typedef.h、adtemc.h、adtemc.lib 文件拷贝入新建的项目文件路径文件夹内，若您在开发 32 位应用程序，则选择 x86 开发包内的文件拷贝；若您在开发 64 位应用程序，则选择 x64 开发包内的文件拷贝

3. 选择您的解决方案，右键-->添加-->现有项，将拷贝于项目路径下的开发包文件加入解决方案

4. 在相应程序源文件、头文件或全局头文件"stdafx.h"的声明部分补充 #include "adtemc.h" #include "adt_typedef.h"

5. 使用动态链接库提供的控制接口，实现您的控制程序，如下方程序实现简单的初始化

```
// Test.cpp : 定义控制台应用程序的入口点。
//

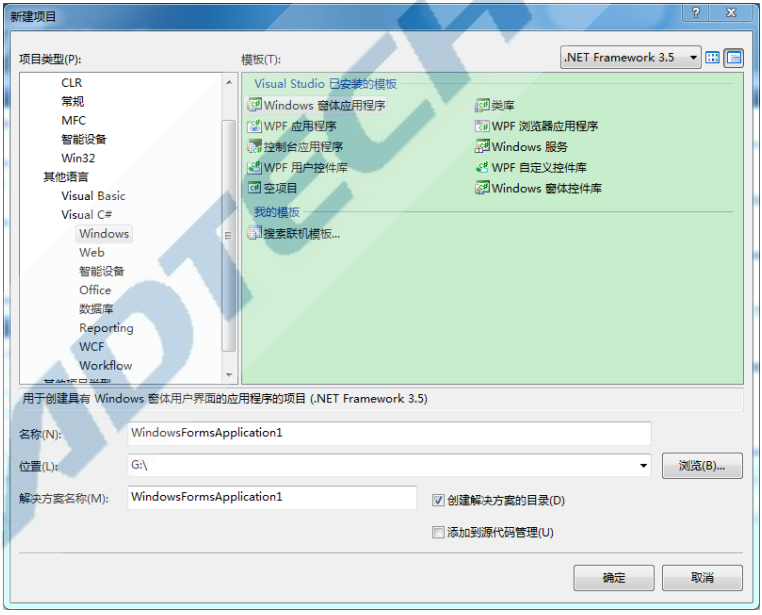
#include "stdafx.h"
#include <windows.h>

int _tmain(int argc, _TCHAR* argv[])
{
    int crd_count = 0;
    int retn = adt_initial(&crd_count, 0);//初始化运动控制卡
    if(0 != retn)
        printf("Initial failed or none card!\n");//初始化失败或系统未安装运动控制卡
    else
        printf("%d ADT-6320E found!\n", crd_count);//初始化成功并打印控制卡数量
    system("pause");
    return 0;
}
```

3.2. C# 调用方法

开发平台以 WIN7 x86 操作系统下 VS2008 为例，其余操作系统下的其他平台操作类似，不再赘述。

1. 文件-->新建-->项目，以 Windows 窗体应用程序为例，其他项目类型类似，不再赘述

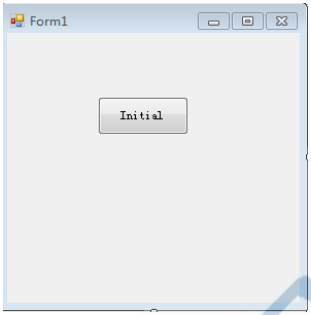


2. 我们为用户提供了 C#所需开发包标准文件 adt_typedef.cs 和 adtemc.cs，将其拷贝入新建的项目文件路径下。我们同时提供了 adt_global.cs 辅助，内含部分全局常量定义和错误码解析例程，用以辅助用

户编程。用户可以有更好的编程辅助方案，adt_global.cs 为可选。

3. 选择您的解决方案，右键-->添加-->现有项，将开发包文件添加至解决方案

4. 在 Form 上添加按钮



在 Form1.cs 添加命名空间

```
using System;
using System.Collections.Generic;
using System.ComponentModel;
using System.Data;
using System.Drawing;
using System.Linq;
using System.Text;
using System.Windows.Forms;
using ADT_CARD_632XE;
```

按钮响应事件代码可编辑如下，实现简易的初始化

```
namespace WindowsFormsApplication1
{
    public partial class Form1 : Form
    {
        public Form1()
        {
            InitializeComponent();
        }

        private void button1_Click(object sender, EventArgs e)
        {
            Int32 crd_count = 0;
            Int32 retn = adt_card_6320e.adt_initial(out crd_count, 0);
            if (0 != retn)
            {
                MessageBox.Show("控制卡初始化失败或系统未安装运动控制卡!");
                return;
            }
            MessageBox.Show("控制卡初始化成功, 系统中有" + crd_count.ToString() + "张ADT-6320E运动控制卡!");
        }
    }
}
```

第四章 基础控制与版本信息

4.1. 功能介绍

基础控制，即控制卡在开发前需要做的准备工作以及部分基本信息的获取。

4.1.1. 控制卡初始化

控制卡初始化是通往控制卡功能的“门户”，只有在对运动控制卡初始化成功后，调用其他算子才有意义。

控制卡初始化算子参考 adt_initial。

4.1.2. 控制卡关闭

控制卡使用完成后，应关闭通讯并合理释放控制卡资源。

控制卡关闭算子参考 adt_close_card。

4.1.3. 控制卡复位

复位控制卡常用于清除硬件急停信号、清除异常停止信息、清除无效缓存数据、清除运动库停止信息。

当硬件急停功能启用，且硬件急停信号触发时，须复位控制卡清除硬件急停锁定后，轴才能正常驱动。

当系统发生运动库异常(adt_get_motion_error)，问题已排查且解决后，须复位控制卡清除运动库异常信息，系统才能正常驱动。

复位控制卡会停止当前正在执行的驱动控制，清除停止信息，若是缓存控制，则停止驱动同时清除剩余未执行的缓存数据。

复位控制卡不会重置用户已执行的脉冲模式、限位模式等基础配置参数，也不会重置已设置成功的轴速度参数及轴当前位置。

复位控制卡不会导致从站配置信息的丢失。

控制卡复位算子参考 adt_reset_card。

4.1.4. 控制卡软件重启

软件重启控制卡将首先减速停止当前控制卡的所有驱动并清空缓存，然后复位控制卡所有数据为上电初始状态。

软件重启控制卡并不会使控制卡断电。

软件重启控制卡后，须再次初始化，才能继续正常使用控制卡。

控制卡软件重启算子参考 adt_soft_reboot。

4.1.5. 可用控制卡索引获取

控制卡索引即控制卡编号，是 SDK 算子中的卡号参数值。

运动控制卡均采用拨码开关的方式用以区分多卡系统的卡号索引。

系统安装控制卡时，可以随意安排安装顺序，只需拨码开关对应正确即可。拨码开关值范围 0~9，不同控制卡其拨码值不可重复。

可用控制卡索引获取算子参考 adt_get_card_index。

4.1.6. 控制卡版本信息获取

版本信息是控制卡升级、更新和维护的有效标识。

运动控制卡的版本信息包括 LIB 库版本信息、MOTION 库版本信息、固件版本信息。

算子参考

adt_get_lib_ver

adt_get_motion_ver

adt_get_firmware_ver

通常，版本信息格式为“LIB 库版本信息.MOTION 库版本信息.固件版本信息”，如



表示为，LIB 库版本 6003，MOTION 库版本 8060，固件版本 6019。

4.2. 例程 4-1 基础控制与版本信息获取

```
int _tmain(int argc, _TCHAR* argv[])
{
    int card_count = 0;//系统控制卡数量
    int index[10] = {0};//系统可用控制卡索引
    int axis_count = 0;//控制卡可用轴数
    //初始化控制卡
    VERIFY_RETURN_MSG (adt_initial (&card_count), "adt_initial", 1);
    printf("控制卡初始化完成!\n");
    //获取可用控制卡索引
    VERIFY_RETURN_MSG(adt_get_card_index (&card_count, index), "adt_get_card_index"
, 1);
```

```
    printf("共检测到 %d 张运动控制卡!\n", card_count);
    //获取可用控制卡版本号
    int lib = 0, motion = 0, fmw = 0;
    for (int i=0; i <card_count; ++i)
    {
        VERIFY_RETURN_MSG(adt_get_lib_ver(index[i], &lib), "adt_get_lib_ver", 1);
        VERIFY_RETURN_MSG(adt_get_motion_ver(index[i], &motion), "adt_get_motion_ve
r", 1);
        VERIFY_RETURN_MSG(adt_get_firmware_ver(index[i], &fmw), "adt_get_firmware_v
er", 1);
        printf("DIP-%d Version:%d, %d, %d\n", index[i], lib, motion, fmw);
    }
    //关闭运动控制卡
    VERIFY_RETURN_MSG(adt_close_card(), "adt_close_card", 1);
    system("pause");
    return 0;
}
```

第五章 从站配置

5.1. 功能介绍

5.1.1. 从站配置

ADT-632XE 系列总线运动控制卡采用 CIA402 标准协议通讯实现主站控制，市面上遵循该标准协议的总线伺服驱动器、总线 IO 模块或其他总线类控制设备均可作为控制卡从站使用。

我们提供 3 种配置方案选择。

- 下载配置参数到 RAM

  配置参数保存在 RAM 里，掉电不保存。通常适用于调试。

- 下载配置参数到 ROM

  配置参数保存在 ROM 里，掉电保存，适用于外设常年不变更的大多数情况。

- 导出离线配置文件

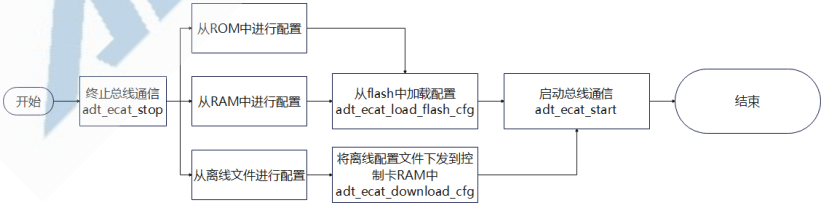
  将配置文件保留在本地，应用程序通过指定路径加载配置。外设发生变化时，也可通过修改配置文件完成自动适配。

上述配置方案通过 AdtWinCat 总线配置工具完成，操作步骤可参考《运动控制卡使用手册.pdf》。下载配置参数到 RAM 或 ROM 需使用接口 adt_ecat_load_flash_cfg。导出的离线配置文件需要在应用程序中使用 adt_ecat_download_cfg 指定路径导入。

5.1.2. 从站通信

从站配置加载完成后，需要启动通信 adt_ecat_start。如遇特殊情况需要暂时停止通信或重启通信，需要调用 adt_ecat_stop。

从站通讯流程如下所示：



5.1.3. 主从站信息获取

从站通信正常建立之后，我们通常需要查询主站和从站的状态信息。

接口 adt_get_ecat_slave_resource 用于获取从站总结点数，伺服节点数量、非伺服节点数量，进而通过 adt_get_ecat_slave_info 获取当前从站的资源数量，如果是伺服资源，则获取该节点有几个轴(双驱伺服、四驱伺服可能有 1 个节点多个轴的情况)，如果是 IO 模块资源，则获取该节点 IO 组数(每组 8 位)。

5.1.4. 从站资源获取

我们也可以通过接口 adt_get_axis_ecat_node 查询轴所在节点号及通道号。通过 adt_get_total_axis 获取从站的轴总数，然后进行相关的使能、清除报警操作。

小提示	清除报警后，轴将处于未使能状态。 本章节之后的例程将都以从 ROM 加载从站配置的方式执行，默认 8 台总线伺服，1 台总线 IO 模块。

5.2. 例程 5-1 从 RAM/ROM 中加载从站配置，解析配置，获取轴资源信息

```
int _tmain(int argc, _TCHAR* argv[])
{
    int card_count = 0;//系统控制卡数量
    int index[10] = {0};//系统可用控制卡索引
    int axis_count = 0;//控制卡可用轴数
    //初始化控制卡
    VERIFY_RETURN_MSG (adt_initial (&card_count), "adt_initial", 1);
    printf("控制卡初始化完成!\n");
    //获取可用控制卡索引与可用轴数
    VERIFY_RETURN_MSG(adt_get_card_index (&card_count, index), "adt_get_card_index",
1);
    printf("共检测到 %d 张运动控制卡!\n", card_count);
    int card = index[0];
    //停止通讯，防止后台有通讯占用
    VERIFY_RETURN_MSG(adt_ecat_stop(card), "adt_ecat_stop", 1);
     //加载配置
```

```
    VERIFY_RETURN_MSG(adt_ecat_load_flash_cfg(card), "adt_ecat_load_flash_cfg", 1);
     //开启通讯
    VERIFY_RETURN_MSG(adt_ecat_start(card), "adt_ecat_start", 1);
     //获取从站资源信息，从站总数，伺服总数，IO 总数
    int slv_count = 0, srv_count = 0, io_count = 0;
     VERIFY_RETURN_MSG(adt_get_ecat_slave_resource(card, &slv_count, &srv_count, &io_
count), "adt_get_ecat_slave_resource", 1);
     printf("当前从站数量:%d，其中伺服从站数量:%d，非伺服从站数
量:%d \n", slv_count, srv_count, io_count);
     //获取每个从站的资源信息
    int type = 0, axs_count = 0, ig = 0, og = 0;
    for(int i=0; i<slv_count; ++i)
    {
        VERIFY_RETURN_MSG(adt_get_ecat_slave_info(card, i+1, &type, &axs_count, &ig,
&og), "adt_get_ecat_slave_info", 1);
        printf("从站%d: ", i+1);
        switch(type)
        {
        case 0:
            printf("空节点\n");
            break;
        case 1:
            printf("伺服节点，%d 轴伺服\n", axis_count);
            break;
        case 2:
            printf("非伺服节点，输入端口数:%d * 8，输出端口数:% * 8\n", ig, og);
            break;
        default:
            break;
        }
     }
    //断开通讯，关闭控制卡
```

```
    VERIFY_RETURN_MSG(adt_ecat_stop(card), "adt_ecat_stop", 1);
    VERIFY_RETURN_MSG(adt_close_card(), "adt_close_card", 1);
    system("pause");
    return 0;
}
```

5.3. 例程 5-2 从本地文件导入从站配置

```
int _tmain(int argc, _TCHAR* argv[])
{
    int card_count = 0;//系统控制卡数量
    int index[10] = {0};//系统可用控制卡索引
    int axis_count = 0;//控制卡可用轴数
    //初始化控制卡
    VERIFY_RETURN_MSG (adt_initial (&card_count), "adt_initial", 1);
    printf("控制卡初始化完成!\n");
    //获取可用控制卡索引与可用轴数
    VERIFY_RETURN_MSG(adt_get_card_index (&card_count, index), "adt_get_card_index", 1
);
    printf("共检测到 %d 张运动控制卡!\n", card_count);
    int card = index[0];
    //停止通讯，防止后台有通讯占用
    VERIFY_RETURN_MSG(adt_ecat_stop(card), "adt_ecat_stop", 1);
    //从本地文件加载配置
    VERIFY_RETURN_MSG(adt_ecat_download_cfg(card, "C:\\eCatMbPara.cfg"), "adt_ecat_dow
nload_cfg", 1);
     //开启通讯
    VERIFY_RETURN_MSG(adt_ecat_start(card), "adt_ecat_start", 1);
     //获取从站资源信息，从站总数，伺服总数，IO 总数
    int slv_count = 0, srv_count = 0, io_count = 0;
     VERIFY_RETURN_MSG(adt_get_ecat_slave_resource(card, &slv_count, &srv_count, &io_c
ount), "adt_get_ecat_slave_resource ", 1);
```

```
    printf("当前从站数量:%d, 其中伺服从站数量:%d, 非伺服从站数量:%d \n", slv_count, srv_count, io_count);
    //获取每个从站的资源信息
    int type = 0, axs_count = 0, ig = 0, og = 0;
    for(int i=0; i<slv_count; ++i)
    {
        VERIFY_RETURN_MSG(adt_get_ecat_slave_info(card, i+1, &type, &axs_count, &ig, &og), "adt_get_ecat_slave_info", 1);
        printf("从站%d: ", i+1);
        switch(type)
        {
        case 0:
            printf("空节点\n");
            break;
        case 1:
            printf("伺服节点, %d 轴伺服\n", axis_count);
            break;
        case 2:
            printf("非伺服节点, 输入端口数:%d * 8, 输出端口数:% * 8\n", ig, og);
            break;
        default:
            break;
        }
    }
    //断开通讯, 关闭控制卡
    VERIFY_RETURN_MSG(adt_ecat_stop (card), "adt_ecat_stop", 1);
    VERIFY_RETURN_MSG (adt_close_card(), "adt_close_card", 1);
    system("pause");
    return 0;
}
```

第六章 资源配置

6.1. 功能介绍

6.1.1. 总线 IO 端口映射

总线控制卡不具备配套接线板或接线盒，不具备固定引脚定义，所以总线控制卡的硬件信号若要正常使用，首先需要用户配置 IO 映射。

硬件限位信号(LMT+/LMT-)和机械原点信号(STOP0)的 IO 映射端口可以是总线伺服驱动器自带的输入端口，也可以是总线 IO 从站的远程输入端口。

硬件停止信号(EMGN)的 IO 端口须映射至总线 IO 从站的远程输入端口。

可参考接口 adt_set_axis_io_map

小提示	硬件限位信号(LMT+/LMT-)和机械原点信号(STOP0)的 IO 映射默认均指向当前总线伺服驱动器的 0 号输入端口。硬件停止信号(EMGN)的 IO 映射默认指向第 1 块 IO 从站的 0 号输入端口。
小提示	所有硬件信号的有无效、有效电平、加减速模式的设置，必须放在 IO 映射完成之后，否则资源可能会发生映射错乱或有效电平不符

6.1.2. 硬件停止信号(EMGN)

硬件停止信号(EMGN)为设备提供了设备整机紧急停止的信号检测功能，一旦硬件停止信号触发，控制卡将立即停止当前所有驱动与控制指令的执行，清空缓存信息，并不再下发任何驱动与控制指令，直至硬件停止信号恢复至未触发状态，并复位运动控制卡。

硬件停止信号可设置为无效，当硬件停止信号无效时，其端子可作为通用输入使用。

硬件停止信号模式的设置接口可参照 adt_set_hardlimit_mode。

小提示	硬件停止信号(EMGN)默认无效。

6.1.3. 硬件限位信号(LMT+/LMT-)

硬件限位信号(LMT+/LMT-)为设备的每个轴提供了驱动的硬件限位安全机制。当指定驱动方向上的硬件限位信号触发时，当前轴驱动将立即停止，清空缓存信息，不再接受相同方向的驱动指令，并在停止信息中有所体现。例如，当正方向驱动 X 轴时，若触发硬件正限位，则 X 轴立即停止并清空缓存信息，并拒绝正方向的一切驱动指令，负方向则可正常驱动，；当负方向驱动 X 轴时，若触发硬件负限位，则 X 轴立即停止并清空缓存信息，

并拒绝负方向的一切驱动指令，正方向则可正常驱动。

硬件正负限位信号可独立设置为有效或无效，无效时，对应的端子可作为通用输入使用。

硬件限位信号模式的设置接口可参照 adt_set_hardlimit_mode。

小提示	硬件限位信号(LMT+/LMT-) 默认有效电平为**低电平**，同时为**不启用**状态。 硬件限位信号的停止模式只有**立即停止**一种

6.1.4. 机械原点信号(STOP0)

硬件原点信号(STOP0)为设备的每个轴提供了机械原点信号。当驱动轴从任意方向触发当前轴的硬件原点信号时，当前轴将以设置的停止模式停止驱动，清空缓存信息，不再接受任何驱动指令。

正常情况下，若无特殊复用需求，无需启用硬件原点信号，控制卡默认禁用该信号。硬件原点信号在回零过程中将不断被使用。控制卡将在回零过程中自动实现其切换使用，无需用户自行控制。

硬件原点信号的停止模式可设置为立即停止或减速停止。

硬件原点信号可设置为无效(默认)，无效时，可做通用输入使用。

硬件原点信号的模式设置接口可参照 adt_set_hardlimit_mode。

小提示	机械原点信号(STOP0)默认**不启用**状态。

6.1.5. 编程模式

运动控制卡支持 2 种轴编程模式：基于脉冲当量的高级编程模式、基于脉冲的标准编程模式。

不同的编程模式下，下列接口速度或位置相关参数数据类型不同。

adt_set_startv/adt_get_startv

adt_set_maxv/adt_get_maxv

adt_set_acc/adt_get_acc

adt_set_dec/adt_get_dec

adt_set_softlimit_mode/adt_get_softlimit_mode

基于脉冲当量的高级编程模式下，其速度或位置相关参数数据类型为**双精度浮点型**,单位为 **unit**、**unit/s**、**unit/s²**。定量驱动接口应使用 adt_pmove_unit，直线插补驱动接口应使用 adt_inp_move_unit。

基于脉冲的标准编程模式下，其速度相关参数数据类型为**双精度浮点**

型，单位为 **pulse/s**、**pulse/s²**， 位置相关参数数据类型为**整型**，单位为 **pulse**。定量驱动接口应使用 adt_pmove_pulse，直线插补驱动接口应使用 adt_inp_move_pulse。

基于脉冲当量的高级编程模式的轴，可以实现诸如二维圆弧插补 adt_inp_arc2_unit、三维圆弧插补 adt_inp_arc3_unit、圆弧速度约束 adt_set_arc_speed_clamp_unit 等功能。

基于脉冲的标准编程模式下的轴，暂不支持诸如二维圆弧插补 adt_inp_arc2_unit、三维圆弧插补 adt_inp_arc3_unit、圆弧速度约束 adt_set_arc_speed_clamp_unit 等功能。

基于脉冲当量的高级编程模式的轴，需设置当前轴的**脉冲当量**，单位为 pulse/unit。默认情况下，轴的脉冲当量为 **1000** pulse/unit。

轴默认设置为**基于脉冲当量**的编程模式。通过 adt_set_unit_mode 设置成功的编程模式参数，可通过 adt_get_unit_mode 获取。

不同编程模式的轴不能参与同一组插补驱动。

小提示	在设置驱动速度参数之后改变编程模式，可能导致实际驱动速度的缩放，其缩放倍数为脉冲当量的值。为插补驱动接口分配不匹配的编程模式的插补轴，将导致驱动接口返回错误码 106，即输入数据模式错误。 默认情况下，所有轴编程模式均为基于脉冲当量的编程模式。

6.1.6. 电子齿轮比

电子齿轮比是对控制卡发送给从站系统的脉冲频率进行缩放。当从站无法进行电子齿轮比设置时，可以通过 adt_set_bus_axis_gear_ratio 进行电子齿轮比设置，

6.1.7. 脉冲当量

当轴的编程模式为**基于脉冲当量**的单位制时(mm 或角度)，该参数则决定了 1unit 对应需要的脉冲数。默认情况下，**1unit=1000pulse**，该参数依赖于外设的脉冲细分参数设置。须匹配的脉冲当量设置，才能确保驱动位置的精确。

处于脉冲编程模式下的轴，设置该参数不起作用。

默认情况下，基于脉冲当量的高级编程模式的轴，其脉冲当量设置接口参考 adt_set_pulse_equiv。设置成功的脉冲当量，可以通过 adt_get_pulse_equiv 获取。

小提示	1 unit 为 1 个单位量，对应 1mm 或 1°。 默认情况下，所有轴，包括实体轴和插补轴，其编程模式均为基于脉冲当量的编程模式，且脉冲当量为 1000pulse/unit。

6.1.8. 脉冲当量与电子齿轮的计算方法

想要运动控制的精度与速度达到理想状态，设置合理的电子齿轮与脉冲当量相当重要。

以下将以常用的机载结构为例来对脉冲当量与电子齿轮的值进行计算；

以下示例的伺服端的电子齿轮比均设置为 1：1；

A：编码器分辨率

B：电机旋转一圈机械移动量

I：最小指令单位（最小精度）

m/n：减速比

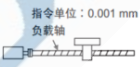
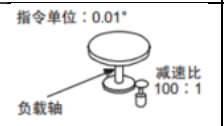
equiv：脉冲当量

E：电子齿轮比分子

P：电子齿轮比分母

$$equiv = \frac{1}{I}$$

$$\frac{E}{P} = \frac{A}{B \times equiv} \times \frac{m}{n}$$

内容	机械结构		
	滚珠丝杆	圆盘	皮带轮
			
A	131072	1048576	8388638
B	10.000mm	360.00°	100×3.14 mm
I	0.001mm	0.01°	0.005mm
m/n	1/1	100/1	50/1
equiv	$\frac{1}{0.001} = 1000$	$\frac{1}{0.01} = 100$	$\frac{1}{0.005} = 200$

E/P	$\frac{A}{B * \mathbf{equiv}} \times \frac{m}{n} = \frac{8192}{625}$	$\frac{A}{B * \mathbf{equiv}} \times \frac{m}{n} = \frac{65536}{225}$	$\frac{A}{B * \mathbf{equiv}} \times \frac{m}{n} = \frac{524288}{3925}$

6.1.9. 软件限位

运动控制卡为设备提供了除硬件限位信号安全机制外的软件限位安全机制。

软件限位将为设备每个轴设置安全的位置范围，当轴驱动位置超出设置的软件限位范围时，当前轴将以设置的停止模式停止驱动，清空缓存信息，限位同向的驱动将仍旧被停止，反向驱动则可正常执行。

正负软件限位可独立被设置为有效或无效。软件限位功能模式的设置可参照接口 adt_set_softlimit_mode。其设置信息获取接口可参照 adt_get_softlimit_mode。

小提示	软件限位功能默认不启用 软件限位功能启用后，不能将正，反方向限位位置设置为相等的数，正方向限位位置应大于反方向限位位置

6.1.10. 编码器反馈分频比

设置编码器反馈分频比的目的是将逻辑位置与实际位置实现 1：1 的关系，即逻辑位置的移动量与实际位置反馈的移动量相等。

对于一些不方便进行编码器反馈分频比进行设置的场合，可通过 adt_set_encoder_scale 设置分频比，此功能可将获取到的编码器位置进行一次比例运算。

设置编码器反馈分频比后，通过 adt_get_actual_pos 获取到的编码器位置则时计算后的编码器位置。

编码器反馈分频比 = 编码器实际值 / 需要得到的编码器值

如：编码器实际值为 **131072** 个脉冲一圈，需要实际得到的值是 **10000** 个脉冲一圈，则编码器分频比设置为 **131072 / 10000**。

6.1.11. 逻辑变环模式

当轴需要只向一个方向进行运行的情况时，如果一直运动下去，时间长了极可能出现数据溢出的情况，这时就需要使用逻辑变环功能，即当前轴逻辑位置到达指定数值后，将从 0 开始重新计数，该功能多用于传送带等单向驱动场合。

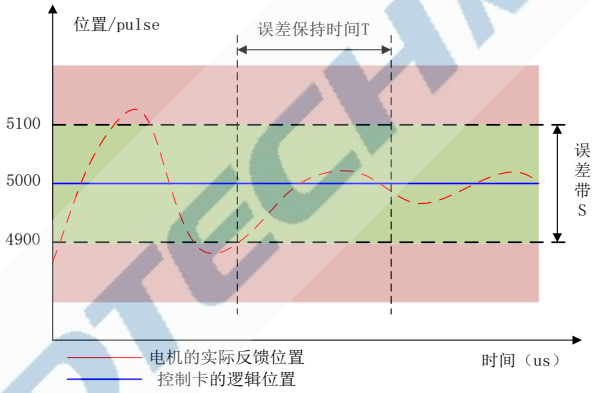
逻辑变环模式设置接口可参照 adt_set_pos_variable_loop。

小提示	默认情况下逻辑位置变环功能不启用。

6.1.12. 到位误差与跟随误差

在运动控制中存在这样的情况，当控制卡的运动规划停止时，电机实际位置还没有到位的情况，可以通过控制卡的到位检测功能来检测电机的到位状态。默认状态下到位检测功能不开启，通过 adt_set_axis_band 设置开启/关闭到位检测功能以及误差带大小和误差保持时间（如下图）。只有在误差保持时间内始终处于误差带内，电机才能到位，建议将误差带及误差保持时间设置在合理的时间内以确保效率。可以通过 adt_get_axis_band 获取到位检测功能的使能状态、误差大和误差保持时间的参数。

可以使用 adt_get_all_axis_reach_status 获取电机的到位状态。



跟随误差是某一时刻，电机的实际反馈位置与控制卡的逻辑位置之间的差值，由于从控制卡下发命令到电机运动存在一定延时，并且存在例如加速等情况，会产生电机位置滞后的情况。使用 adt_set_axis_pos_error 设置轴最大跟随误差，当伺服滞后脉冲数大于设置的限制值时，会根据设置的 mode 模式来限制轴的运动功能。

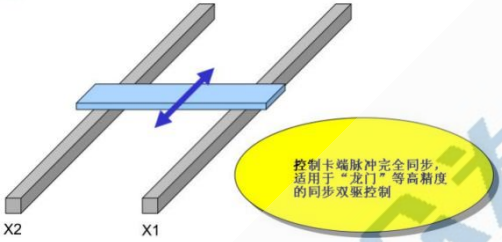
小提示	在使用此功能前需要保证逻辑位置与实际位置实现 1:1 的关系。可以通过设置编码器反馈分频比来实现逻辑位置与实际位置成 1：1 的关系

6.1.13. 龙门双驱

龙门双驱能保持 1 个主轴与 N 个从轴之间的脉冲严格同步，其同步精度远高于插补驱动的同步效果，适用于龙门结构的同步双驱控制。

龙门双驱控制前，应设置好从轴的各个基本参数，如编程模式、脉冲当量等，龙门双驱设置好主从轴关系后，从轴的一些基本参数将不能被实时更改。

龙门双驱控制接口可参照 adt_set_follow_axis。



6.2. 例程 6-1 总线 IO 端口映射

```
int _tmain(int argc, _TCHAR* argv[])
{
    int card_count = 0;//系统控制卡数量
    int index[10] = {0};//系统可用控制卡索引
    int axis_count = 0;//控制卡可用轴数
    //初始化控制卡
    VERIFY_RETURN_MSG (adt_initial(&card_count), "adt_initial", 1);
    printf("控制卡初始化完成!\n");
    //获取可用控制卡索引与可用轴数
    VERIFY_RETURN_MSG(adt_get_card_index (&card_count, index), "adt_get_card_index",
1);
    printf("共检测到 %d 张运动控制卡!\n", card_count);
    int card = index[0];
    //停止通讯，防止后台有通讯占用
    VERIFY_RETURN_MSG(adt_ecat_stop (card), "adt_ecat_stop", 1);
    //加载配置
    VERIFY_RETURN_MSG(adt_ecat_download_cfg(card), "adt_ecat_load_flash_cfg", 1)
    //开启通讯
    VERIFY_RETURN_MSG(adt_ecat_start (card), "adt_ecat_start", 1);
    //映射轴 1 的正限位信号到当前轴伺服的 IN0 上
    VERIFY_RETURN_MSG(adt_set_axis_io_map(card, 1, 0, 0, 0), "adt_set_axis_io_map", 1
);
    //映射轴 1 的负限位信号到站点 2 的 IN2 上
    VERIFY_RETURN_MSG(adt_set_axis_io_map(card, 1, 1, 2, 2), "adt_set_axis_io_map", 1
);
    //断开通讯，关闭控制卡
    VERIFY_RETURN_MSG(adt_ecat_stop (card), "adt_ecat_stop", 1);
    VERIFY_RETURN_MSG(adt_close_card (), "adt_close_card", 1);
    system("pause");
    return 0;
}
```

6.3. 例程 6-2 配置轴基本参数

```
int _tmain(int argc, _TCHAR* argv[])
{
    int card_count = 0;//系统控制卡数量
    int index[10] = {0};//系统可用控制卡索引
    int axis_count = 0;//控制卡可用轴数
    //控制卡初始化
    VERIFY_RETURN_MSG (adt_initial (&card_count), "adt_initial", 1);
    //获取可用控制卡索引与可用轴数
    VERIFY_RETURN_MSG(adt_get_card_index (&card_count, index), "adt_get_card_index",
1);
    printf("共检测到 %d 张 ADT-632XE 系列运动控制卡!\n", card_count);
    int card = index[0];
    //停止通讯，防止后台有通讯占用
    VERIFY_RETURN_MSG (adt_ecat_stop(card), "adt_ecat_stop", 1);
     //加载配置
    VERIFY_RETURN_MSG(adt_ecat_load_flash_cfg(card), "adt_ecat_load_flash_cfg", 1);
     //开启通讯
    VERIFY_RETURN_MSG (adt_ecat_start(card), "adt_ecat_start", 1);
    //配置编程模式为脉冲当量模式
    VERIFY_RETURN_MSG (adt_set_unit_mode(card, 1, 0), "adt_set_unit_mode", 1);
    //配置电子齿轮 伺服编码器分辨率 131072  对应伺服电机转一圈的脉冲数是 10000
    VERIFY_RETURN_MSG(adt_set_bus_axis_gear_ratio(card, 1, 131072, 10000), "adt_set_b
us_axis_gear_ratio", 1);
    //配置脉冲当量 丝杆导程 10mm
    VERIFY_RETURN_MSG(adt_set_pulse_equiv(card, 1, 1000), "adt_set_pulse_equiv", 1);
    //断开通讯，关闭控制卡
    VERIFY_RETURN_MSG (adt_ecat_stop(card), "adt_ecat_stop", 1);
    VERIFY_RETURN_MSG (adt_close_card(), "adt_close_card", 1);
    system("pause");
    return 0;
}
```

第七章 轴状态监视与控制

7.1. 功能介绍

7.1.1. 逻辑位置

逻辑位置，即控制卡从最近一次重置坐标位置开始下发的脉冲总数。

逻辑位置的控制接口参考

adt_set_command_pos/adt_get_command_pos

7.1.2. 实际位置

编码器位置，本手册称之为实际位置，即有编码器反馈时，从最近一次重置坐标位置开始，编码器实际接收到的脉冲总数。

实际位置的控制接口参考

adt_set_actual_pos/adt_get_actual_pos

7.1.3. 目标位置

目标位置，即控制卡当前指令执行完成后，指定轴即将到达的逻辑位置。该位置通常用于驱动结束后校验当前位置与目标位置是否相符来保证设备驱动精度。

目标位置的控制接口参考

adt_get_target_pos_unit/adt_get_target_pos_pulse

7.1.4. 伺服使能

通过 adt_set_axis_enable 控制伺服使能的开启与关闭，通过 adt_get_axis_enable 获取伺服使能的开启与关闭状态。

7.1.5. 轴规划速度

通过 adt_get_speed 获取轴的当前轴的规划速度，被获取速度的轴，可以是实体轴，也可以是插补轴，可以获取基于脉冲当量编程模式的轴速度，也可以获取基于脉冲编程模式的轴速度。

7.1.6. 编码器反馈速度

通过 adt_get_encoder_speed 获取当前轴的编码器反馈速度，此速度值是根据编码器反馈所计算出的速度。

在使用此功能前，需要先确保逻辑位置与实际位置的比例是 1：1 关系，且方向相同。可通过 adt_set_encoder_scale 设置编码器反馈分频比。

7.2. 获取轴状态信息

7.2.1. 轴运动状态

通过 adt_get_axis_status_ex 可获取当前轴的状态信息，可根据每一位的值判断轴的各种情况；当对应的位值为 1 时，说明当前状态有效；

每位对应的情况如下：

位		12	11	10	9	8	7
状态		总线伺服使能标志	无	伺服报警	位置误差超限	到位标志	软件负限位
位	6	5	4	3	2	1	0
状态	软件正限位	Z相限位	原点限位	负限位	正限位	暂停	运动中

可以通过 adt_get_all_axis_move_status 获取所有轴的运动状态，可根据每一位的值判断轴的是不是在运行；当对应的位值为 1 时，说明当前轴正在运动中。

通过 adt_get_all_axis_reach_status 获取所有轴的到位状态，可根据每一位的值判断轴的是否到位；当对应的位值为 1 时，说明当前轴已经到达目标位置，需要先设定好相应轴的到位误差 adt_set_axis_band 才能使用此功能。

7.2.2. 停止信息

接口 adt_get_axis_status 可以获取指定轴的驱动状态，即静止状态(0)运动状态(1)或暂停状态(2)，该接口对虚拟的插补轴同样适用。

当驱动状态停止时，可通过 adt_get_stopdata 获取指定轴停止信息，即轴是否为正常停止(脉冲已驱动完全)，或其他因素导致非正常停止(限位触发、紧急停止等)。

调用 adt_set_axis_stop 可停止指定轴的驱动。

点位驱动或连续驱动，当轴倍率为 0，即暂停时调用轴停止指令，轴的驱动数据将被清空，即使倍率恢复，驱动也将无法恢复。插补驱动时，当轴倍率为 0，即暂停时，调用轴停止指令，轴的驱动数据将被保留，倍率恢复后，驱动也将恢复。该工艺通常用于支持插补对刀的场合。若要轴驱动停止时，完全清除插补驱动数据，则须调用 adt_reset_card。

驱动执行完毕的正常停止和调用 adt_set_axis_stop 造成的驱动停止，不会产生异常的停止信息，即其停止信息为 0。

7.3. 例程 7-1 轴状态监控

```
int _tmain(int argc, _TCHAR* argv[])
{
    int card_count = 0;//系统控制卡数量
    int index[10] = {0};//系统可用控制卡索引
    int axis_count = 0;//控制卡可用轴数
    //初始化控制卡
    VERIFY_RETURN_MSG (adt_initial (&card_count), "adt_initial", 1);
    printf("控制卡初始化完成!\n");
    //获取可用控制卡索引与可用轴数
    VERIFY_RETURN_MSG (adt_get_card_index (&card_count, index), "adt_get_card_index"
, 1);
    printf("共检测到 %d 张运动控制卡!\n", card_count);
    int card = index[0];
    //停止通讯，防止后台有通讯占用
    VERIFY_RETURN_MSG (adt_ecat_stop(card), "adt_ecat_stop", 1);
    //加载配置
    VERIFY_RETURN_MSG (adt_ecat_load_flash_cfg(card), "adt_ecat_load_flash_cfg", 1);
    //开启通讯
    VERIFY_RETURN_MSG (adt_ecat_start(card), "adt_ecat_start", 1);
    long comPos, actPos;
    //获取轴 1 逻辑位置
    VERIFY_RETURN_MSG (adt_get_command_pos(card, 1, &comPos), "adt_get_command_pos",
 1);
    //获取轴 1 实际位置
    VERIFY_RETURN_MSG (adt_get_actual_pos(card, 1, &actPos), "adt_get_actual_pos", 1
);
    printf("轴 1 逻辑位置：%d，实际位置%d，", comPos, actPos);
    //断开通讯，关闭控制卡
    VERIFY_RETURN_MSG (adt_ecat_stop (card), "adt_ecat_stop", 1);
    VERIFY_RETURN_MSG (adt_close_card(), "adt_close_card", 1);
```

```
    system("pause");
    return 0;
}
```

7.4. 例程 7-2 停止信息解释示例

该辅助算子旨在将标准算子或扩展算子执行时的非正常返回解析为可读字符串错误信息，即错误码解析。

```
void DecodeStopData(char *msg, int len, int card, int axis, int stp_data)
{
    if(0 == msg || 0>=len)
        return;
    ZeroMemory(msg, len);
    if(stp_data&1 == 1)
        sprintf(msg, "DIP-%d AXLS-%d 硬件正限位触发!", card, axis);
    else if(stp_data&2 == 2)
        sprintf(msg, "DIP-%d AXLS-%d 硬件负限位触发!", card, axis);
    else if(stp_data&4 == 4)
        sprintf(msg, "DIP-%d AXLS-%d 原点信号(STOP0)触发!", card, axis);
    else if(stp_data&8 == 8)
        sprintf(msg, "DIP-%d AXLS-%d 编码器 Z 相信号(STOP1)触发!", card, axis);
    else if(stp_data&16 == 16)
        sprintf(msg, "硬件停止信号触发!", card, axis);
    else if(stp_data&32 == 32)
        sprintf(msg, "DIP-%d AXLS-%d 软件正限位触发!", card, axis);
    else if(stp_data&64 == 64)
        sprintf(msg, "DIP-%d AXLS-%d 软件负限位触发!", card, axis);
    else if(0 != stp_data)
        sprintf(msg, "DIP-%d AXLS-%d 未知停止信息!", card, axis);
    else
        sprintf(msg, "DIP-%d AXLS-%d 驱动完成!", card, axis);
    strcat(msg, "\n");
}
```

第八章 速度规划

8.1. 功能介绍

8.1.1. 基础速度规划

ADT-632XE 运动控制卡支持多种加减速模式，包括 T 型对称/非对称加减速、S 型对称/非对称加减速、指数型(EXP)加减速、三角函数型(COS)型加减速。

基础速度规划涉及以下算子：

adt_set_startv/adt_get_startv – 起始速度的设置与获取

adt_set_endv/adt_get_endv – 终止速度点设置与获取

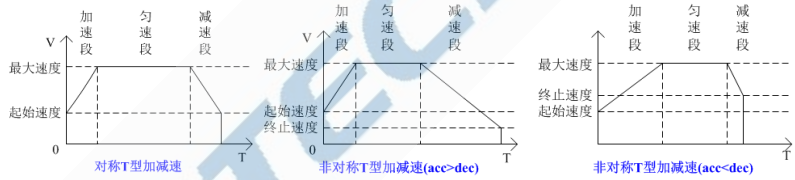
adt_set_maxv/adt_get_maxv – 最大速度的设置与获取

adt_set_acc/adt_get_acc – 加速度的设置与获取

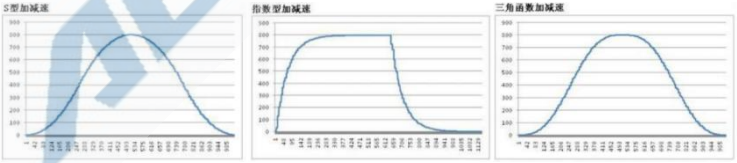
adt_set_dec/adt_get_dec – 减速度的设置与获取

adt_set_admode /adt_get_admode – 加减速模式的设置与获取

其中起始速度、终止速度、最大速度、加速度、减速度参数对速度曲线的影响可以参考下述图形理解。



加减速模式则决定了速度曲线的平滑程度，除上述的 T 型加减速模式外，运动控制卡还支持如下几种加速模型。



8.1.2. 集成速度设置

ADT-632XE 运动控制卡针对当量编程模式下的点位运动提供了一个集成速度设置算子 adt_set_axis_move_para_unit，可一次性设置加减速模式、起步速度、最大速度、加速度、减速度。

8.1.3. 速度倍率

ADT-632XE 运动控制卡可以设置轴驱动时的速度倍率，设定倍率的轴可以是驱动轴，也可以是插补轴。

设置轴倍率会立即刷新轴的当前速度，故而建议变化率设置适当小，以防发生速度阶跃。理想的方式是通过定时的逐步设置，产生加减速效果。当速度倍率设置为 0 时，当前轴将处于**暂停**状态，已下发的驱动数据不会被清除，速度倍率恢复后，当前轴将继续按照预先设定的轨迹驱动。

速度倍率的设置与获取参考 adt_set_rate/adt_get_rate。

当轴处于静止状态时，若要获取当前轴是否处于暂停状态，可使用 adt_get_axis_status 获取当前轴状态，状态为 2 时即为暂停中。

插补驱动时，设置任何一个参与插补的轴的速度倍率都是无效的，若要设置插补速度倍率，须是设置插补轴的速度倍率。

8.1.4. 速度前瞻

ADT-632XE 运动控制卡具有超大容量多轴缓存区，使得插补离散数据能很好地还原为加工模型。

速度前瞻功能则使多段插补驱动连续运行，控制卡将自动计算并优化插补轨迹拐点速度，保证加工轨迹保证精度的同时实现速度的平滑过渡。

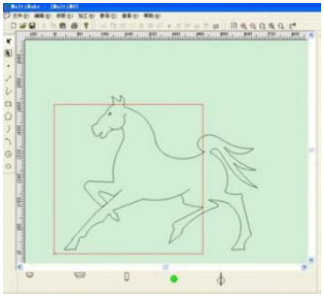
速度前瞻与缓存插补是相辅相成的，缓存插补，即超大容量缓存，配合速度前瞻功能，可以实现高精高速下的速度自动优化和轨迹自动调整。速度过渡平滑，机械振动减弱，高质量的加工精度，同时保证高速高效加工过程。

速度前瞻的功能仅在执行插补类驱动中生效，定量驱动和持续驱动功能与速度前瞻无关。

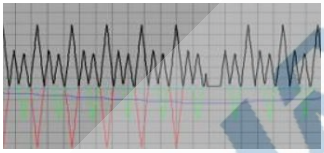
开启速度前瞻功能时，控制卡将自动为插补开启缓存模式，速度前瞻功能接口可参考 adt_set_speed_pretreat_mode。

有关插补功能的介绍，请参考。

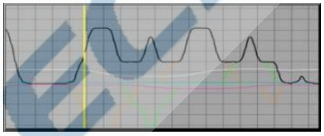
例如，加工如下轨迹时。



若不启用速度前瞻功能，即复杂的轨迹曲线使用多段小线段或小圆弧插补拟合时，不使用缓存插补功能，则其速度曲线将十分尖锐，加工设备也将振动剧烈。



启用速度前瞻功能，即使用缓存插补功能拟合多段小线段或小圆弧插补拟合，则速度曲线大幅平滑，其设备振动也将大大降低。



8.1.5. 圆弧插补速度约束

ADT-632XE 运动控制卡支持设置插补轴在圆弧插补时依据圆弧半径系数约束其驱动速度。

平面圆弧或空间圆弧插补中，若实际半径小于半径系数，实际半径越小则圆弧速度被限制的越小；若实际半径大于半径系数，实际半径越大则圆弧速度受到的约束值越大。当实际径与半径系数相等时，圆弧的最大速度与速度系数相等。

实际速度约束 Va、实际圆弧半径 Ra、速度约束设置 Vs、圆弧半径系数设置值 Rs 的关系如下：

Va = sqrt(Ra/Rs)*Vs。

接口参考 adt_set_arc_speed_clamp_unit。

圆弧插补速度约束只能用于基于脉冲当量的编程方式。

8.1.6. 拐角加速度平滑等级

拐角加速度平滑用于在缓存插补过程中，限制小线段轨迹拐角处的速度变化率，在调试过程中，在不影响驱动精度和效率的前提下，逐渐提高平滑系数，有利于降低插补驱动时的机械抖动。

平滑系数 0~99 对应的加速度约束值如下，单位 mm/s²。

//0-9

100,258,416, 574, 732, 890, 1048, 1206, 1364,1522,

//10-19

1680, 1838, 1996, 2154, 2312, 2470, 2628, 2786, 2944, 3102,

//20-29

3260, 3418, 3576, 3734, 3892, 4050, 4208, 4366, 4524, 4682,

//30-39

4840, 4998, 5156, 5314, 5472, 5630, 5788, 5946, 6104, 6262,

//40-49

6420, 6578, 6736, 6894, 7052, 7210, 7368, 7526, 7684, 7842,

//50-59

8000,9878,11756,13634,15512,17390,19268,21146,23024,24902,

//60-69

26780, 28658, 30536, 32414, 34292, 36170, 38048, 39926,
41804,43682,

//70-79

45560, 47438, 49316, 51194, 53072, 54950, 56828, 58706,
60584,62462,

//80-89

64340, 66218, 68096, 69974, 71852, 73730, 75608, 77486,
79364,81242,

//90-99

83120,84998,86876,88754,90632, 92510, 94388, 96266, 98144,100000

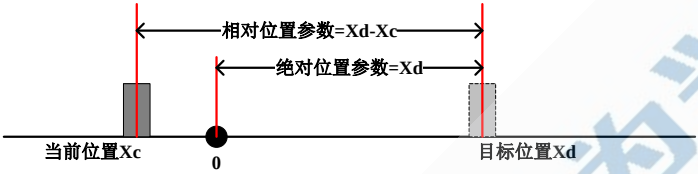
接口参考 adt_set_corner_speed_smooth_level，默认平滑系数为 50。

第九章 点位运动

9.1. 功能介绍

9.1.1. 相对位置与绝对位置

相对位置与绝对位置多用于驱动指令中的位置模式。相对位置即目标位置与当前位置之差。绝对位置即目标位置相对于 0 点的坐标位置。

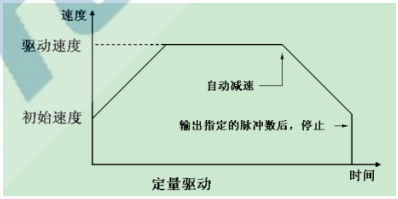


9.1.2. 定量驱动

定量驱动指以定速或加减速输出指定数量的脉冲，从当前位置以设置的速度模式驱动到指定位置后准确停止。

定量驱动只关注目标位置，对运动轨迹的精度没有要求，其驱动距离由脉冲数决定，其驱动速度由脉冲频率决定。

T 型加减速的定量驱动速度/时间曲线如下图，输出脉冲剩余数小于减速累计脉冲数时，驱动开始减速，减速至设定的初始速度时，轴位置将准确地停止在下发的指定脉冲位置。



定量驱动前须合理设置速度相关参数， 基于脉冲当量编程模式下的定量驱动接口可参考 adt_pmove_unit，基于脉冲编程模式下的定量驱动接口可参考 adt_pmove_pulse。

小提示	多轴同时做定量驱动时，称之为多轴联动。多条定量驱动指令下发，其指令周期间隔仅在微秒级别，对于机械而言，可以认为是同时启动，但若追求多轴同时启动、其轨迹须是空间直线时，多轴联动则不可取，此时须采用直线插补功能。

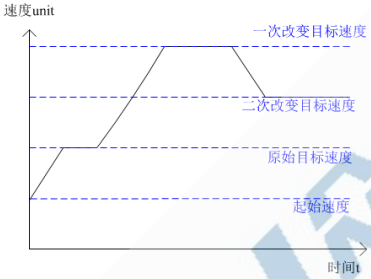
9.1.3. 连续驱动

连续驱动指电机从起始速度按照预设参数加速至最大速度后连续运动，直至接受主动停止命令或软硬件停止信号时，方才立即停止或减速停止。

连续驱动多用于传送带等上下料的速度控制场合。

连续驱动前须合理设置驱动速度相关参数，连续驱动接口可参照 adt_continue_move，驱动过程中可以实时改变目标速度(adt_set_maxv)。

以 T 型加减速为例，连续驱动及过程中实时改变目标速度曲线案例如下。



9.1.4. 在线改变目标位置

ADT-632XE 运动控制卡提供了在线改变目标位置的功能，即可在点位运动过程中改变最终的目标位置。

在线改变目标位置接口可参照：adt_change_pmove_pos

9.1.5. 轴驱动停止

在轴进行点位运动的过程中可以调用 adt_set_axis_stop 实现当前轴停止运动，可以根据实际使用情况进行减速停止与立即停止。

9.2. 例程 9-1 定量驱动

```
int _tmain(int argc, _TCHAR* argv[])
{
    int card_count = 0;//系统控制卡数量
    int index[10] = {0};//系统可用控制卡索引
    int axis_count = 0;//控制卡可用轴数
    //初始化控制卡
    VERIFY_RETURN_MSG (adt_initial (&card_count), "adt_initial", 1);
    printf("控制卡初始化完成!\n");
    //获取可用控制卡索引与可用轴数
```

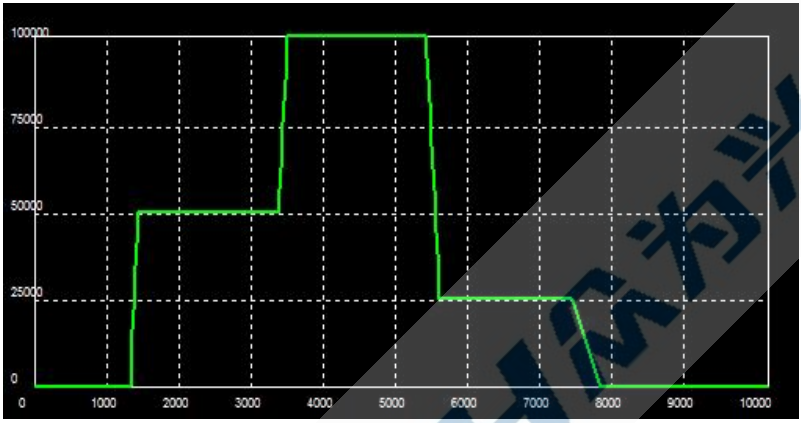
```
    VERIFY_RETURN_MSG (adt_get_card_index (&card_count, index), "adt_get_card_index"
, 1);
    printf("共检测到 %d 张运动控制卡!\n", card_count);
    int card = index[0];
    //停止通讯，防止后台有通讯占用
    VERIFY_RETURN_MSG (adt_ecat_stop(card), "adt_ecat_stop", 1);
    //加载配置
    VERIFY_RETURN_MSG (adt_ecat_load_flash_cfg(card), "adt_ecat_load_flash_cfg", 1);
    //开启通讯
    VERIFY_RETURN_MSG (adt_ecat_start(card), "adt_ecat_start", 1);
    //伺服从站使能
    VERIFY_RETURN_MSG (adt_set_axis_enable(card, 1, 1), "adt_set_axis_enable", 1);
    //伺服速度设置
    VERIFY_RETURN_MSG(adt_set_axis_move_para_unit(card, 1, 1, 5, 20, 200, 200), "adt_
set_axis_move_para_unit", 1);
    //正方向相对驱动轴 10mm
    VERIFY_RETURN_MSG (adt_pmove_unit(card, 1, 100, 0), "adt_pmove_unit", 1);
    //执行确认
    CONFIRM_DRIVE(1);
    //断开通讯，关闭控制卡
    VERIFY_RETURN_MSG (adt_ecat_stop (card), "adt_ecat_stop", 1);
    VERIFY_RETURN_MSG (adt_close_card(), "adt_close_card", 1);
    system("pause");
    return 0;
}
```

9.3. 例程 9-2 连续驱动

```
int _tmain(int argc, _TCHAR* argv[])
{
    int crd_count = 0, crd_index[10] = {0};
    //初始化控制卡
    VERIFY_RETURN_MSG (adt_initial (&card_count), "adt_initial", 1);
    printf("控制卡初始化完成!\n");
```

```
    //获取可用控制卡索引与可用轴数
    VERIFY_RETURN_MSG (adt_get_card_index (&card_count, index), "adt_get_card_index"
, 1);
    printf("共检测到 %d 张运动控制卡!\n", card_count);
    int card = index[0];
    //停止通讯，防止后台有通讯占用
    VERIFY_RETURN_MSG (adt_ecat_stop(card), "adt_ecat_stop", 1);
    //加载配置
    VERIFY_RETURN_MSG (adt_ecat_load_flash_cfg(card), "adt_ecat_load_flash_cfg", 1);
    //开启通讯
    VERIFY_RETURN_MSG (adt_ecat_start(card), "adt_ecat_start", 1);
    //伺服从站使能
    VERIFY_RETURN_MSG(adt_set_axis_enable(card, 1, 1), "adt_set_axis_enable", 1);
    //伺服速度设置
    VERIFY_RETURN_MSG (adt_set_startv(card, 1, 10), "adt_set_startv", 1);
    VERIFY_RETURN_MSG (adt_set_maxv(card, 1, 50), "adt_set_maxv", 1);
    VERIFY_RETURN_MSG (adt_set_acc(card, 1, 400), "adt_set_acc", 1);
    //执行确认
    CONFIRM_DRIVE(1);
    //正向连续驱动轴
    VERIFY_RETURN_MSG (adt_continue_move(card, 1, 0), "adt_continue_move", 1);
    //2s 后变速
    Sleep(2000);
    VERIFY_RETURN_MSG (adt_set_maxv(card, 1, 100), "adt_set_maxv", 1);
    //2s 后再变速
    Sleep(2000);
    VERIFY_RETURN_MSG (adt_set_maxv(card, 1, 25), "adt_set_maxv", 1);
    //2s 后停止
    Sleep(2000);
    VERIFY_RETURN_MSG (adt_set_axis_stop(card, 1, 0), "adt_set_axis_stop", 1);
    //断开通讯，关闭控制卡
    VERIFY_RETURN_MSG (adt_ecat_stop (card), "adt_ecat_stop", 1);
```

```
    VERIFY_RETURN_MSG_(adt_close_card(), "adt_close_card", 1);
    system("pause");
    return 0;
}
```



9.4. 例程 9-6 龙门双驱

```
int _tmain(int argc, _TCHAR* argv[])
{
    int crd_count = 0, crd_index[10] = {0};
    //初始化控制卡
    VERIFY_RETURN_MSG_(adt_initial (&card_count), "adt_initial", 1);
    printf("控制卡初始化完成!\n");
    //获取可用控制卡索引与可用轴数
    VERIFY_RETURN_MSG_(adt_get_card_index (&card_count, index), "adt_get_card_index"
, 1);
    printf("共检测到 %d 张运动控制卡!\n", card_count);
    //总线从站配置，以配置台 ADTECH EtherCAT 总线伺服驱动器 QXE+2 台 ADTECH EtherCAT 总线 IO
模块 ADT-ET1616A 为例
    int card = crd_index[0];//第一块可用 ADT-632XE 总线控制卡索引
    //停止通讯，防止后台有通讯占用
    VERIFY_RETURN_MSG_(adt_ecat_stop(card), "adt_ecat_stop", 1);
    //加载配置
```

```
VERIFY_RETURN_MSG (adt_ecat_load_flash_cfg(card), "adt_ecat_load_flash_cfg", 1);
//开启通讯
VERIFY_RETURN_MSG (adt_ecat_start(card), "adt_ecat_start", 1);
//伺服从站使能
VERIFY_RETURN_MSG(adt_set_axis_enable(card, 1, 1), "adt_set_axis_enable", 1);
VERIFY_RETURN_MSG(adt_set_axis_enable(card, 2, 1), "adt_set_axis_enable", 1);
//编程模式设置
VERIFY_RETURN_MSG (adt_set_unit_mode(card, 1, 0), "adt_set_unit_mode", 1);
VERIFY_RETURN_MSG (adt_set_unit_mode (card, 2, 0), "adt_set_unit_mode", 1);
VERIFY_RETURN_MSG(adt_set_pulse_equiv(card, 1, 1000), "adt_set_pulse_equiv", 1);
VERIFY_RETURN_MSG(adt_set_pulse_equiv(card, 2, 1000), "adt_set_pulse_equiv", 1);
//速度设置
VERIFY_RETURN_MSG (adt_set_startv(card, 1, 10), "adt_set_startv", 1);
VERIFY_RETURN_MSG (adt_set_maxv(card, 1, 50), "adt_set_maxv", 1);
VERIFY_RETURN_MSG (adt_set_acc(card, 1, 400), "adt_set_acc", 1);
//依据用户输入确定是否同步跟随
int axis = 0;
printf("是否使轴跟随轴同步驱动?\n1--跟随 0--不跟随\n");
cin>>axis;
VERIFY_RETURN_MSG(adt_set_follow_axis(card, 2, axis), "adt_set_follow_axis", 1);
//执行确认
CONFIRM_DRIVE(1);
//轴定量驱动
VERIFY_RETURN_MSG (adt_pmove_unit(card, 1, 100, 0), "adt_pmove_unit", 1);
//等待驱动停止
adt_till_stop(card, 1);
//取消跟随
VERIFY_RETURN_MSG (adt_set_follow_axis(card, 2, 0), "adt_set_follow_axis", 1);
printf("驱动结束，解除同步!\n");
//断开通讯，关闭控制卡
VERIFY_RETURN_MSG (adt_ecat_stop (card), "adt_ecat_stop", 1);
VERIFY_RETURN_MSG (adt_close_card(), "adt_close_card", 1);
```

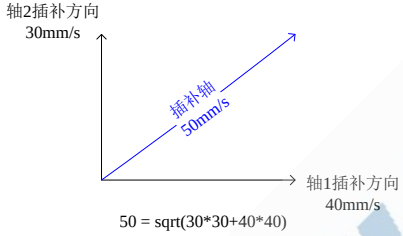
```
    system("pause");
    return 0;
}
```

第十章 插补运动

10.1. 功能介绍

插补驱动广泛应用于点胶、切割、焊接等多种数控机床加工工艺当中。插补驱动实现多轴的协调驱动，将轨迹数据密化，加工出需求的轮廓。

插补轴，为控制卡在插补过程中虚拟的驱动轴，其驱动方向与速度均为参与插补的 N 个实体轴组成的空间运动坐标系的方向与速度矢量和。例如简单的平面直线插补如下图所示，其矢量速度与方向满足如下运算关系，空间直线或更复杂的插补，其插补轴的速度与方向的运算则几何类推。



控制卡提供了 63－60 四组插补轴，即可以实现同时进行四组独立的插补驱动。

插补驱动常见类型有：直线插补，平面圆弧插补，球面圆弧插补等。

10.2. 创建插补坐标系

在控制卡初始化始化状态、复位或未使用过插补状态下，所有的运动控制卡的实体轴（以下简称实体轴）都处于单轴点位运动模式下，插补轴是无效的。

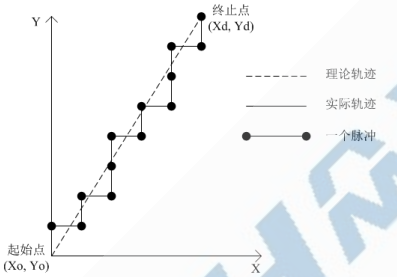
在进行插补运动之前需要创建插补坐标系，调 adt_set_inp_coordinate 将实体轴添加到插补坐标系中 ，即将实体轴与虚拟插补轴进行关联。

注意：

1. 同一个实体轴不能同时加入多个坐标系中，若需要将实体轴改换到其他坐标系中，需要将其当前所在的坐标系关闭后再创建到其他坐标系中。
2. 坐标系在不进行更改的情况下，只需要在控制卡初始化后创建一次，不需要每次插补时都进行创建。

10.3. 直线插补

所谓直线插补就是指能用于理论轮廓是直线或可以用有限个小线段拟合成直线的插补方式。首先假设在实际轮廓起始点处沿 x 方向走一小段(一个脉冲当量)，发现终点在实际轮廓的下方,则下一条线段沿 y 方向走一小段，此时如果线段终点还在实际轮廓下方，则继续沿 y 方向走一小段,直到在实际轮廓上方以后,再向 x 方向走一小段，依次循环类推，直到到达轮廓终点为止。实际轮廓由精度允许范围内的小线段折线拼接而成。

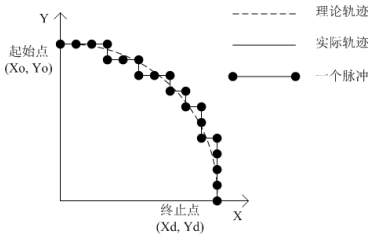


ADT-632XE 支持 1-16 轴直线插补，基于脉冲当量编程模式下的直线插补接口可参照 adt_inp_move_unit,基于脉冲编程模式下的直线插补接口可参照 adt_inp_move_pulse。

小提示	不同编程模式下的插补，一定要在速度配置和插补驱动指令之前，为对应的插补轴配置相同的编程模式。 例如，当轴 1 轴 2 已被 adt_set_unit_mode 为基于脉冲的编程模式，若轴 1 轴 2 参与直线插补，其使用的插补轴 INPA_AXIS 或 INPB_AXIS 须在速度和驱动指令之前，一并被 adt_set_unit_mode 为基于脉冲的编程模式

10.4. 平面圆弧插补

二维圆弧插补根据两端点间的插补数字信息，计算出逼近实际圆弧的点群，控制轨迹沿这些点运动，加工出圆弧曲线。其基本原理为用直线运动的两个轴 X 和 Y 共同确定一个点，X 直线运动，控制 Y 的坐标画圆。

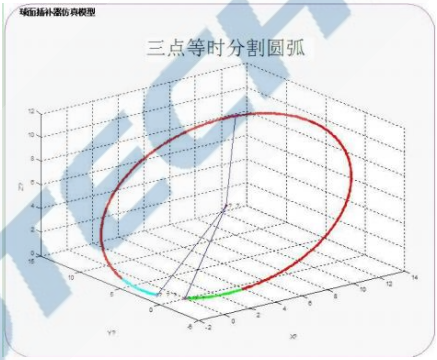


ADT-632XE 二维圆弧插补接口可参照 adt_inp_arc2_unit。

小提示	平面圆弧插补功能为基于脉冲当量编程模式下的高级插补功能，基于脉冲编程模式的轴不能参与平面圆弧插补

10.5. 球面圆弧插补

球面圆弧插补适用于简化复杂图形的教导操作，其精度由插补速度动态决定，避免了小线段拟合法在空间图形中的离散误差和速度的权衡矛盾。螺旋线插补和平面圆弧插补均可基于球面圆弧插补轻松实现。



ADT-632XE 球面圆弧插补接口可参照 adt_inp_arc3_unit。

小提示	球面圆弧插补功能为基于脉冲当量编程模式下的高级插补功能，基于脉冲编程模式的轴不能参与球面圆弧插补

10.6. 缓存插补

实际加工轨迹中，不免出现上述几种轨迹的组合，或大容量小线段连续插补的工艺需求。

控制卡具有超大容量多轴缓存区，使得 CAM 离散数据能很好地还原为加工模型。

缓存插补功能与速度前瞻功能是相辅相成的，缓存插补功能会随着速

度前瞻功能的开启而自动开启，其功效说明，也与速度前瞻一并介绍。

控制卡提供了 5000 段缓存插补余量，即同时可以缓存 5000 段插补指令，当缓存区满时，须等待缓存区有空缺时才能继续压入插补指令。缓存余量的查询可参考 adt_get_inp_fifo_len。

10.7. 插补暂停与恢复

可以通过 adt_set_inp_stop 对正进行的插补运动进行暂停与恢复运动。

插补运动暂停后，未完成的插补缓存数据依然保存着，此时停止运动的轴可以进行插补之外的运动，如点位运动。

需要暂停恢复时，则需将通过 adt_get_inp_pause_pos 获取插补暂停时的位置，并使轴移动到暂停位置，才可能通过 adt_set_inp_stop 进行暂停恢复。

10.8. 例程 10-1 直线插补

```
int _tmain(int argc, _TCHAR* argv[])
{
    int crd_count = 0, crd_index[10] = {0};
    //初始化运动控制卡
    VERIFY_RETURN_MSG(adt_initial(&crd_count, 0), "adt_initial", 1);
    printf("共检测到 %d 张运动控制卡!\n", crd_count);
    //查询可用控制卡索引
    VERIFY_RETURN_MSG(adt_get_card_index(&crd_count, crd_index), "adt_get_card_index"
, 1);
    //总线从站配置，以配置台 ADTECH EtherCAT 总线伺服驱动器 QXE+2 台 ADTECH EtherCAT 总线 IO
模块 ADT-ET1616A 为例
    int card = crd_index[0];//第一块可用 ADT-632XE 总线控制卡索引
    //停止通讯，防止后台有通讯占用
    VERIFY_RETURN_MSG (adt_ecat_stop(card), "adt_ecat_stop", 1);
    //加载配置
    VERIFY_RETURN_MSG(adt_ecat_load_flash_cfg(card), "adt_ecat_load_flash_cfg", 1);
    //开启通讯
    VERIFY_RETURN_MSG (adt_ecat_start(card), "adt_ecat_start", 1);
```

```
    //伺服从站使能
    VERIFY_RETURN_MSG (adt_set_axis_enable(card, 1, 1), "adt_set_axis_enable", 1);
    VERIFY_RETURN_MSG (adt_set_axis_enable(card, 2, 1), "adt_set_axis_enable", 1);
    printf("伺服从站使能完成!\n");
    //编程模式设置,使用的插补轴也要设置编程模式
    VERIFY_RETURN_MSG (adt_set_unit_mode(card, 1, 0), "adt_set_unit_mode", 1);
    VERIFY_RETURN_MSG (adt_set_unit_mode (card, 2, 0), "adt_set_unit_mode", 1);
    //速度设置
    VERIFY_RETURN_MSG (adt_set_startv(card, INPA_AXIS, 10), "adt_set_startv", 1);
    VERIFY_RETURN_MSG (adt_set_maxv(card, INPA_AXIS, 50), "adt_set_maxv", 1);
    VERIFY_RETURN_MSG (adt_set_acc(card, INPA_AXIS, 400), "adt_set_acc", 1);
//建立插补坐标系
    int axs_list[2] = {1,2};
    VERIFY_RETURN_MSG(adt_set_inp_coordinate(card,INPA_AXIS,2,axs_list), "VERIFY_RETU
RN_MSG", 1);
    //执行确认
    CONFIRM_DRIVE(0);
    //确认插补缓存空间
    int inp_len = 0;
    do
    {
        VERIFY_RETURN_MSG(adt_get_inp_fifo_len(card, INPA_AXIS, &inp_len), "adt_get_i
np_fifo_len", 1);
    } while (inp_len < 3);
    //直线插补驱动
    int inp_list[] = {1,2};//插补坐标系中进行插补运动的轴序号
    double pos_list[2] = {100,100};
    VERIFY_RETURN_MSG(adt_inp_move_unit(card, INPA_AXIS, 0, 2, inp_list, pos_list, 0)
, "adt_inp_move_unit", 1);
    //等待插补轴驱动停止
    VERIFY_RETURN_MSG (adt_till_stop(card, INPA_AXIS), "adt_till_stop", 1);
    system("pause");
```

```
    return 0;
}
```

10.9. 例程 10-2 平面圆弧插补

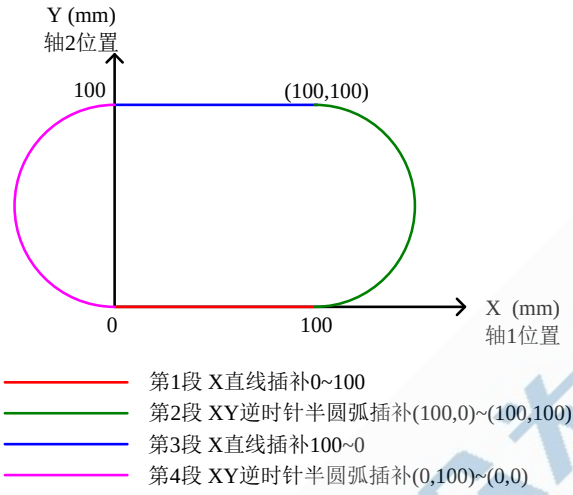
```
int _tmain(int argc, _TCHAR* argv[])
{
    int crd_count = 0, crd_index[10] = {0};
    //初始化运动控制卡
    VERIFY_RETURN_MSG(adt_initial(&crd_count, 0), "adt_initial", 1);
    printf("共检测到 %d 张运动控制卡!\n", crd_count);
    //查询可用控制卡索引
    VERIFY_RETURN_MSG(adt_get_card_index(&crd_count, crd_index), "adt_get_card_index"
, 1);
    //总线从站配置，以配置台 ADTECH EtherCAT 总线伺服驱动器 QXE+2 台 ADTECH EtherCAT 总线 IO
模块 ADT-ET1616A 为例
    int card = crd_index[0];//第一块可用 ADT-632XE 总线控制卡索引
    //停止通讯，防止后台有通讯占用
    VERIFY_RETURN_MSG (adt_ecat_stop(card), "adt_ecat_stop", 1);
    //加载配置
    VERIFY_RETURN_MSG(adt_ecat_load_flash_cfg(card), "adt_ecat_load_flash_cfg", 1);
    //开启通讯
    VERIFY_RETURN_MSG (adt_ecat_start(card), "adt_ecat_start", 1);
    //伺服从站使能
    VERIFY_RETURN_MSG (adt_set_axis_enable(card, 1, 1), "adt_set_axis_enable", 1);
    VERIFY_RETURN_MSG (adt_set_axis_enable(card, 2, 1), "adt_set_axis_enable", 1);
    printf("伺服从站使能完成!\n");
    //编程模式设置,使用的插补轴也要设置编程模式
    VERIFY_RETURN_MSG (adt_set_unit_mode(card, 1, 0), "adt_set_unit_mode", 1);
    VERIFY_RETURN_MSG (adt_set_unit_mode (card, 2, 0), "adt_set_unit_mode", 1);
    //速度设置
    VERIFY_RETURN_MSG (adt_set_startv(card, INPA_AXIS, 10), "adt_set_startv", 1);
    VERIFY_RETURN_MSG (adt_set_maxv(card, INPA_AXIS, 50), "adt_set_maxv", 1);
VERIFY_RETURN_MSG (adt_set_acc(card, INPA_AXIS, 400), "adt_set_acc", 1);
```

```
//建立插补坐标系
    int axs_list[2] = {1,2};
    VERIFY_RETURN_MSG(adt_set_inp_coordinate(card,INPA_AXIS,2,axs_list), "VERIFY_RETU
RN_MSG", 1);
      VERIFY_RETURN_MSG ( (card, INPA_AXIS, 400), "adt_set_acc", 1);
    //执行确认
    CONFIRM_DRIVE(0);
    //确认插补缓存空间
    int inp_len = 0;
    do
    {
        VERIFY_RETURN_MSG(adt_get_inp_fifo_len(card, INPA_AXIS, &inp_len), "adt_get_i
np_fifo_len", 1);
    } while (inp_len < 3);
    //直线插补驱动
    int inp_list[] = {1,2};//插补坐标系中进行插补运动的轴序号
    double pos_list[2] = {0,0}, ctr_list[2] = {50, 0};
    //轴/2 平面圆弧插补
    //当前位置为起点，相对位置驱动，当前位置为终点，则目标位置相对坐标为(0,0)
    //圆心坐标为(50,0)，平面整圆圆弧插补
    VERIFY_RETURN_MSG(adt_inp_arc2_unit(card, INPA_AXIS, 0, inp_list, pos_list, ctr_l
ist, 0, 0), "adt_inp_arc2_unit", 1);
    //等待插补轴驱动停止
    VERIFY_RETURN_MSG (adt_till_stop(card, INPA_AXIS), "adt_till_stop", 1);
    system("pause");
    return 0;

}
```

10.10. 例程 10-3 缓存插补

本案例分别在标准插补驱动和缓存插补驱动下实现下图轨迹。



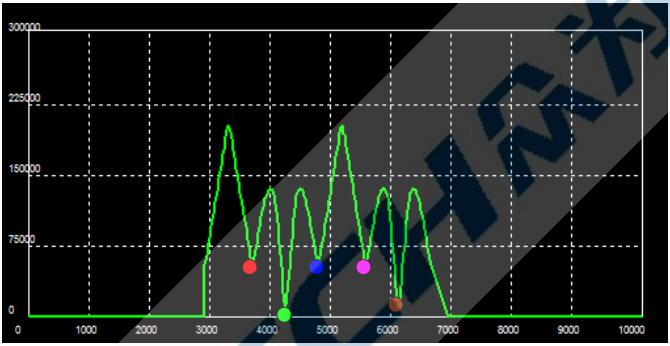

```
int _tmain(int argc, _TCHAR* argv[])
{
    int crd_count = 0, crd_index[10] = {0};
    //初始化运动控制卡
    VERIFY_RETURN_MSG(adt_initial(&crd_count, 0), "adt_initial", 1);
    printf("共检测到 %d 张运动控制卡!\n", crd_count);
    //查询可用控制卡索引
    VERIFY_RETURN_MSG(adt_get_card_index(&crd_count, crd_index), "adt_get_card_index"
, 1);
    //总线从站配置，以配置台 ADTECH EtherCAT 总线伺服驱动器 QXE+2 台 ADTECH EtherCAT 总线 IO
模块 ADT-ET1616A 为例
    int card = crd_index[0];//第一块可用 ADT-632XE 总线控制卡索引
    //停止通讯，防止后台有通讯占用
    VERIFY_RETURN_MSG (adt_ecat_stop(card), "adt_ecat_stop", 1);
    //加载配置
    VERIFY_RETURN_MSG(adt_ecat_load_flash_cfg(card), "adt_ecat_load_flash_cfg", 1);
    //开启通讯
    VERIFY_RETURN_MSG (adt_ecat_start(card), "adt_ecat_start", 1);
    //伺服从站使能
    VERIFY_RETURN_MSG (adt_set_axis_enable(card, 1, 1), "adt_set_axis_enable", 1);
```

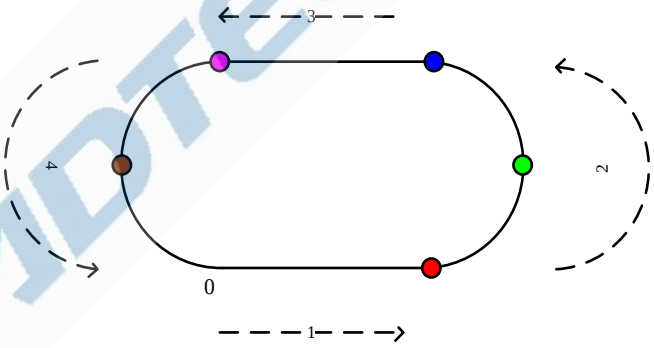
```
    VERIFY_RETURN_MSG (adt_set_axis_enable(card, 2, 1), "adt_set_axis_enable", 1);
    printf("伺服从站使能完成!\n");
    //编程模式设置,使用的插补轴也要设置编程模式
    VERIFY_RETURN_MSG (adt_set_unit_mode(card, 1, 0), "adt_set_unit_mode", 1);
    VERIFY_RETURN_MSG (adt_set_unit_mode (card, 2, 0), "adt_set_unit_mode", 1);
    //速度设置
    VERIFY_RETURN_MSG (adt_set_startv(card, INPA_AXIS, 10), "adt_set_startv", 1);
    VERIFY_RETURN_MSG (adt_set_maxv(card, INPA_AXIS, 50), "adt_set_maxv", 1);
    VERIFY_RETURN_MSG (adt_set_acc(card, INPA_AXIS, 400), "adt_set_acc", 1);
//建立插补坐标系
    int axs_list[2] = {1,2};
    VERIFY_RETURN_MSG(adt_set_inp_coordinate(card,INPA_AXIS,2,axs_list), "VERIFY_RETU
RN_MSG", 1);
    //依据用户输入确定是否启用速度前瞻
    int mode = 0;
    printf("是否启用速度前瞻? 1--启用 0--不启用\n");
    cin>>mode;
    VERIFY_RETURN_MSG(adt_set_speed_pretreat_mode(card, INPA_AXIS, mode), "adt_set_sp
eed_pretreat_mode", 1);
    //执行确认
    CONFIRM_DRIVE(0);
    //确认插补缓存空间
    int inp_len = 0;
    do
    {
        VERIFY_RETURN_MSG(adt_get_inp_fifo_len(card, INPA_AXIS, &inp_len), "adt_get_i
np_fifo_len", 1);
    } while (inp_len < 100);
    //直线插补驱动
    int inp_list[] = {1,2};//插补坐标系中进行插补运动的轴序号
    //---------------第 1 段,X 直线插补 0~100mm---------------//
    double pos1[] = {100, 0};
```

```
    VERIFY_RETURN_MSG(adt_inp_move_unit(card, INPA_AXIS, 0, 2, inp_list, pos1, 0), "a
dt_inp_move_unit", 1);
    //-----第 2 段,XY 逆时针半圆弧插补(100,0)~(100,100)----//
    //相对位置模式下圆弧参数设置方法
    //X 的目标位置是 100,当前位置是 100,相对位置参数=目标位置-当前位置=0
    //Y 的目标位置是 100,当前位置是 0,相对位置参数=目标位置-当前位置=100
    //X 的圆心目标位置是 100,当前位置是 100,相对位置参数=圆心目标位置-当前位置=0
    //Y 的圆心目标位置是 50,当前位置是 0,相对位置参数=目标位置-当前位置=50
    double pos_list[] = {0, 100};
    double ctr_list[] = {0, 50};
    VERIFY_RETURN_MSG(adt_inp_arc2_unit(card, INPA_AXIS, 0, axs_list, pos_list, ctr_l
ist,
        0/*逆时针*/,
        0/*相对位置驱动模式*/),
        "adt_inp_arc2_unit", 1);
    //---------------第 3 段,X 直线插补 100~0---------------//
    pos1[1] = 100;
    VERIFY_RETURN_MSG (adt_inp_move_unit (card, INPA_AXIS, 0, 2, inp_list, pos1,
        1/*绝对位置模式*/),
        "adt_inp_move_unit", 1);
    //-------第 4 段,XY 逆时针半圆弧插补(0,100)~(0,0)------//
    //绝对位置模式下圆弧参数设置方法
    //X 的目标位置是 0,绝对位置参数=目标位置=0
    //Y 的目标位置是 0,绝对位置参数=目标位置=0
    //X 的圆心目标位置是 0,绝对位置参数=圆心目标位置=0
    //Y 的圆心目标位置是 50,绝对位置参数=目标位置=50
    pos_list[0] = 0, pos_list[1] = 0;
    ctr_list[0] = 0, ctr_list[1] = 50;
    VERIFY_RETURN_MSG(adt_inp_arc2_unit(card, INPA_AXIS, 0, axs_list, pos_list, ctr_l
ist,
        0/*逆时针*/,
        0/*绝对位置模式*/),
```

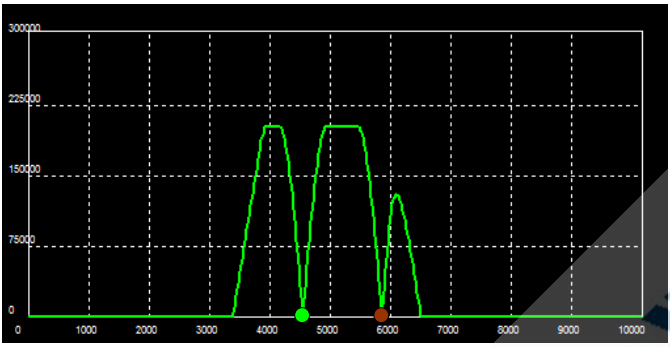
```
        "adt_inp_arc2_unit", 1);
    //等待插补轴驱动停止
    VERIFY_RETURN_MSG (adt_till_stop(card, INPA_AXIS), "adt_till_stop", 1);
    system("pause");
    return 0;
}
```

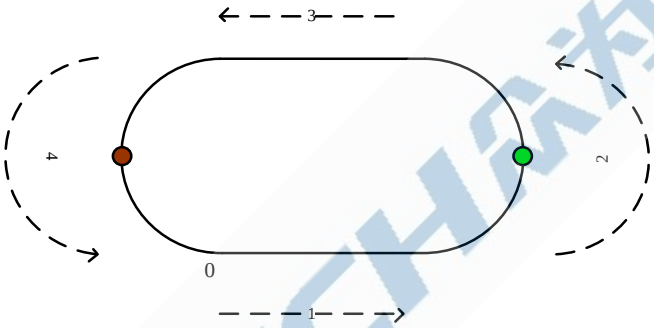
不开启速度前瞻功能时，4段插补为标准插补，参与插补的轴1速度曲线如下图所示，图中的5个速度转折点按颜色分别对应着插补轨迹中的5个位置点。





开启速度前瞻功能时，4段插补为缓存插补，参与插补的轴1速度曲线如下图所示，图中的2个速度转折点按颜色分别对应着插补轨迹中的2个位置点。





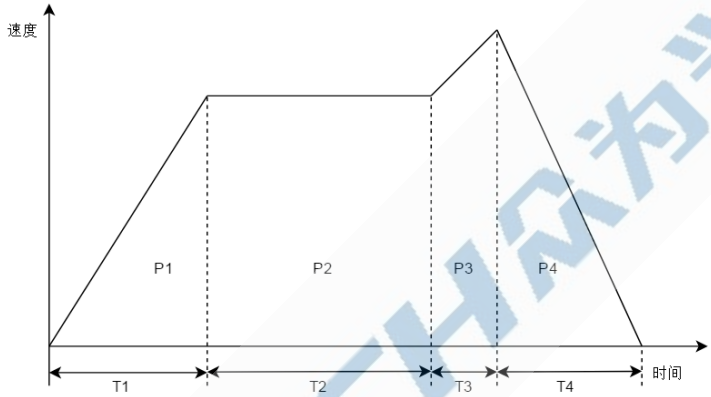
由实际效果可知，绿色和褐色两个速度转折点是圆弧插补中轴 1 必然要经历的速度将为 0 的速度反向点，其余的 3 个速度转折点则可通过速度前瞻模式开启缓存插补后平滑过渡。

第十一章 PT

11.1. 功能介绍

11.1.1. 点位模式

PT（点位）模式使用一系列“位置、时间”数据点描述运动过程，用户需要将速度曲线分割成若干段，如图所示：



整个运动过程分为 4 段，位移和时间分别为 P1、P2、P3、P4 和 T1、T2、T3、T4。在 T1 时间段内，起始速度为 0，终点速度为 2*P1/T1；在 T2 时间段内，速度始终为 2*P1/T1；在 T3 时间段内，起始速度为 2*P1/T1，终点速度为 2*P4/T4；在 T4 时间段内，起始速度为 2*P4/T4，终点速度为 0。

PT 接口参考：

adt_set_pt_mode
adt_set_pt_data
adt_set_pt_table
adt_clr_pt_data
adt_get_pt_space
adt_get_pt_index
adt_pt_start
adt_set_pt_loop/adt_get_pt_loop

第十二章 回零控制

12.1. 功能介绍

12.1.1. DSP 回零

控制卡为用户提供了**重复精度<1 脉冲**的二次回零功能。

在进行精确的运动控制之前，需要设置每个驱动轴的原点。绝大多数运动平台上都会设有原点传感器或可用作原点传感器的开关信号(如正、负限位传感器与原点传感器复用等)，寻找原点传感器的位置，并将该位置设置为当前驱动轴的坐标原点的过程即为回零。

控制卡提供的回零功能适用于直线模块、圆周模块等多种模组的回零，可控制回零方向、回零信号(原点传感器 STOP0)的启用和禁用、原点停靠在传感器的哪一侧、回零过程中各个硬件信号的启用禁用以及有效电平、回零过程速度、原点偏移等参数。

回零过程中高速搜索原点信号位置时，首次触及硬件正负限位(LMT+/LMT-)，则回零轴自动反向继续搜索原点信号位置。若再次触及硬件正负限位(LMT+/LMT-)或任意时刻触及硬件停止信号(EMGN)，或是任意时刻发生外部减速或紧急停止，回零过程将终止并返回相应错误码。

部分工艺中考虑到成本或其他外部因素，用户可能为模组做如下装配：

①装配硬件正负限位传感器，无硬件原点传感器

回零过程必须有可用的原点信号，须将端子板处原点信号 STOP0 与硬件负限位(或硬件正限位)信号短接以保证有可用的机械原点信号。与硬件负限位短接时，须合理设置原点停靠在硬件负限位传感器靠近硬件正限位传感器的一侧；与硬件正限位短接时，须合理设置原点停靠在硬件正限位传感器靠近硬件负限位传感器的一侧。

②装配硬件正限位传感器和硬件原点传感器，无硬件负限位传感器

多数如此装配的目的，是在回零完成后将硬件原点作硬件负限位复用。如此则须在回零完成后启用硬件原点停止信号(adt_set_stop0_mode)，以确保硬件原点信号触发时停止当前轴驱动。但硬件原点信号终究与硬件负限位信号有差异，即当硬件负限位信号触发时，当前轴可往正方向驱动，而硬件原点信号触发时，由于无法确定触发时当前轴的驱动方向，当前轴将无法以任何方式驱动，只能禁用硬件原点信号后驱动轴离开信号触发位置，然后再启用之，或者关闭轴使能，人工推动轴离开硬件原点信号触发位置。

若要在不装配硬件负限位传感器时完整使用硬件负限位功能，可在端子板将硬件负限位与硬件原点信号短接并设置同样的有效电平，如此则回零完成后，硬件原点信号可作为硬件负限位信号复用。

装配硬件负限位传感器和硬件原点传感器，无硬件正限位传感器的情况与此类似，不再赘述。

③装配硬件原点传感器，无硬件正负限位传感器

这种装配方式暂无更好的方法保证回零过程的万无一失，只能依赖人为保证模组始终位于原点信号传感器的同一侧或通过其他方式始终保证回零方向的正确。可在端子板将原点信号与硬件正限位或硬件负限位信号其中之一短接，回零后使能其中一个硬件限位信号增加设备安全性，切勿将原点信号与硬件正负限位同时短接。

DSP 回零接口参考

adt_set_home_mode/adt_set_home_speed

adt_set_home_process/adt_get_home_status

12.1.2. 总线伺服回零

EtherCAT 总线协议的伺服驱动器，都支持一些标准的内部回零模式。

总线伺服内部回零模式有[0,35]共计 36 种。每种回零模式的含义可参考具体总线伺服说明书或 CIA402 标准协议。

控制卡提供了支持 CIA402 协议的 36 种回零模式的接口，参考

adt_set_servo_home_para/ adt_set_servo_home

adt_get_servo_home_status / adt_stop_servo_home

小提示	并非所有的伺服都拥有完全的 36 种回零方式，具体的应用需要查询伺服说明书后再进行具体应用 若要使用总线伺服回零需要将正、负限位、原点映射到对应轴伺服输入点中去，需要与总线伺服自身设定的端口相对应（相关的限位与原点信号应接到总线伺服设定的相应端口上

12.2. 例程 12-1 DSP 回零

```
int _tmain(int argc, _TCHAR* argv[])
{
    int crd_count = 0, crd_index[10] = {0};
    //初始化运动控制卡
    VERIFY_RETURN_MSG(adt_initial(&crd_count, 0), "adt_initial", 1);
```

```
    printf("共检测到 %d 张运动控制卡!\n", crd_count);
    //查询可用控制卡索引
    VERIFY_RETURN_MSG(adt_get_card_index(&crd_count, crd_index), "adt_get_card_index"
, 1);
    //总线从站配置，以配置台 ADTECH EtherCAT 总线伺服驱动器 QXE+2 台 ADTECH EtherCAT 总线 IO
模块 ADT-ET1616A 为例
    int card = crd_index[0];//第一块可用 ADT-632XE 总线控制卡索引
    //停止通讯，防止后台有通讯占用
    VERIFY_RETURN_MSG (adt_ecat_stop(card), "adt_ecat_stop", 1);
    //加载配置
    VERIFY_RETURN_MSG(adt_ecat_load_flash_cfg(card), "adt_ecat_load_flash_cfg", 1);
    //开启通讯
    VERIFY_RETURN_MSG (adt_ecat_start(card), "adt_ecat_start", 1);
    //伺服从站使能
    VERIFY_RETURN_MSG (adt_set_axis_enable(card, 1, 1), "adt_set_axis_enable", 1);
    printf("伺服从站使能完成!\n");
    //mode = 0 -- 负向回零，固定正向退出原点，然后负方向接近原点
    //            STOP0 信号作为机械原点、STOP1 信号作为编码器原点
    //stop0 = 1 -- 机械原点高电平有效
    //limit = 0 -- 硬件正负限位不启用，低电平有效
    //back_range = 20 -- 退出机械原点距离设置为 mm
    //offset = 5 -- 原点偏移 mm
    VERIFY_RETURN_MSG(adt_set_home_mode(card, 1, 0, 1, 0, 20, 5), "adt_set_home_mode"
, 1);
    VERIFY_RETURN_MSG(adt_set_home_speed(card, 1, 0, 20, 5, 40), "adt_set_home_speed"
, 1);
    //执行确认
    CONFIRM_DRIVE(1);
    //开始回零
    int stt = 0;
    VERIFY_RETURN_MSG (adt_set_home_process(card, 1), "adt_set_home_process", 1);
    do
```

```
    {
        stt = adt_get_home_status(card, 1);
    } while (stt > 0);
    VERIFY_RETURN_MSG (stt, "adt_get_home_status", 1);
    system("pause");
    return 0;
}
```

12.3. 例程 12-2 总线伺服回零

```
int _tmain(int argc, _TCHAR* argv[])
{
    int crd_count = 0, crd_index[10] = {0};
    //初始化运动控制卡
    VERIFY_RETURN_MSG(adt_initial(&crd_count, 0), "adt_initial", 1);
    printf("共检测到 %d 张运动控制卡!\n", crd_count);
    //查询可用控制卡索引
    VERIFY_RETURN_MSG(adt_get_card_index(&crd_count, crd_index), "adt_get_card_index"
, 1);
    //总线从站配置，以配置台 ADTECH EtherCAT 总线伺服驱动器 QXE+2 台 ADTECH EtherCAT 总线 IO
模块 ADT-ET1616A 为例
    int card = crd_index[0];//第一块可用 ADT-632XE 总线控制卡索引
    //停止通讯，防止后台有通讯占用
    VERIFY_RETURN_MSG (adt_ecat_stop(card), "adt_ecat_stop", 1);
    //加载配置
    VERIFY_RETURN_MSG(adt_ecat_load_flash_cfg(card), "adt_ecat_load_flash_cfg", 1);
    //开启通讯
    VERIFY_RETURN_MSG (adt_ecat_start(card), "adt_ecat_start", 1);
    //伺服从站使能
    VERIFY_RETURN_MSG (adt_set_axis_enable(card, 1, 1), "adt_set_axis_enable", 1);
    printf("伺服从站使能完成!\n");
    //执行确认
    CONFIRM_DRIVE(1);
```

```
    //回零偏移，第一回零速度 pulse/s，第二回零速度 pulse/s，回零加速度 pulse/s2
    VERIFY_RETURN_MSG(adt_set_servo_home_para(card, 1, 0, 50000, 10000, 400000), "adt
_set_servo_home_para",1);
    //回零模式，即当前位置负向驱动至第一个 Z 相信号停止
    VERIFY_RETURN_MSG (adt_set_servo_home(card, 1, 33), "adt_set_servo_home",1);
    //开始回零
    int stt = 0;
    do
    {
        stt = adt_get_servo_home_status(card, 1);
    } while (stt > 0);
    switch(stt)
    {
    case 0:
        printf_s("伺服内部回零完成!\n");
        break;
    case -1:
        printf_s("伺服内部回零命令错误!\n");
        break;
    case -2:
        printf_s("伺服内部回零过程出错或者回零超时!\n");
        break;
    case -3:
        printf_s("伺服内部回零伺服报警!\n");
        break;
    default:
        printf_s("伺服内部回零未知错误!\n");
        break;
    }
    system("pause");
    return 0;
}
```

第十三章 总线 IO 控制

13.1. 功能介绍

总线 IO 多分布在总线伺服自带 IO、总线扩展 IO 模块，以及为实现特殊功能定制的其他总线通讯模块，例如：总线转脉冲模块。

13.1.1. 伺服 IO 控制

总线伺服驱动器通常会自带一些 IO 用于限位、原点、使能、报警清除等用途。控制卡提供了伺服输入、输出端口的读取接口

adt_read_servo_inbit/adt_read_servo_inport

分别用于读取单个输入端口和读取一组输入端口(16 个一组)。

adt_read_servo_outbit/adt_write_servo_outbit

分别用于读取单个输入端口和写单个输出端口(16 个一组)。

小提示	并非所有的伺服（步进）都可以对其自带的 IO 进行控制，具体需要跟据实际的伺服（步进）驱动器情况而定

13.1.2. 总线 IO 模块控制

总线运动控制通常会使用总线 IO 模块满足系统对 IO 的控制。

ADT-632XE 可以适配遵循 RTEX 协议的任何总线 IO 模块。

例如，众为兴推出总线 IO 模块 ET-1616A，含 16 个通用输入端口和 16 个通用输出端口。

小提示	总线 IO 模块上的输入端口可以作为总线伺服驱动器的硬件限位和机械原点信号的映射使用

总线 IO 模块控制接口参考

adt_read_busio_inbit/adt_read_busio_inport

adt_read_busio_outbit/adt_read_busio_outport

adt_write_busio_outbit/adt_write_busio_outport

分别用于控制单个端口和控制一组端口(16 个一组)。

第十四章 缓存事件控制

14.1. 功能介绍

14.1.1. 缓存输出控制

缓存输出控制即在缓存插补过程中实现指定输出端口的无缝控制。

缓存输出控制时还可约束输出控制时的最大速度，即缓存插补时已达到最大速度 V0 后，用户约定在指定插补轴到达空间坐标 P 时，控制输出端口 B，且要求插补轴在 P 点的速度须<=V1。若满足 V0<=V1，则缓存输出控制时插补轴驱动速度不会发生变化，若满足 V0>V1，则插补轴会在到达位置 P 之前依据既定设置减速，确保到达位置 P 时速度为 V1，且对端口 B 做输出控制。缓存输出控制完成后，插补轴将按照剩余缓存插补轨迹重新规划剩余插补的驱动速度。

缓存输出控制接口参考

adt_set_fifo_outbit/adt_set_fifo_multi_io

分别用于缓存 1 路或 1 组输出控制，1 组=16 路。

14.1.2. 缓存延时控制

缓存延时控制暂停缓存插补驱动的行为，即缓存驱动将被停止，当前缓存 IO 事件将持续执行，但暂时不会继续执行下一条缓存 IO 事件指令。状态维持指定时间后，恢复缓存驱动或缓存 IO 控制。其优势在于可以在缓存控制期间更精准更便捷地响应延时控制。缓存延时控制精度可达 ms 级。

缓存延时控制接口参考

adt_set_fifo_delay

14.1.3. 缓存运动暂停等待输入

在缓存运动中，可以通过 adt_set_fifo_wait_inbit 进行输入信号等待控制，当缓存运动执行至此后，需要等待设置的输入信号有效后（或超出设定的等待时间）才能进行一步。

14.1.4. 缓存事件余量

控制卡为用户提供了 1000 条缓存控制事件容量，即高速多段缓存插补驱动过程中，同时最多可容纳 1000 条缓存控制事件。

小提示	缓存 IO 事件的 1000 条缓存容量与缓存插补的 5000 条缓存容量是相互独立的。

当缓存控制事件被用尽，则用户需要等待有缓存 IO 事件被执行，缓存事

件余量有空余之后，才能继续缓存下一条缓存 IO 控制事件。

用户可以通过接口 adt_get_fifo_event_len 查询当前缓存事件余量。

14.1.5. 缓存事件清空

可以通过调用adt_clear_fifo_event清空缓存事件空间的剩余缓存事件指令。

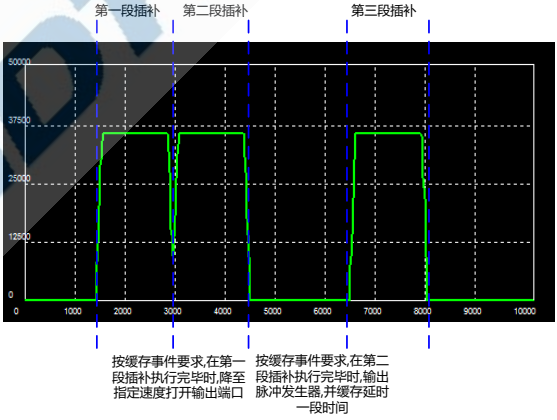
小提示	缓存 IO 事件，自然是要嵌入缓存驱动过程中的，所以要开启速度前瞻，要在一组缓存驱动中执行缓存 IO 事件才能体现其功能优势。

14.2. 例程 14-1 缓存事件控制

```
int _tmain(int argc, _TCHAR* argv[])
{
    int crd_count = 0, crd_index[10] = {0};
    //初始化运动控制卡
    VERIFY_RETURN_MSG(adt_initial(&crd_count, 0), "adt_initial", 1);
    printf("共检测到 %d 张运动控制卡!\n", crd_count);
    //查询可用控制卡索引
    VERIFY_RETURN_MSG(adt_get_card_index(&crd_count, crd_index), "adt_get_card_index"
, 1);
    //总线从站配置，以配置台 ADTECH EtherCAT 总线伺服驱动器 QXE+2 台 ADTECH EtherCAT 总线 IO
模块 ADT-ET1616A 为例
    int card = crd_index[0];//第一块可用 ADT-632XE 总线控制卡索引
    //停止通讯，防止后台有通讯占用
    VERIFY_RETURN_MSG (adt_ecat_stop(card), "adt_ecat_stop", 1);
    //加载配置
    VERIFY_RETURN_MSG(adt_ecat_load_flash_cfg(card), "adt_ecat_load_flash_cfg", 1);
    //开启通讯
    VERIFY_RETURN_MSG (adt_ecat_start(card), "adt_ecat_start", 1);
    //伺服从站使能
    VERIFY_RETURN_MSG (adt_set_axis_enable(card, 1, 1), "adt_set_axis_enable", 1);
    VERIFY_RETURN_MSG (adt_set_axis_enable(card, 2, 1), "adt_set_axis_enable", 1);
    printf("伺服从站使能完成!\n");
    //编程模式设置,使用的插补轴也要设置编程模式
```

```
    VERIFY_RETURN_MSG (adt_set_unit_mode(card, 1, 0), "adt_set_unit_mode", 1);
    VERIFY_RETURN_MSG (adt_set_unit_mode (card, 2, 0), "adt_set_unit_mode", 1);
    //速度设置
    VERIFY_RETURN_MSG (adt_set_startv(card, INPA_AXIS, 10), "adt_set_startv", 1);
    VERIFY_RETURN_MSG (adt_set_maxv(card, INPA_AXIS, 50), "adt_set_maxv", 1);
    VERIFY_RETURN_MSG (adt_set_acc(card, INPA_AXIS, 400), "adt_set_acc", 1);
//建立插补坐标系
    int axs_list[2] = {1,2};
    VERIFY_RETURN_MSG(adt_set_inp_coordinate(card,INPA_AXIS,2,axs_list), "VERIFY_RETU
RN_MSG", 1);
    //依据用户输入确定是否启用速度前瞻
    int mode = 0;
    printf("是否启用速度前瞻? 1--启用 0--不启用\n");
    cin>>mode;
    VERIFY_RETURN_MSG(adt_set_speed_pretreat_mode(card, INPA_AXIS, mode), "adt_set_sp
eed_pretreat_mode", 1);
    //执行确认
    CONFIRM_DRIVE(0);
    //确认插补缓存空间
    int inp_len = 0;
    int fifo_len = 0;
    do
    {
        VERIFY_RETURN_MSG(adt_get_inp_fifo_len(card, INPA_AXIS, &inp_len), "adt_get_i
np_fifo_len", 1);
        VERIFY_RETURN_MSG(adt_get_fifo_event_len(card, INPA_AXIS, &fifo_len), "adt_ge
t_fifo_event_len", 1);
    } while (inp_len < 100 || fifo_len == 0);
    int inp_list[] = {1,2};//插补坐标系中进行插补运动的轴序号
    //---------------第一段缓存插补---------------//
    double pos_list[] = {50, 50};
```

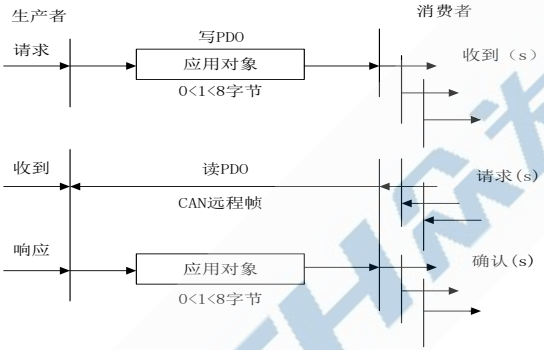
```
    VERIFY_RETURN_MSG(adt_inp_move_unit(card, INPA_AXIS, 0, 2, inp_list, pos_list, 0)
, "adt_inp_move_unit", 1);
    //-----缓存输出----//
    //指定 IO 节点打开端口，打开时速度保持在 mm/s
    VERIFY_RETURN_MSG(adt_set_fifo_outbit(card, INPA_AXIS, 0, 1, 0, 1, 20), "adt_set_
fifo_outbit", 1);
    //第二段缓存插补
    VERIFY_RETURN_MSG(adt_inp_move_unit(card, INPA_AXIS, 0, 2, inp_list, pos_list, 0)
, "adt_inp_move_unit", 1);
    //缓存状态保持 ms，此时缓存驱动将静止，缓存输出状态将保持直至完成
    VERIFY_RETURN_MSG(adt_set_fifo_delay(card, INPA_AXIS, 0, 2000), "adt_set_fifo_del
ay", 1);
    //第三段缓存插补
    VERIFY_RETURN_MSG(adt_inp_move_unit(card, INPA_AXIS, 0, 2, axs_list, pos_list, 0)
, "adt_inp_move_unit", 1);
    //等待插补轴驱动停止
    VERIFY_RETURN_MSG (adt_till_stop(card, INPA_AXIS), "adt_till_stop", 1);
    system("pause");
    return 0;
}
```



第十五章 SDO/PDO

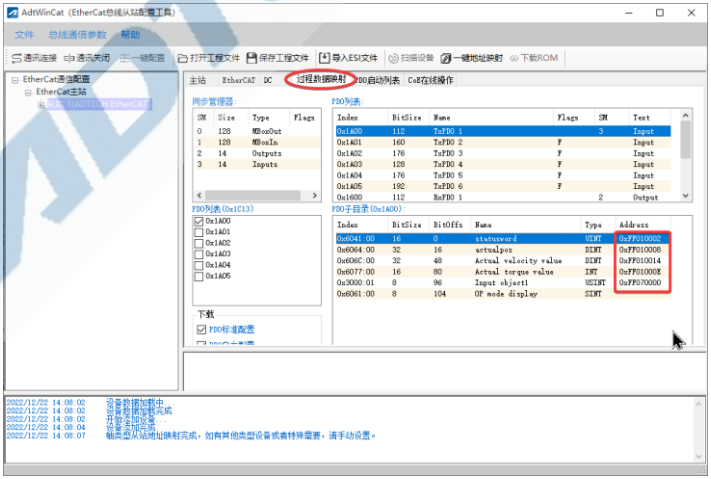
15.1. 功能介绍

PDO：过程数据对象（Process Data Object），过程数据的发送，实时、速度快，提供对设备应用对象的直接访问通道，它用来传输实时短帧数据，具有较高的优先权。PDO 传输的数据必须少于或等于 8 个字节，在应用层上不包含传输控制信息，报文利用率极高。



PDO 通信是基于生产者/消费者模型(如上图)，主要用于传输实时数据。产生数据的节点将带有自己节点号的数据放到总线上，需要该数据的节点，可以配置为接收该 PDO。

PDO 的地址由 AdtWinCat 中进行配置得到如下图所示

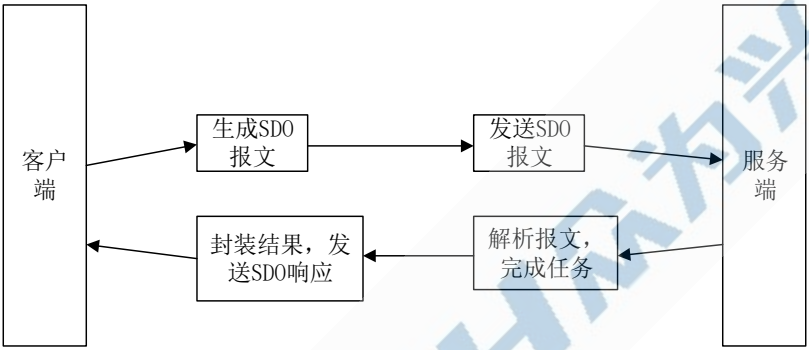


用户控制算子参考：

adt_ecat_pdo_read

adt_ecat_pdo_write

SDO：服务数据对象（Service Data Obiect），服务数据的发送接收，实时性要求不高，SDO 一般用来配置和获得节点的配置参数（即主要用于从站的配置），其优先级只比心跳(Heartbeat)高。



SDO 不能实现从节点之间数据的直接传送，SDO 通讯可以描述为客户/服务器模型，SDO 的客户/服务器通讯模式（如上图），两节点中请求进行读写操作的节点为客户端节点，数据被读写的节点为服务器节点，SDO 通讯用于主节点对从节点对象字典的读写访问，以实现对从节点参数的设置，下载程序以及定义 PDO 通讯类型和数据格式等。

用户控制算子参考：

adt_ecat_sdo_read

adt_ecat_sdo_write

小提示	SDO 的地址由从站的 ESI 文件（从站设备信息描术文件）中提供。

第十六章 用户控制

16.1. 功能介绍

控制卡提供用户密码控制，控制卡设置密码后，若校验失败，控制卡将无法被正确使用。

控制卡出厂时无密码，如需使用密码控制，则用户需要先进行密码设定，密码长度须在 128 位以内，且新旧密码不能相同。

如果忘记密码，需要寄回原厂破解，所以密码设定后请自行备份。

用户可以在应用软件开启时加入密码校验加密，密码校验失败超过 3 次将锁卡，若需继续校验，须断电重启控制卡，即对 PC 进行关机后再进行启动。

控制卡第一次使用用户控制时，不会校验旧密码，可随意输入；一旦设置用户控制密码成功，则以后均须校验。

用户控制算子参考

adt_set_user_password – 修改/设置用户密码

adt_check_password – 用户密码校验

小提示	用户控制功能仅仅是为应用程序提供了诸如登录、权限校验等功能的便捷方案，校验不通过并不会影响控制卡控制功能的正常使用。

16.2. 例程 16-1 用户控制

```
int _tmain(int argc, _TCHAR* argv[])
{
    int crd_count = 0, crd_index[10] = {0};
    //初始化运动控制卡
    VERIFY_RETURN_MSG(adt_initial(&crd_count, 0), "adt_initial", 1);
    printf("共检测到 %d 张运动控制卡!\n", crd_count);
    //查询可用控制卡索引
    VERIFY_RETURN_MSG(adt_get_card_index(&crd_count, crd_index), "adt_get_card_index"
, 1);
    int card = crd_index[0];//第一块可用 ADT-632XE 总线控制卡索引
    //用户控制，设置密码为 adtech
    VERIFY_RETURN_MSG(adt_set_user_password(card, "", "adtech"), "adt_set_user_passwo
rd", 1);
```

```
    printf("用户密码设置为'adtech'\n");
    int retn = adt_check_password(card, "111");
    printf("密码'111'校验");
    retn? printf("失败!\n"):printf("成功!\n");
    retn = adt_check_password(card, "adtech");
    printf("密码'adtech'校验");
    retn? printf("失败!\n"):printf("成功!\n");
    //设置密码为空
    VERIFY_RETURN_MSG(adt_set_user_password(card, "adtech", ""), "adt_set_user_passwo
rd", 1);
    system("pause");
    return 0;
}
```

第十七章 ADT-9188 扩展板控制

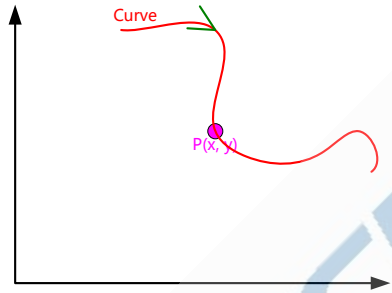
17.1. 功能介绍

17.1.1. 位置比较

控制卡目前提供一到二维位置比较功能，下面针对二维位置比较进行说明。

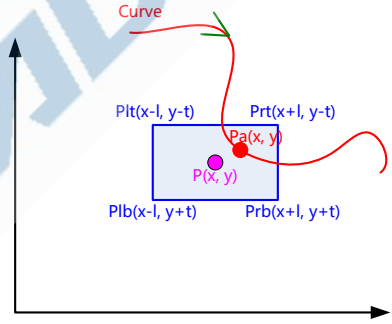
二维位置比较功能描述为平面轨迹中，XY 到达指定位置点，或到达指定位置点附近指定区域范围内最接近指定位置点的坐标后，实施高速输出端口实时控制的行为。指定位置的描述，可以是逻辑位置，也可以是编码器位置。

例如下图



二维坐标系下，假设 P(x,y)为理论的 2D 位置比较输出点，Curve 为经过二维位置比较输出坐标的驱动轨迹，则 ADT-9188 将按照预先设定的控制方式，在 Curve 经过 P(x,y)的瞬间，立即对绑定比较器的输出端口实施硬件级控制。

倘若 Curve 并不一定会经过 P(x,y)，比如编码器位置的抖动导致 P(x,y)总是不能被准确到达，此时二维位置比较仍旧可能被有效执行，如下图所示



二维位置比较允许在指定位置点设置运算区域，该区域为可设置长宽的矩

形，当 Curve 轨迹进入矩形区域时，二维位置比较运算才真正开始，若 Curve 并未准确经过 P(x,y)，Curve 将被描述为一种逐渐靠近 P(x,y)后又逐渐远离 P(x,y)的趋势，2D 位置比较的触发点将是 Curve 最接近 P(x,y)后发生远离的第一个点 Pa(x,y)。

小提示	总线运动控制卡的二维位置比较功能依托于 ADTECH 定制的总线转脉冲模块 ADT-9188 专用的高速输出端口。所以，若要使用总线运动控制卡实现二维位置比较功能，须配套使用 ADT-9188 模块。 二维位置比较触发的条件仅与轨迹是否与位置比较运算区域有交点有关，与轨迹从哪个方向或矢量进入位置比较运算区域无关。

17.1.2. 高速 IO 端口

ADT9188 模块携带 8 个高速输入端口和 8 个高速输出端口，支持单个输入端口状态读取 adt_read_inbit 和输入端口状态按组读取 adt_read_inport。支持单个输出端口状态的写入 adt_write_outbit 和单个输出端口状态的读取 adt_read_outbit。高速输出端口能够使总线卡实现位置比较与飞拍等高阶功能。

17.2. 例程 17-1 2D 位置比较

```
int _tmain(int argc, _TCHAR* argv[])
{
    int crd_count = 0, crd_index[10] = {0};
    //初始化运动控制卡
    VERIFY_RETURN_MSG(adt_initial(&crd_count, 0), "adt_initial", 1);
    printf("共检测到 %d 张运动控制卡!\n", crd_count);
    //查询可用控制卡索引
    VERIFY_RETURN_MSG(adt_get_card_index(&crd_count, crd_index), "adt_get_card_index"
, 1);
    //总线从站配置，以配置台 ADTECH EtherCAT 总线伺服驱动器 QXE+2 台 ADTECH EtherCAT 总线 IO
模块 ADT-ET1616A 为例
    int card = crd_index[0];//第一块可用 ADT-632XE 总线控制卡索引
    //停止通讯，防止后台有通讯占用
    VERIFY_RETURN_MSG (adt_ecat_stop(card), "adt_ecat_stop", 1);
    //加载配置
```

```
    VERIFY_RETURN_MSG(adt_ecat_load_flash_cfg(card), "adt_ecat_load_flash_cfg", 1);
    //开启通讯
    VERIFY_RETURN_MSG (adt_ecat_start(card), "adt_ecat_start", 1);
    //伺服从站使能
    VERIFY_RETURN_MSG (adt_set_axis_enable(card, 1, 1), "adt_set_axis_enable", 1);
    VERIFY_RETURN_MSG (adt_set_axis_enable(card, 2, 1), "adt_set_axis_enable", 1);
    printf("伺服从站使能完成!\n");
    //编程模式设置,使用的插补轴也要设置编程模式
    VERIFY_RETURN_MSG (adt_set_unit_mode(card, 1, 0), "adt_set_unit_mode", 1);
    VERIFY_RETURN_MSG (adt_set_unit_mode (card, 2, 0), "adt_set_unit_mode", 1);
    VERIFY_RETURN_MSG(adt_set_bus_axis_gear_ratio(card, 1, 131072, 10000), "adt_set_b
us_axis_gear_ratio", 1);
    VERIFY_RETURN_MSG(adt_set_bus_axis_gear_ratio(card, 2, 131072, 10000), "adt_set_b
us_axis_gear_ratio", 1);
    VERIFY_RETURN_MSG(adt_set_pulse_equiv(card, 1, 1000), "adt_set_pulse_equiv", 1);
//伺服脉冲当量为 1000
    VERIFY_RETURN_MSG(adt_set_pulse_equiv(card, 2, 1000), "adt_set_pulse_equiv", 1);
//伺服脉冲当量为 1000
    //速度设置
    VERIFY_RETURN_MSG (adt_set_startv(card, INPA_AXIS, 10), "adt_set_startv", 1);
    VERIFY_RETURN_MSG (adt_set_maxv(card, INPA_AXIS, 50), "adt_set_maxv", 1);
    VERIFY_RETURN_MSG (adt_set_acc(card, INPA_AXIS, 400), "adt_set_acc", 1);
    //建立插补坐标系
    int axs_list[2] = {1,2};
    VERIFY_RETURN_MSG(adt_set_inp_coordinate(card,INPA_AXIS,2,axs_list), "VERIFY_RETU
RN_MSG", 1);
    //依据用户输入确定是否启用速度前瞻
    int mode = 0;
    printf("是否启用速度前瞻? 1--启用 0--不启用\n");
    cin>>mode;
    VERIFY_RETURN_MSG(adt_set_speed_pretreat_mode(card, INPA_AXIS, mode), "adt_set_sp
eed_pretreat_mode", 1);
```

```
    //执行确认
    CONFIRM_DRIVE(0);
    //2D 比较设置，9188 接线板编码器位置依据，采用 0 号 OUT 口作为位置比较输出，输出模式为反转模式
    int cmp_axs_list[] = {1,2};//9188 接线板对应的编码器接收轴的序号
    VERIFY_RETURN_MSG(adt_set_compare_xd_mode(card, 0, 1, 2, cmp_axs_list, 1, 0, 0), "adt_set_compare_xd_mode", 1);
    //2D 位置比较运算区域为长宽的矩形区域
    long area[] = {50, 50};
    VERIFY_RETURN_MSG(adt_set_compare_xd_area(card, 0, area), "adt_set_compare_xd_area", 1);
    //添加比较点，脉冲时间设置为 10ms
    long pos1[2] = {10000, 10000}, pos2[2] = {20000, 20000}, pos3[2] = {30000, 30000};
    VERIFY_RETURN_MSG(adt_add_compare_xd_point(card, 0, pos1, 10, 10), "adt_add_compare_xd_point", 1);
    VERIFY_RETURN_MSG(adt_add_compare_xd_point(card, 0, pos2, 10, 10), "adt_add_compare_xd_point", 1);
    VERIFY_RETURN_MSG(adt_add_compare_xd_point(card, 0, pos3, 10, 10), "adt_add_compare_xd_point", 1);
    //驱动确认
    CONFIRM_DRIVE(1);
    //确认插补缓存空间
    int inp_len = 0;
    do
    {
        VERIFY_RETURN_MSG(adt_get_inp_fifo_len(card, INPA_AXIS, &inp_len), "adt_get_inp_fifo_len", 1);
    } while (inp_len < 100);
    int inp_list[] = {1,2};//插补坐标系中进行插补运动的轴序号
    double pos_list[] = {50, 50};
```

```
    VERIFY_RETURN_MSG(adt_inp_move_unit(card, INPA_AXIS, 0, 2, inp_list, pos_list, 0)
, "adt_inp_move_unit", 1);
    //等待插补轴驱动停止
    VERIFY_RETURN_MSG (adt_till_stop(card, INPA_AXIS), "adt_till_stop", 1);
    //检查位置比较器
    int  status = 0;
    VERIFY_RETURN_MSG(adt_get_compare_status(card, 0, &status), "adt_get_compare_stat
us", 1);
    system("pause");
    return 0;
}
```

第十八章 标准指令详述

18.1. 基础控制类

adt_initial

指令格式			
int _stdcall adt_initial(int *count, int mode=0)			
指令描述			
控制卡初始化，只有在调用该函数对运动控制卡初始化成功后，再调用其他函数才有意义。			
参数说明			
名称	**数据类型**	**范围**	**说明**
count	int*	[0-10]	系统可用运动控制卡数量
mode	int	[0,1]	备用参数，默认 0
返回值			
0 表示执行成功，其他返回值表示执行失败，失败原因参考标准错误码			
指令分类	指令列表_基础控制类		
相关章节	控制卡初始化		
相关例程	例程 4-1 基础控制与版本信息获取		

adt_close_card

指令格式	
int _stdcall adt_close_card()	
指令描述	
控制卡使用完成后，需关闭运动控制卡以释放控制卡资源	
参数说明	
无	
返回值	
0 表示执行成功，其他返回值表示执行失败，失败原因参考标准错误码	
指令分类	指令列表_基础控制类
相关章节	控制卡关闭
相关例程	例程 4-1 基础控制与版本信息获取

adt_reset_card

指令格式
int _stdcall adt_reset_card(int cardno)
指令描述
复位运动控制卡 1. 常用于清除硬件急停信号、清除异常停止信息、清除无效缓存数据、清除运动库停止信息，当硬件急停功能启用，且硬件急停信号触发时，须使用复位运动控制卡清除硬件急停锁定后，轴才能正常驱动。 2. 当系统发生运动库异常，问题已排查且解决后，须使用复位运动控制卡清除运动库异常信息后，系统才能正常驱动。

参数说明			
名称	**数据类型**	**范围**	**说明**
cardno	int	[0-9]	控制卡索引号

返回值
0 表示执行成功，其他返回值表示执行失败，失败原因参考标准错误码

指令分类	指令列表_基础控制类
相关章节	控制卡复位
相关例程	

adt_soft_reboot

指令格式
int _stdcall adt_soft_reboot(int cardno)
指令描述
软件重启控制卡 1. 首先减速停止当前控制卡的所有驱动并清空缓存，然后复位控制卡所有数据为上电初始状态。 2. 软件重启控制卡并不会使控制卡断电。 3. 软件重启控制卡后，须重启应用软件并重新初始化，才能继续正常使用控制卡

参数说明			
名称	**数据类型**	**范围**	**说明**
cardno	int	[0-9]	控制卡索引号

返回值

0 表示执行成功，其他返回值表示执行失败，失败原因参考标准错误码	
指令分类	指令列表_基础控制类
相关章节	控制卡软件重启
相关例程	

adt_get_card_index

指令格式			
int _stdcall adt_get_card_index(int *count, int index[10])			
指令描述			
在初始化后获取当前系统的可用控制卡数量及其索引。			
参数说明			
名称	**数据类型**	**范围**	**说明**
count	int*		当前系统可用运动控制卡数量
index	int*		当前系统可用运动控制卡索引，数组尺寸须大于当前系统可用运动控制卡数量
返回值			
0 表示执行成功，其他返回值表示执行失败，失败原因参考标准错误码			
指令分类	指令列表_基础控制类		
相关章节	可用控制卡索引获取		
相关例程	例程 4-1 基础控制与版本信息获取		

adt_get_motion_error

指令格式			
int _stdcall adt_get_motion_error(int cardno, int *errno)			
指令描述			
获取当前运动控制卡运动库异常错误号。			
参数说明			
名称	**数据类型**	**范围**	**说明**
cardno	int	[0-9]	控制卡索引号
errno	int*		运动库异常错误号
返回值			
0 表示执行成功，其他返回值表示执行失败，失败原因参考标准错误码			
指令分类	指令列表_基础控制类		

相关章节	
相关例程	

18.2. 版本信息类

adt_get_lib_ver

指令格式			
int _stdcall adt_get_lib_ver(int cardno, int *lib)			
指令描述			
获取当前可用控制卡 LIB 库版本号。			
参数说明			
名称	**数据类型**	**范围**	**说明**
cardno	int	[0-9]	控制卡索引号
lib	int*		当前可用控制卡 LIB 库版本号
返回值			
0 表示执行成功，其他返回值表示执行失败，失败原因参考标准错误码			
指令分类	指令列表_版本信息类		
相关章节	控制卡版本信息获取		
相关例程	例程 4-1 基础控制与版本信息获取		

adt_get_motion_ver

指令格式			
int _stdcall adt_get_motion_ver(int cardno, int * motion)			
指令描述			
获取当前可用控制卡 LIB 库版本号。			
参数说明			
名称	**数据类型**	**范围**	**说明**
cardno	int	[0-9]	控制卡索引号
motion	int*		当前可用控制卡 MOTION 库版本号
返回值			
0 表示执行成功，其他返回值表示执行失败，失败原因参考标准错误码			
指令分类	指令列表_版本信息类		
相关章节	控制卡版本信息获取		
相关例程	例程 4-1 基础控制与版本信息获取		

adt_get_firmware_ver

指令格式			
int _stdcall adt_get_firmware_ver(int cardno, int *fmw)			
指令描述			
获取当前可用控制卡固件版本号。			
参数说明			
名称	**数据类型**	**范围**	**说明**
cardno	int	[0-9]	控制卡索引号
fmw	int*		当前可用控制卡固件版本号
返回值			
0 表示执行成功，其他返回值表示执行失败，失败原因参考标准错误码			
指令分类	指令列表_版本信息类		
相关章节	控制卡版本信息获取		
相关例程	例程 4-1 基础控制与版本信息获取		

18.3. 总线从站控制类

adt_ecat_download_cfg

指令格式			
int _stdcall adt_ecat_download_cfg(int cardno, const char *pPath)			
指令描述			
下发总线启动配置文件			
参数说明			
名称	**数据类型**	**范围**	**说明**
cardno	int	[0-9]	控制卡索引号
pPath	Char*		配置文件路径
返回值			
0 表示执行成功，其他返回值表示执行失败，失败原因参考标准错误码			
指令分类	指令列表_总线从站控制类		
相关章节	从站通信		
相关例程	例程 5-2 从本地文件导入从站配置		

adt_ecat_load_flash_cfg

指令格式			
int _stdcall adt_ecat_load_flash_cfg (int cardno)			
指令描述			
加载控制卡存储的总线启动配置文件			
参数说明			
名称	**数据类型**	**范围**	**说明**
cardno	int	[0-9]	控制卡索引号
返回值			
0 表示执行成功，其他返回值表示执行失败，失败原因参考标准错误码			
指令分类	指令列表_总线从站控制类		
相关章节	从站通信		
相关例程	例程 5-1 从 RAM/ROM 中加载从站配置，解析配置，获取轴资源信息		

adt_ecat_stop

指令格式			
int _stdcall adt_ecat_stop(int cardno)			
指令描述			
终止总线通信			
参数说明			
名称	**数据类型**	**范围**	**说明**
cardno	int	[0-9]	控制卡索引号
返回值			
0 表示执行成功，其他返回值表示执行失败，失败原因参考标准错误码			
指令分类	指令列表_总线从站控制类		
相关章节	从站通信		
相关例程	例程 5-1 从 RAM/ROM 中加载从站配置，解析配置，获取轴资源信息		

adt_ecat_start

指令格式			
int _stdcall adt_ecat_start(int cardno)			
指令描述			
启动总线通信			
参数说明			
名称	**数据类型**	**范围**	**说明**
cardno	int	[0-9]	控制卡索引号
返回值			
0 表示执行成功，其他返回值表示执行失败，失败原因参考标准错误码			
指令分类	指令列表_总线从站控制类		
相关章节	从站通信		
相关例程	例程 5-1 从 RAM/ROM 中加载从站配置，解析配置，获取轴资源信息		

adt_get_ecat_status

指令格式			
int _stdcall adt_get_ecat_status（int cardno, int * pStatus）			
指令描述			
获取 EtherCat 总线运行状态			
参数说明			
名称	**数据类型**	**范围**	**说明**
cardno	int	[0-9]	控制卡索引号
pStatus	int		0:主站未初始化 1:主站进入 OP 状态 118:主站链接断线 119:总线轴报警
返回值			
0 表示执行成功，其他返回值表示执行失败，失败原因参考标准错误码			
指令分类	指令列表_总线从站控制类		
相关章节	从站通信		
相关例程			

adt_get_ecat_slave_resource

指令格式			
int _stdcall adt_get_ecat_slave_resource(int cardno, int *SlaveNum, int *ServoNum, int *IoNum)			
指令描述			
获取 Ethercat 从站资源信息			
参数说明			
名称	**数据类型**	**范围**	**说明**
cardno	int	[0-9]	控制卡索引号
SlaveNum	int*		从站节点数
ServoNum	int*		伺服节点数
IoNum	int*		IO 节点数，非伺服都认为是 IO 设备
返回值			
0 表示执行成功，其他返回值表示执行失败，失败原因参考标准错误码			

指令分类	指令列表_总线从站控制类
相关章节	从站通信
相关例程	例程 5-1 从 RAM/ROM 中加载从站配置，解析配置，获取轴资源信息

adt_get_ecat_real_slave_num

指令格式			
int _stdcall adt_get_ecat_real_slave_num (int cardno, int * pSlvNum)			
指令描述			
获取实时的从站节点数			
参数说明			
名称	数据类型	范围	说明
cardno	int	[0-9]	控制卡索引号
pSlvNum	int*		从站节点数
返回值			
0 表示执行成功，其他返回值表示执行失败，失败原因参考标准错误码			

指令分类	指令列表_总线从站控制类
相关章节	从站通信
相关例程	例程 5-1 从 RAM/ROM 中加载从站配置，解析配置，获取轴资源信息

adt_get_ecat_slave_info

指令格式			
int _stdcall adt_get_ecat_slave_info（int cardno, int node, int *type, int *AxNum, int *DiGrp, int *DoGrp）			
指令描述			
获取 Ethercat 从站节点信息			
参数说明			
名称	数据类型	范围	说明
cardno	int	[0-9]	控制卡索引号
node	int		节点号 ID
type	int		节点类型： 0，空节点 1，伺服设备 2，非伺服设备

AxNum	int		节点轴数，用作伺服设备统计轴通道资源数，用于获取多驱伺服轴数
DiGrp	int*		节点输入资源组数，1 组 8 位，不满 1 组按 1 组计算
DoGrp	int*		节点输出资源组数，1 组 8 位，不满 1 组按 1 组计算
返回值			
0 表示执行成功，其他返回值表示执行失败，失败原因参考标准错误码			
指令分类	指令列表_总线从站控制类		
相关章节	从站通信		
相关例程	例程 5-1 从 RAM/ROM 中加载从站配置，解析配置，获取轴资源信息		

adt_get_axis_ecat_node

指令格式			
int _stdcall adt_get_axis_ecat_node（int cardno, int axis, int *node, int *channel）			
指令描述			
获取轴号 Ethercat 从站节点映射			
参数说明			
名称	**数据类型**	**范围**	**说明**
cardno	int	[0-9]	控制卡索引号
axis	int		轴号
node	int*		节点号
channel	int*		节点上通道号从 1 开始，一般用于多驱伺服和 ECAT 脉冲转换板，底层最多支持 1 个节点 8 个通道判断，通过轴号获取总线伺服对应节点和节点通道，可以通过轴号访问绑定在轴资源上的总线设备。
返回值			
0 表示执行成功，其他返回值表示执行失败，失败原因参考标准错误码			
指令分类	指令列表_总线从站控制类		
相关章节	从站通信		
相关例程	例程 5-1 从 RAM/ROM 中加载从站配置，解析配置，获取轴资源信息		

adt_get_total_axis

指令格式			
int _stdcall adt_get_total_axis(int cardno, int *count)			
指令描述			
获取当前可用控制卡的可用实体轴数。			
参数说明			
名称	**数据类型**	**范围**	**说明**
cardno	int	[0-9]	控制卡索引号
count	int*		控制卡可用实体轴数量
返回值			
0 表示执行成功，其他返回值表示执行失败，失败原因参考标准错误码			
指令分类	指令列表_总线从站控制类		
相关章节	控制卡可用实体轴数获取		
相关例程			

18.4. 资源配置类

adt_set_axis_io_map

指令格式			
int _stdcall adt_set_axis_io_map(int cardno, int axis, int mode, int board, int port)			
指令描述			
自定义控制卡可用轴硬件信号映射			
参数说明			
名称	**数据类型**	**范围**	**说明**
cardno	int	[0-9]	控制卡索引号
axis	int	[1-MAXAXIS]	轴号
mode	int	[0-4]	硬件信号标识 0：正限位 1：负限位 2：原点信号(STOP0) 3：保留 4：急停(EMG)，此模式下，axis 轴参数无意义，IO 触发，所有轴立即停止
board	int	[0-32]	0：总线伺服自带 IO 1~16：总线 IO 模块节点号
port	int	[0-15]	映射端口号
返回值			
0 表示执行成功，其他返回值表示执行失败，失败原因参考标准错误码			
指令分类	指令列表_资源配置类		
相关章节	总线 IO 端口映射		
相关例程	例程 6-1 总线 IO 端口映射		

adt_get_axis_io_map

指令格式
int _stdcall adt_get_axis_io_map(int cardno, int axis, int mode, int *board, int *port)
指令描述
获取当前控制卡可用轴硬件信号映射
参数说明

名称	数据类型	范围	说明
cardno	int	[0-9]	控制卡索引号
axis	int	[1-MAXAXIS]	轴号
mode	int	[0-4]	硬件信号标识 0：正限位 1：负限位 2：原点信号(STOP0) 3：保留 4：硬件停止信号
board	int*		0：总线伺服自带 IO 1~16：总线 IO 模块节点号
port	int*		映射端口号
返回值			
0 表示执行成功，其他返回值表示执行失败，失败原因参考标准错误码			
指令分类	指令列表_资源配置类		
相关章节	总线 IO 端口映射		
相关例程			

adt_set_hardlimit_mode

指令格式			
int _stdcall adt_set_hardlimit_mode(int cardno, int axis, int mode, int enable, int logic, int admode)			
指令描述			
设置硬件限位输入模式			
参数说明			
名称	**数据类型**	**范围**	**说明**
cardno	int	[0-9]	控制卡索引号
axis	int	[1-MAXAXIS]	轴号
mode	int	[0-4]	硬件信号标识 0：正限位 1：负限位 2：原点信号(STOP0) 3：保留 4：硬件停止信号
enable	int	[0,1]	0：禁用 1：启用
logic	int	[0,1]	有效电平 0：低电平 1：高电平
admode	int	[0,1]	停止模式 0：减速停止 1：立即停止
返回值			
0 表示执行成功，其他返回值表示执行失败，失败原因参考标准错误码			
指令分类	指令列表_资源配置类		
相关章节			
相关例程			

adt_get_hardlimit_mode

指令格式			
int _stdcall adt_get_hardlimit_mode(int cardno, int axis, int mode, int *enable, int *logic, int *admode)			
指令描述			
获取硬件限位输入模式信息			
参数说明			
名称	**数据类型**	**范围**	**说明**

cardno	int	[0-9]	控制卡索引号
axis	int	[1-MAXAXIS]	轴号
mode	int	[0-4]	硬件信号标识 0：正限位 1：负限位 2：原点信号(STOP0) 3：保留 4：硬件停止信号
enable	int*		0：禁用 1：启用
logic	int*		有效电平 0：低电平 1：高电平
admode	int*		停止模式 0：减速停止 1：立即停止
返回值			
0 表示执行成功，其他返回值表示执行失败，失败原因参考标准错误码			
指令分类	指令列表_资源配置类		
相关章节			
相关例程			

adt_set_emergency_stop

指令格式
int _stdcall adt_set_emergency_stop(int cardno, int enable, int logic)
指令描述
设置当前控制卡紧急停止信号模式。

参数说明			
名称	**数据类型**	**范围**	**说明**
cardno	int	[0-9]	控制卡索引号
enable	int	[0,1]	硬件停止功能是否启用 0：禁用 1：启用
logic	int	[0,1]	硬件停止信号有效电平 0：低电平有效 1：高电平有效
返回值			
0 表示执行成功，其他返回值表示执行失败，失败原因参考标准错误码			
指令分类	指令列表_资源配置类		
相关章节	硬件停止信号(EMGN)		

相关例程	

adt_get_emergency_stop

指令格式
int _stdcall adt_get_emergency_stop(int cardno, int* enable, int* logic)

指令描述
获取急停信号模式状态

参数说明			
名称	数据类型	范围	说明
cardno	int	[0-9]	控制卡索引号
enable	int*		0：禁用　1：启用
logic	int*		有效电平： 0，低电平有效 1，高电平有效

返回值
0 表示执行成功，其他返回值表示执行失败，失败原因参考标准错误码

指令分类	指令列表_资源配置类
相关章节	硬件停止信号(EMGN)
相关例程	

adt_set_limit_mode

指令格式
int _stdcall adt_set_limit_mode(int cardno, int axis, int pel_enable, int mel_enable, int logic)

指令描述
设置控制卡当前轴硬件限位信号模式。

参数说明			
名称	数据类型	范围	说明
cardno	int	[0-9]	控制卡索引号
axis	int	[1-MAXAXISS]	轴号
pel_enable	int	[0,1]	硬件正限位是否启用 0：禁用　1：启用
mel_enable	int	[0,1]	硬件负限位是否启用

			0：禁用　1：启用
logic	int	[0,1]	有效电平 0：低电平有效　1：高电平有效
返回值			
0 表示执行成功，其他返回值表示执行失败，失败原因参考标准错误码			
指令分类	指令列表_资源配置类		
相关章节	硬件限位信号(LMT+/LMT-)		
相关例程			

adt_get_limit_mode

指令格式			
int _stdcall adt_get_limit_mode(int cardno, int axis, int *pel_enable, int *mel_enable, int *logic)			
指令描述			
获取正负限位信号模式			
参数说明			
名称	**数据类型**	**范围**	**说明**
cardno	int	[0-9]	控制卡索引号
axis	int	[1-MAXAXIS]	轴数
pel_enable	int		硬件正限位是否启用 0：禁用　1：启用
mel_enable	int		硬件负限位是否启用 0：禁用　1：启用
logic	int		有效电平： 0，低电平有效 1，高电平有效
返回值			
0 表示执行成功，其他返回值表示执行失败，失败原因参考标准错误码			
指令分类	指令列表_资源配置类		
相关章节	硬件限位信号(LMT+/LMT-)		
相关例程			

adt_set_stop0_mode

指令格式			
int _stdcall adt_set_stop0_mode(int cardno, int axis, int enable, int logic, int admode)			
指令描述			
设置控制卡当前轴机械原点信号(STOP0)模式。			
参数说明			
名称	**数据类型**	**范围**	**说明**
cardno	int	[0-9]	控制卡索引号
axis	int	[1-MAXAXIS]	轴号
enable	int	[0,1]	原点信号是否启用 0：禁用 1：启用
logic	int	[0,1]	原点信号有效电平 0：低电平有效 1：高电平有效
admode	int	[0,1]	触发停止模式 0：减速停止 1：立即停止
返回值			
0 表示执行成功，其他返回值表示执行失败，失败原因参考标准错误码			
指令分类	指令列表_资源配置类		
相关章节	机械原点信号(STOP0)		
相关例程			

adt_get_stop0_mode

指令格式			
int _stdcall adt_get_stop0_mode(int cardno, int axis, int *enable, int *logic, int *admode)			
指令描述			
获取控制卡当前轴机械原点信号(STOP0)模式。			
参数说明			
名称	**数据类型**	**范围**	**说明**
cardno	int	[0-9]	控制卡索引号
axis	int	[1-MAXAXIS]	轴号

enable	int		原点信号是否启用 0：禁用　1：启用
logic	int		原点信号有效电平 0：低电平有效　1：高电平有效
admode	int		触发停止模式 0：减速停止　1：立即停止
返回值			
0 表示执行成功，其他返回值表示执行失败，失败原因参考标准错误码			
指令分类	指令列表_资源配置类		
相关章节	机械原点信号(STOP0)		
相关例程			

adt_set_unit_mode

指令格式			
int _stdcall adt_set_unit_mode(int cardno, int axis, int mode)			
指令描述			
设置控制卡当前轴编程模式。			
参数说明			
名称	**数据类型**	**范围**	**说明**
cardno	int	[0-9]	控制卡索引号
axis	int	[1-MAXAXIS]	实体轴或插补轴号
mode	int	[0,1]	轴编程模式 0：基于脉冲当量的编程模式 1：基于脉冲的编程模式
返回值			
0 表示执行成功，其他返回值表示执行失败，失败原因参考标准错误码			
指令分类	指令列表_资源配置类		
相关章节	编程模式		
相关例程			

adt_get_unit_mode

指令格式
int _stdcall adt_get_unit_mode(int cardno, int axis, int *mode)

指令描述			
获取控制卡当前轴编程模式。			
参数说明			
名称	**数据类型**	**范围**	**说明**
cardno	int	[0-9]	控制卡索引号
axis	int	[1-MAXAXIS]	实体轴或插补轴号
mode	int*		轴编程模式 0：基于脉冲当量的编程模式 1：基于脉冲的编程模式
返回值			
0 表示执行成功，其他返回值表示执行失败，失败原因参考标准错误码			
指令分类	指令列表_资源配置类		
相关章节	编程模式		
相关例程			

adt_set_pulse_equiv

指令格式			
int _stdcall adt_set_pulse_equiv(int cardno, int axis, double equiv)			
指令描述			
设置控制卡当前轴脉冲当量。			
参数说明			
名称	**数据类型**	**范围**	**说明**
cardno	int	[0-9]	控制卡索引号
axis	int	[1-MAXAXIS]	轴号
equiv	double	[50-20000]	轴脉冲当量
返回值			
0 表示执行成功，其他返回值表示执行失败，失败原因参考标准错误码			
指令分类	指令列表_资源配置类		
相关章节	脉冲当量		
相关例程	例程 6-2 配置轴基本参数		

adt_get_pulse_equiv

指令格式

int _stdcall adt_get_pulse_equiv(int cardno, int axis, double *equiv)			
指令描述			
获取控制卡当前轴脉冲当量。			
参数说明			
名称	**数据类型**	**范围**	**说明**
cardno	int	[0-9]	控制卡索引号
axis	int	[1-MAXAXIS]	轴号
equiv	double*		轴脉冲当量
返回值			
0 表示执行成功，其他返回值表示执行失败，失败原因参考标准错误码			
指令分类	指令列表_资源配置类		
相关章节	脉冲当量		
相关例程			

adt_set_bus_axis_gear_ratio

指令格式			
int _stdcall adt_set_bus_axis_gear_ratio(int cardno, int axis, int enc, int elec)			
指令描述			
设置总线脉冲电子齿轮比			
参数说明			
名称	**数据类型**	**范围**	**说明**
cardno	int	[0-9]	控制卡索引号
axis	int	[1-MAXAXIS]	轴号
enc	int	[1-∞]	编码器分辨率，即编码器每圈脉冲数
elec	int	[1-∞]	电机转动一圈控制卡需下发的脉冲数
返回值			
0 表示执行成功，其他返回值表示执行失败，失败原因参考标准错误码			
指令分类	指令列表_资源配置类		
相关章节	电子齿轮比		
相关例程	例程 6-2 配置轴基本参数		

adt_get_bus_axis_gear_ratio

指令格式

int _stdcall adt_get_bus_axis_gear_ratio(int cardno, int axis, int *enc, int *elec)			
指令描述			
获取总线脉冲电子齿轮比			
参数说明			
名称	**数据类型**	**范围**	**说明**
cardno	int	[0-9]	控制卡索引号
axis	int	[1-MAXAXIS]	轴号
enc	int*		编码器分辨率，即编码器每圈脉冲数
elec	int*		电机转动一圈控制卡需下发的脉冲数
返回值			
0 表示执行成功，其他返回值表示执行失败，失败原因参考标准错误码			
指令分类	指令列表_资源配置类		
相关章节	电子齿轮比		
相关例程			

adt_set_softlimit_mode

指令格式			
int _stdcall adt_set_softlimit_mode(int cardno, int axis, int enable, double ppos, double npos, int admode)			
指令描述			
设置当前轴软件限位功能工作模式			
参数说明			
名称	**数据类型**	**范围**	**说明**
cardno	int	[0-9]	控制卡索引号
axis	int	[1-MAXAXIS]	轴号
enable	int	[0, 1]	软件限位功能是否启用 0：禁用　1：启用
ppos	double		软件正限位位置
npos	double		软件负限位位置
admode	int	[0, 1]	触发时停止模式 0：减速停止　1：立即停止
返回值			

0 表示执行成功，其他返回值表示执行失败，失败原因参考标准错误码	
指令分类	指令列表_资源配置类
相关章节	软件限位
相关例程	例程 6-2 配置轴基本参数

adt_get_softlimit_mode

指令格式			
int _stdcall adt_get_softlimit_mode(int cardno, int axis, int *enable, double *ppos, double *npos, int *admode)			
指令描述			
获取当前轴软件限位功能工作模式			
参数说明			
名称	**数据类型**	**范围**	**说明**
cardno	int	[0-9]	控制卡索引号
axis	int	[1-MAXAXIS]	轴号
enable	int*		软件限位功能是否启用 0：禁用 1：启用
ppos	double*		软件正限位位置
npos	double*		软件负限位位置
admode	int*		触发时停止模式 0：减速停止 1：立即停止
返回值			
0 表示执行成功，其他返回值表示执行失败，失败原因参考标准错误码			
指令分类	指令列表_资源配置类		
相关章节	软件限位		
相关例程			

adt_set_pos_variable_loop

指令格式
int _stdcall adt_set_pos_variable_loop(int cardno, int axis, int enable, long pos)
指令描述
设置逻辑位置可变环功能。
参数说明

名称	数据类型	范围	说明
cardno	int	[0-9]	控制卡索引号
axis	int	[1-MAXAXIS]	轴号
enable	int	[0,1]	0：禁用　1：启用
pos	long	[0-∞)	变环位置，逻辑位置计数器的最大数值
返回值			
0 表示执行成功，其他返回值表示执行失败，失败原因参考标准错误码			
指令分类	指令列表_资源配置类		
相关章节	逻辑变环模式		
相关例程			

adt_get_pos_variable_loop

指令格式
int _stdcall adt_get_pos_variable_loop(int cardno, int axis, int *enable, long *CompPos)
指令描述
获取逻辑位置可变环功能参数
参数说明

名称	数据类型	范围	说明
cardno	int	[0-9]	控制卡索引号
axis	int	[1-MAXAXIS]	轴号
enable	int		0：禁用　1：启用
CompPos	long		变环位置，逻辑位置计数器的最大数值
返回值			
0 表示执行成功，其他返回值表示执行失败，失败原因参考标准错误码			
指令分类	指令列表_资源配置类		
相关章节	逻辑变环模式		
相关例程			

adt_set_encoder_scale

指令格式
int _stdcall adt_set_encoder_scale(int cardno, int axis, int beta, int alpha, int

mode=0)			
指令描述			
设置编码器反馈分频比			
参数说明			
名称	**数据类型**	**范围**	**说明**
cardno	int	[0-9]	控制卡索引号
axis	int	[1-MAXAXIS]	轴号
beta	int	beta>alpha	分子(编码器实际值)
alpha	int	alpha<beta	分母(编码器当量值)
mode	int	[0,1]	编码器返回值模式，默认 0 返回编码器实际值，1 返回编码器分频后位置
返回值			
0 表示执行成功，其他返回值表示执行失败，失败原因参考标准错误码			
指令分类	指令列表_资源配置类		
相关章节	编码器反馈分频比		
相关例程			

adt_set_axis_band

指令格式			
int _stdcall adt_set_axis_band(int cardno, int axis, int enable, long band, long time)			
指令描述			
电机到位检测，设置轴到位误差带			
参数说明			
名称	**数据类型**	**范围**	**说明**
cardno	int	[0-9]	控制卡索引号
axis	int	[1-MAXAXIS]	轴号
enable	int	[0,1]	0：禁用　1：启用
band	long	[0-∞]	误差带大小，单位：pulse,
time	long	[0-∞]	误差带保持时间，单位：指令周期，最小为一个指令周期。
返回值			

0 表示执行成功，其他返回值表示执行失败，失败原因参考标准错误码	
指令分类	指令列表_资源配置类
相关章节	到位误差与跟随误差
相关例程	

adt_get_axis_band

指令格式			
int _stdcall adt_get_axis_band(int cardno, int axis, int*enable, long*band, long*time)			
指令描述			
获取轴到位误差带参数			
参数说明			
名称	**数据类型**	**范围**	**说明**
cardno	int	[0-9]	控制卡索引号
axis	int	[1-MAXAXIS]	轴号
enable	int*		0：禁用　1：启用
band	long*		误差带大小，单位：pulse,
time	long*		误差带保持时间，单位：指令周期，最小为一个指令周期。
返回值			
0 表示执行成功，其他返回值表示执行失败，失败原因参考标准错误码			
指令分类	指令列表_资源配置类		
相关章节	到位误差与跟随误差		
相关例程			

adt_set_axis_pos_error

指令格式			
int _stdcall adt_set_axis_pos_error(int cardno, int axis, int enable, int pulse, int mode)			
指令描述			
设置轴最大跟随误差限制			
参数说明			
名称	**数据类型**	**范围**	**说明**

cardno	int	[0-9]	控制卡索引号
axis	int	[1-MAXAXIS]	轴号
enable	int	[0,1]	0：禁用　1：启用
pulse	int	[0-∞]	跟随误差脉冲数
mode	int	[0-2]	0，减速；1，立即停；2，不限制轴运动功能
返回值			
0 表示执行成功，其他返回值表示执行失败，失败原因参考标准错误码			
指令分类	指令列表_资源配置类		
相关章节	到位误差与跟随误差		
相关例程			

adt_set_follow_axis

指令格式			
int _stdcall adt_set_follow_axis(int cardno, int slave, int master)			
指令描述			
设置轴跟随驱动模式(开环龙门双驱)。			
参数说明			
名称	**数据类型**	**范围**	**说明**
cardno	int	[0-9]	控制卡索引号
slave	int	[1-MAXAXIS]	从轴号
master	int	[0-MAXAXIS]	主轴号，0 表示取消跟随
返回值			
0 表示执行成功，其他返回值表示执行失败，失败原因参考标准错误码			
指令分类	指令列表_资源配置类		
相关章节	龙门双驱		
相关例程	例程 9-6 龙门双驱		

18.5. 轴状态监视与控制

adt_set_command_pos

指令格式			
int _stdcall adt_set_command_pos(int cardno, int axis, long pos)			
指令描述			
设置控制卡设指定轴当前逻辑位置。			
参数说明			
名称	**数据类型**	**范围**	**说明**
cardno	int	[0-9]	控制卡索引号
axis	int	[1-MAXAXIS]	插补轴号
pos	long	[-2147483648, 2147483647]	逻辑位置值
返回值			
0 表示执行成功，其他返回值表示执行失败，失败原因参考标准错误码			
指令分类	指令列表_轴状态监视与控制		
相关章节	逻辑位置		
相关例程	例程 7-1 轴状态监控		

adt_get_command_pos

指令格式			
int _stdcall adt_get_command_pos(int cardno, int axis, long *pos)			
指令描述			
获取控制卡设指定轴当前逻辑位置。			
参数说明			
名称	**数据类型**	**范围**	**说明**
cardno	int	[0-9]	控制卡索引号
axis	int	[1-MAXAXIS]	插补轴号
pos	long*		逻辑位置值
返回值			
0 表示执行成功，其他返回值表示执行失败，失败原因参考标准错误码			
指令分类	指令列表_轴状态监视与控制		
相关章节	逻辑位置		

相关例程	

adt_set_actual_pos

指令格式			
int _stdcall adt_set_actual_pos(int cardno, int axis, long pos)			
指令描述			
设置控制卡设指定轴当前编码器实际位置。			
参数说明			
名称	**数据类型**	**范围**	**说明**
cardno	int	[0-9]	控制卡索引号
axis	int	[1-MAXAXIS]	插补轴号
pos	long	[-2147483648, 2147483647]	编码器位置值
返回值			
0 表示执行成功，其他返回值表示执行失败，失败原因参考标准错误码			
指令分类	指令列表_轴状态监视与控制		
相关章节	实际位置		
相关例程			

adt_get_actual_pos

指令格式			
int _stdcall adt_get_actual_pos(int cardno, int axis, long *pos)			
指令描述			
获取控制卡设指定轴当前编码器位置。			
参数说明			
名称	**数据类型**	**范围**	**说明**
cardno	int	[0-9]	控制卡索引号
axis	int	[1-MAXAXIS]	插补轴号
pos	long*		编码器位置值
返回值			
0 表示执行成功，其他返回值表示执行失败，失败原因参考标准错误码			
指令分类	指令列表_轴状态监视与控制		
相关章节	实际位置		

相关例程	

adt_get_target_pos_unit

指令格式			
int _stdcall adt_get_target_pos_unit(int cardno, int axis, double *pos)			
指令描述			
获取控制卡设指定轴目标位置(单位：mm)，适用于基于脉冲当量编程模式的轴。			
参数说明			
名称	**数据类型**	**范围**	**说明**
cardno	int	[0-9]	控制卡索引号
axis	int	[1-MAXAXIS]	插补轴号
pos	double*		轴目标位置值(mm)
返回值			
0 表示执行成功，其他返回值表示执行失败，失败原因参考标准错误码			
指令分类	指令列表_轴状态监视与控制		
相关章节	目标位置		
相关例程			

adt_get_target_pos_pulse

指令格式			
int _stdcall adt_get_target_pos_pulse(int cardno, int axis, long *pos)			
指令描述			
获取控制卡设指定轴目标位置(单位：pulse)，适用于基于脉冲编程模式的轴。			
参数说明			
名称	**数据类型**	**范围**	**说明**
cardno	int	[0-9]	控制卡索引号
axis	int	[1-MAXAXIS]	插补轴号
pos	long*		轴目标位置值(pulse)
返回值			
0 表示执行成功，其他返回值表示执行失败，失败原因参考标准错误码			
指令分类	指令列表_轴状态监视与控制		
相关章节	目标位置		
相关例程			

adt_set_axis_enable

指令格式			
int _stdcall adt_set_axis_enable(int cardno, int axis, int enbale)			
指令描述			
轴使能控制			
参数说明			
名称	**数据类型**	**范围**	**说明**
cardno	int	[0-9]	控制卡索引号
axis	int	[1-MAXAXIS]	轴号
enable	int	[0,1]	0,关闭使能；1,打开使能
返回值			
0 表示执行成功，其他返回值表示执行失败，失败原因参考标准错误码			
指令分类	指令列表_轴状态监视与控制		
相关章节	目标位置		
相关例程			

adt_get_axis_enable

指令格式			
int _stdcall adt_get_axis_enable(int cardno, int axis, int* enbale)			
指令描述			
读取轴使能的状态			
参数说明			
名称	**数据类型**	**范围**	**说明**
cardno	int	[0-9]	控制卡索引号
axis	int	[1-MAXAXIS]	轴号
enable	int*		0,关闭使能；1,打开使能
返回值			
0 表示执行成功，其他返回值表示执行失败，失败原因参考标准错误码			
指令分类	指令列表_轴状态监视与控制		
相关章节	伺服使能		
相关例程			

adt_clear_bus_axis_alarm

指令格式			
int _stdcall adt_clear_bus_axis_alarm(int cardno, int axis)			
指令描述			
清除总线伺服驱动器报警状态(涉及通讯的伺服报警无法被清除)			
参数说明			
名称	**数据类型**	**范围**	**说明**
cardno	int	[0-9]	控制卡索引号
axis	int	[1-MAXAXIS]	轴号
返回值			
0 表示执行成功，其他返回值表示执行失败，失败原因参考标准错误码			
指令分类	指令列表_轴状态监视与控制		
相关章节			
相关例程			

adt_get_axis_alarm_info

指令格式			
int _stdcall adt_get_axis_alarm_info(int cardno, int axis, int *alarm)			
指令描述			
获取总线伺服驱动器报警状态			
参数说明			
名称	**数据类型**	**范围**	**说明**
cardno	int	[0-9]	控制卡索引号
axis	int	[1-MAXAXIS]	轴号
alarm	int*		总线伺服通信状态 0: 通信正常 118: 总线通信报警 119: 总线伺服报警
返回值			
0 表示执行成功，其他返回值表示执行失败，失败原因参考标准错误码			
指令分类	指令列表_轴状态监视与控制		
相关章节			
相关例程			

adt_get_axis_status

指令格式			
int _stdcall adt_get_axis_status(int cardno, int axis, int *status)			
指令描述			
获取控制卡指定轴驱动状态，可以是实体轴，也可以是插补轴。			
参数说明			
名称	**数据类型**	**范围**	**说明**
cardno	int	[0-9]	控制卡索引号
axis	int	[1-MAXAXIS],[60-63]	轴号,插补轴号
status	int		轴驱动状态
返回值			
0 表示执行成功，其他返回值表示执行失败，失败原因参考标准错误码			
指令分类	指令列表_轴状态监视与控制		
相关章节	轴运动状态		
相关例程	例程 7-1 轴状态监控		

adt_get_stopdata

指令格式			
int _stdcall adt_get_stopdata(int cardno, int axis, int *value)			
指令描述			
获取控制卡指定轴停止信息。			
参数说明			
名称	**数据类型**	**范围**	**说明**
cardno	int	[0-9]	控制卡索引号
axis	int	[1-MAXAXIS]	轴号
value	int		轴停止信息
返回值			
0 表示执行成功，其他返回值表示执行失败，失败原因参考标准错误码			
指令分类	指令列表_轴状态监视与控制		
相关章节	停止信息		
相关例程	例程 7-2 停止信息解释示例		

adt_get_all_axis_move_status

指令格式			
int _stdcall adt_get_all_axis_move_status(int cardno, unsigned int *status)			
指令描述			
获取所有轴运动状态			
参数说明			
名称	**数据类型**	**范围**	**说明**
cardno	int	[0-9]	控制卡索引号
status	unsiged int*		按位 bit0-31 表示，分别对应轴 1-MAXAXIS，0 停止，1 运动规划中
返回值			
0 表示执行成功，其他返回值表示执行失败，失败原因参考标准错误码			
指令分类	指令列表_轴状态监视与控制		
相关章节	轴运动状态		
相关例程	例程 7-1 轴状态监控		

adt_get_all_axis_reach_status

指令格式			
int _stdcall adt_get_all_axis_reach_status(int cardno, unsigned int *status)			
指令描述			
获取所有轴到位状态			
参数说明			
名称	**数据类型**	**范围**	**说明**
cardno	int	[0-9]	控制卡索引号
status	unsiged int*		按位 bit0-31 表示，分别对应轴 1-MAXAXIS，0 没有到位，1 到位
返回值			
0 表示执行成功，其他返回值表示执行失败，失败原因参考标准错误码			
指令分类	指令列表_轴状态监视与控制		
相关章节	轴运动状态		

相关例程	例程 7-1 轴状态监控

adt_get_axis_status_ex

指令格式
int _stdcall adt_get_axis_status_ex (int cardno, int axis, int *status)
指令描述
获取轴状态信息
参数说明

名称	数据类型	范围	说明
cardno	int	[0-9]	控制卡索引号
axis	int	[1-MAXAXIS]	轴号
status	int*		当前轴状态，按位表示 B0--运动 B1--暂停 B2--正限位 B3--负限位 B4--原点限位 B5--Z 相限位 B6--软正限位 B7--软负限位 B8--到位标志 B9--超差标志 B10--伺服报警 B11--保留 B12--总线伺服使能标志

返回值
0 表示执行成功，其他返回值表示执行失败，失败原因参考标准错误码

指令分类	指令列表_轴状态监视与控制
相关章节	轴运动状态
相关例程	例程 7-1 轴状态监控

adt_clear_axis_status

指令格式
int _stdcall adt_clear_axis_status(int cardno, int axis)
指令描述
清除轴状态
参数说明

名称	数据类型	范围	说明
cardno	int	[0-9]	控制卡索引号
axis	int	[1-MAXAXIS]	轴号

返回值	
0 表示执行成功，其他返回值表示执行失败，失败原因参考标准错误码	
指令分类	指令列表_轴状态监视与控制
相关章节	轴运动状态
相关例程	

adt_get_speed

指令格式			
int _stdcall adt_get_speed(int cardno, int axis, double *speed)			
指令描述			
获取控制卡当前轴当前驱动速度。			
参数说明			
名称	**数据类型**	**范围**	**说明**
cardno	int	[0-9]	控制卡索引号
axis	int	[1-MAXAXIS]	轴号
speed	double*		当前驱动速度
返回值			
0 表示执行成功，其他返回值表示执行失败，失败原因参考标准错误码			
指令分类	指令列表_轴状态监视与控制		
相关章节	轴规划速度		
相关例程			

adt_get_encoder_speed

指令格式			
int _stdcall adt_get_encoder_speed(int cardno, int axis, double *speed)			
指令描述			
获取当前轴编码器速度。			
参数说明			
名称	**数据类型**	**范围**	**说明**
cardno	int	[0-9]	控制卡索引号
axis	int	[1-MAXAXIS]	轴号
speed	double*		当前驱动速度
返回值			

0 表示执行成功，其他返回值表示执行失败，失败原因参考标准错误码	
指令分类	指令列表_轴状态监视与控制
相关章节	编码器反馈速度
相关例程	

18.6. 速度规划类

adt_set_startv

指令格式			
int _stdcall adt_set_startv(int cardno, int axis, double speed)			
指令描述			
设置控制卡当前轴驱动起始速度。			
参数说明			
名称	**数据类型**	**范围**	**说明**
cardno	int	[0-9]	控制卡索引号
axis	int	[1-MAXAXIS],[60-63]	轴号，插补轴号
speed	double	脉冲当量编程模式： speed*equiv<5M 脉冲编程模式： speed<5M equiv -- 脉冲当量	驱动起始速度
返回值			
0 表示执行成功，其他返回值表示执行失败，失败原因参考标准错误码			
指令分类	指令列表_速度规划类		
相关章节	基础速度规划		
相关例程			

adt_get_startv

指令格式			
int _stdcall adt_get_startv(int cardno, int axis, double *speed)			
指令描述			
获取控制卡当前轴驱动起始速度。			
参数说明			
名称	**数据类型**	**范围**	**说明**
cardno	int	[0-9]	控制卡索引号
axis	int	[1-MAXAXIS],[60-63]	轴号，插补轴号
speed	double*		驱动起始速度
返回值			

0 表示执行成功，其他返回值表示执行失败，失败原因参考标准错误码	
指令分类	指令列表_速度规划类
相关章节	基础速度规划
相关例程	

adt_set_maxv

指令格式			
int _stdcall adt_set_maxv(int cardno, int axis, double speed)			
指令描述			
设置控制卡当前轴驱动最大速度。			
参数说明			
名称	**数据类型**	**范围**	**说明**
cardno	int	[0-9]	控制卡索引号
axis	int	[1-MAXAXIS],[60-63]	轴号，插补轴号
speed	double	脉冲当量编程模式： speed*equiv<5M 脉冲编程模式： speed<5M equiv -- 脉冲当量	驱动最大速度
返回值			
0 表示执行成功，其他返回值表示执行失败，失败原因参考标准错误码			
指令分类	指令列表_速度规划类		
相关章节	基础速度规划		
相关例程			

adt_get_maxv

指令格式			
int _stdcall adt_get_maxv(int cardno, int axis, double *speed)			
指令描述			
获取控制卡当前轴驱动最大速度。			
参数说明			
名称	**数据类型**	**范围**	**说明**
cardno	int	[0-9]	控制卡索引号

axis	int	[1-MAXAXIS],[60-63]	轴号，插补轴号
speed	double*		驱动最大速度
返回值			
0 表示执行成功，其他返回值表示执行失败，失败原因参考标准错误码			
指令分类	指令列表_速度规划类		
相关章节	基础速度规划		
相关例程			

adt_set_endv

指令格式			
int _stdcall adt_set_endv (int cardno, int axis, double speed)			
指令描述			
设置控制卡当前轴结束速度。			
参数说明			
名称	**数据类型**	**范围**	**说明**
cardno	int	[0-9]	控制卡索引号
axis	int	[1-MAXAXIS],[60-63]	轴号，插补轴号
speed	double	脉冲当量编程模式： speed*equiv<5M 脉冲编程模式： speed<5M equiv -- 脉冲当量	结束速度
返回值			
0 表示执行成功，其他返回值表示执行失败，失败原因参考标准错误码			
指令分类	指令列表_速度规划类		
相关章节	基础速度规划		
相关例程			

adt_get_endv

指令格式
int _stdcall adt_get_endv (int cardno, int axis, double *speed)
指令描述
设置控制卡当前轴结束速度。

参数说明			
名称	**数据类型**	**范围**	**说明**
cardno	int	[0-9]	控制卡索引号
axis	int	[1-MAXAXIS],[60-63]	轴号，插补轴号
speed	Double*		结束速度
返回值			
0 表示执行成功，其他返回值表示执行失败，失败原因参考标准错误码			
指令分类	指令列表_速度规划类		
相关章节	基础速度规划		
相关例程			

adt_set_acc

指令格式			
int _stdcall adt_set_acc(int cardno, int axis, double acc)			
指令描述			
设置控制卡当前轴驱动加速度。			
参数说明			
名称	**数据类型**	**范围**	**说明**
cardno	int	[0-9]	控制卡索引号
axis	int	[1-MAXAXIS],[60-63]	轴号，插补轴号
acc	double	脉冲当量编程模式： acc*equiv<100M 脉冲编程模式： acc<100M equiv -- 脉冲当量	驱动加速度
返回值			
0 表示执行成功，其他返回值表示执行失败，失败原因参考标准错误码			
指令分类	指令列表_速度规划类		
相关章节	基础速度规划		
相关例程			

adt_get_acc

指令格式			
int _stdcall adt_get_acc(int cardno, int axis, double *acc)			
指令描述			
获取控制卡当前轴驱动加速度。			
参数说明			
名称	**数据类型**	**范围**	**说明**
cardno	int	[0-9]	控制卡索引号
axis	int	[1-MAXAXIS],[60-63]	轴号，插补轴号
acc	double*		驱动加速度
返回值			
0 表示执行成功，其他返回值表示执行失败，失败原因参考标准错误码			
指令分类	指令列表_速度规划类		
相关章节	基础速度规划		
相关例程			

adt_set_dec

指令格式			
int _stdcall adt_set_dec(int cardno, int axis, double dec)			
指令描述			
设置控制卡当前轴驱动减速度。			
参数说明			
名称	**数据类型**	**范围**	**说明**
cardno	int	[0-9]	控制卡索引号
axis	int	[1-MAXAXIS],[60-63]	轴号，插补轴号
dec	double	脉冲当量编程模式： dec*equiv<100M 脉冲编程模式： dec<100M equiv -- 脉冲当量	驱动减速度
返回值			
0 表示执行成功，其他返回值表示执行失败，失败原因参考标准错误码			

指令分类	指令列表_速度规划类
相关章节	基础速度规划
相关例程	

adt_get_dec

指令格式			
int _stdcall adt_get_dec(int cardno, int axis, double *dec)			
指令描述			
获取控制卡当前轴驱动减速度。			
参数说明			
名称	数据类型	范围	说明
cardno	int	[0-9]	控制卡索引号
axis	int	[1-MAXAXIS],[60-63]	轴号，插补轴号
dec	double *		驱动减速度
返回值			
0 表示执行成功，其他返回值表示执行失败，失败原因参考标准错误码			
指令分类	指令列表_速度规划类		
相关章节	基础速度规划		
相关例程			

adt_set_admode

指令格式			
int _stdcall adt_set_admode(int cardno, int axis, int mode)			
指令描述			
设置控制卡当前轴加减速模式。			
参数说明			
名称	数据类型	范围	说明
cardno	int	[0-9]	控制卡索引号
axis	int	[1-MAXAXIS],[60-63]	轴号，插补轴号
mode	int	[0-3]	轴加减速模式 0：S 型加减速 1：T 型加减速 2：EXP 指数型加减速

			3：COS 余弦型加减速
返回值			
0 表示执行成功，其他返回值表示执行失败，失败原因参考标准错误码			
指令分类	指令列表_速度规划类		
相关章节	基础速度规划		
相关例程			

adt_get_admode

指令格式			
int _stdcall adt_get_admode(int cardno, int axis, int *mode)			
指令描述			
获取控制卡当前轴加减速模式。			
参数说明			
名称	**数据类型**	**范围**	**说明**
cardno	int	[0-9]	控制卡索引号
axis	int	[1-MAXAXIS],[60-63]	轴号，插补轴号
mode	int*		轴加减速模式
返回值			
0 表示执行成功，其他返回值表示执行失败，失败原因参考标准错误码			
指令分类	指令列表_速度规划类		
相关章节	基础速度规划		
相关例程			

adt_set_axis_move_para_unit

指令格式			
int _stdcall adt_set_axis_move_para_unit(int cardno, int axis, int admode, double strv, double maxv, double acc, double dec)			
指令描述			
集成设置轴运动参数。			
参数说明			
名称	**数据类型**	**范围**	**说明**
cardno	int	[0-9]	控制卡索引号
axis	int	[1-MAXAXIS]	轴号

admode	int	[0,-3]	轴加减速模式 0：S 型加减速 1：T 型加减速 2：EXP 指数型加减速 3：COS 余弦型加减速
strv	double	脉冲当量编程模式： speed*equiv<5M 脉冲编程模式：speed<5M equiv -- 脉冲当量	驱动起始速度
maxv	double	脉冲当量编程模式： speed*equiv<5M 脉冲编程模式：speed<5M equiv -- 脉冲当量	驱动最大速度
acc	double	脉冲当量编程模式： acc*equiv<100M 脉冲编程模式：acc<100M equiv -- 脉冲当量	驱动加速度
dec	double	脉冲当量编程模式： dec*equiv<100M 脉冲编程模式：dec<100M equiv -- 脉冲当量	驱动减速度

返回值
0 表示执行成功，其他返回值表示执行失败，失败原因参考标准错误码

指令分类	指令列表_速度规划类
相关章节	集成速度设置
相关例程	

adt_set_rate

指令格式
int _stdcall adt_set_rate(int cardno, int axis, double rate)
指令描述

设置控制卡当前轴速度倍率。			
参数说明			
名称	**数据类型**	**范围**	**说明**
cardno	int	[0-9]	控制卡索引号
axis	int	[1-MAXAXIS],[60-63]	轴号，插补轴号
rate	double	[0, 2]	驱动速度倍率
返回值			
0 表示执行成功，其他返回值表示执行失败，失败原因参考标准错误码			
指令分类	指令列表_速度规划类		
相关章节	速度倍率		
相关例程			

adt_get_rate

指令格式			
int _stdcall adt_get_rate(int cardno, int axis, double *rate)			
指令描述			
获取控制卡当前轴速度倍率。			
参数说明			
名称	**数据类型**	**范围**	**说明**
cardno	int	[0-9]	控制卡索引号
axis	int	[1-MAXAXIS],[60-63]	轴号，插补轴号
rate	double*		驱动速度倍率
返回值			
0 表示执行成功，其他返回值表示执行失败，失败原因参考标准错误码			
指令分类	指令列表_速度规划类		
相关章节	速度倍率		
相关例程			

adt_set_corner_speed_smooth_level

指令格式
int _stdcall adt_set_corner_speed_smooth_level(int cardno, int axis, int level)
指令描述
设置控制卡当前轴拐角加速度平滑等级。

参数说明			
名称	**数据类型**	**范围**	**说明**
cardno	int	[0-9]	控制卡索引号
axis	int	[1-MAXAXIS]	轴号
level	int	[0-100)	轴拐角速度平滑系数，默认平滑系数为 50
返回值			
0 表示执行成功，其他返回值表示执行失败，失败原因参考标准错误码			
指令分类	指令列表_速度规划类		
相关章节	拐角加速度平滑等级		
相关例程			

adt_set_arc_speed_clamp_unit

指令格式
int _stdcall adt_set_arc_speed_clamp_unit(int cardno, int inp, double radius, double speed)
指令描述
设置控制卡插补轴拐角圆弧速度约束。

参数说明			
名称	**数据类型**	**范围**	**说明**
cardno	int	[0-9]	控制卡索引号
inp	int	[60-63]	插补轴号
radius	double	[0.001，∞)	半径系数，默认 10mm
speed	double	[0.01, 5000]	速度系数，默认 100mm/s
返回值			
0 表示执行成功，其他返回值表示执行失败，失败原因参考标准错误码			
指令分类	指令列表_速度规划类		
相关章节	圆弧插补速度约束		
相关例程			

adt_set_speed_pretreat_mode

指令格式
int _stdcall adt_set_speed_pretreat_mode(int cardno, int inp, int enable)

指令描述			
设置控制卡插补轴速度前瞻功能，开启缓存插补模式。			
参数说明			
名称	**数据类型**	**范围**	**说明**
cardno	int	[0-9]	控制卡索引号
inp	int	[60-63]	插补轴号
enable	int	[0,1]	速度前瞻功能标识 0：禁用　　1：开启
返回值			
0 表示执行成功，其他返回值表示执行失败，失败原因参考标准错误码			
指令分类	指令列表_速度规划类		
相关章节			
相关例程			

18.7. 点位运动类

adt_pmove_unit

指令格式			
int _stdcall adt_pmove_unit(int cardno, int axis, double pos, int mode)			
指令描述			
基于脉冲当量编程模式的单轴定量驱动。			
参数说明			
名称	**数据类型**	**范围**	**说明**
cardno	int	[0-9]	控制卡索引号
axis	int	[1-MAXAXIS]	轴号
pos	double		目标位置(mm)
mode	int	[0, 1]	位置模式 0：相对位置 1：绝对位置
返回值			
0 表示执行成功，其他返回值表示执行失败，失败原因参考标准错误码			
指令分类	指令列表_点位运动		
相关章节	定量驱动		
相关例程	例程 9-1 定量驱动		

adt_multi_pmove_unit

指令格式			
int _stdcall adt_multi_pmove_unit (int cardno, int AxisNum, int AxisList[], double PosList[], int mode)			
指令描述			
基于当量编程模式的多轴定量驱动			
参数说明			
名称	**数据类型**	**范围**	**说明**
cardno	int	[0-9]	控制卡索引号
AxisNum	int	[1-32]	运动轴数
AxisList	int*		轴号列表
PosList	double*		目标位置列表，单位(mm)
mode	int	[0, 1]	位置模式

			0：相对位置 1：绝对位置
返回值			
0 表示执行成功，其他返回值表示执行失败，失败原因参考标准错误码			
指令分类	指令列表_点位运动		
相关章节			
相关例程			

adt_pmove_pulse

指令格式			
int _stdcall adt_pmove_pulse(int cardno, int axis, long pos, int mode)			
指令描述			
基于脉冲编程模式的单轴定量驱动。			
参数说明			
名称	**数据类型**	**范围**	**说明**
cardno	int	[0-9]	控制卡索引号
axis	int	[1-MAXAXIS]	轴号
pos	long		目标位置(pulse)
mode	int	[0, 1]	轴位置模式 0：相对位置 1：绝对位置
返回值			
0 表示执行成功，其他返回值表示执行失败，失败原因参考标准错误码			
指令分类	指令列表_点位运动		
相关章节			
相关例程			

adt_continue_move

指令格式			
int _stdcall adt_continue_move(int cardno, int axis, int dir)			
指令描述			
单轴持续驱动。			
参数说明			
名称	**数据类型**	**范围**	**说明**
cardno	int	[0-9]	控制卡索引号

axis	int	[1-MAXAXIS]	轴号
dir	int	[0,1]	驱动方向 0：正向 1：负向

返回值
0 表示执行成功，其他返回值表示执行失败，失败原因参考标准错误码

指令分类	指令列表_点位运动
相关章节	连续驱动
相关例程	例程 9-2 连续驱动

adt_change_pmove_pos

指令格式
int _stdcall adt_change_pmove_pos(int cardno, int axis, double pos, int PosType)
指令描述
运动中改变点位运动目标位置
参数说明

名称	数据类型	范围	说明
cardno	int	[0-9]	控制卡索引号
axis	int	[1-MAXAXIS]	轴号
pos	double		位置，单位 unit
PosType	int	[1-16]	坐标类型： 0，相对位置 1，绝对位置

返回值
0 表示执行成功，其他返回值表示执行失败，失败原因参考标准错误码

指令分类	指令列表_点位运动
相关章节	在线改变目标位置
相关例程	

adt_set_axis_stop

指令格式
int _stdcall adt_set_axis_stop(int cardno, int axis, int admode)
指令描述
停止指定轴的驱动。
参数说明

名称	数据类型	范围	说明
cardno	int	[0-9]	控制卡索引号
axis	int	[1-MAXAXIS]	轴号
admode	int	[0,1]	轴停止模式 0：减速停止　1：立即停止
返回值			
0 表示执行成功，其他返回值表示执行失败，失败原因参考标准错误码			
指令分类	指令列表_点位运动		
相关章节	轴驱动停止		
相关例程			

18.8. 插补运动

adt_set_inp_coordinate

指令格式			
int _stdcall adt_set_inp_coordinate(int cardno, int InpAxis, int dim, int axislist[])			
指令描述			
创建插补坐标系			
参数说明			
名称	**数据类型**	**范围**	**说明**
cardno	int	[0-9]	控制卡索引号
InpAxis	int	[60-63]	插补轴号
dim	int	[1-16]	插补轴数，最多支持 16 轴插补， dim=0,取消插补坐标系
axislist	int*		实际物理轴列表
返回值			
0 表示执行成功，其他返回值表示执行失败，失败原因参考标准错误码			
指令分类	指令列表_插补运动		
相关章节	创建插补坐标系		
相关例程	例程 10-1 直线插补 例程 10-2 平面圆弧插补 例程 10-3 缓存插补		

adt_set_inp_stop

指令格式			
int _stdcall adt_set_inp_stop(int cardno, int InpAxis, int mode)			
指令描述			
0，设置插补运动停止、暂停、暂停恢复			
参数说明			
名称	**数据类型**	**范围**	**说明**
cardno	int	[0-9]	控制卡索引号
InpAxis	int	[60-63]	插补轴号
mode	int	[0-3]	停止模式： 0，减速停止

			1，立即停止 2，暂停 3，暂停恢复 暂停插补后，可以单轴运动；插补暂停恢复，插补各轴需要回到暂停位置，可以使用 adt_get_inp_pause_pos 获取最后一次插补暂停位置。
返回值			
0 表示执行成功，其他返回值表示执行失败，失败原因参考标准错误码			
指令分类	指令列表_插补运动		
相关章节	插补暂停与恢复		
相关例程			

adt_get_inp_pause_pos

指令格式			
int _stdcall adt_get_inp_pause_pos(int cardno, int InpAxis, long pos[])			
指令描述			
获取插补坐标系各轴暂停位置			
参数说明			
名称	**数据类型**	**范围**	**说明**
cardno	int	[0-9]	控制卡索引号
InpAxis	int	[60-63]	插补轴号
pos[]	long*		坐标系各轴暂停时的逻辑位置，注意 pos 数组长度为建立坐标系轴数，单位：脉冲
返回值			
0 表示执行成功，其他返回值表示执行失败，失败原因参考标准错误码			
指令分类	指令列表_插补运动		
相关章节	插补暂停与恢复		
相关例程			

adt_get_inp_fifo_len

指令格式

int _stdcall adt_get_inp_fifo_len(int cardno, int inp_axis, int *len)			
指令描述			
获取指定插补轴缓存插补余量			
参数说明			
名称	**数据类型**	**范围**	**说明**
cardno	int	[0-9]	控制卡索引号
inp_axis	int	[60-63]	插补轴号
len	int*		缓存插补余量
返回值			
0 表示执行成功，其他返回值表示执行失败，失败原因参考标准错误码			
指令分类	指令列表_插补运动		
相关章节	缓存插补		
相关例程	例程 10-3 缓存插补		

adt_get_inp_index

指令格式			
int _stdcall adt_get_inp_index(int cardno, int inp_axis, int *index)			
指令描述			
获取最新触发的缓存插补指令索引。			
参数说明			
名称	**数据类型**	**范围**	**说明**
cardno	int	[0-9]	控制卡索引号
inp_axis	int	[60-63]	插补轴号
index	int*		最新触发的缓存插补指令索引
返回值			
0 表示执行成功，其他返回值表示执行失败，失败原因参考标准错误码			
指令分类	指令列表_插补运动		
相关章节			
相关例程			

adt_inp_move_unit

指令格式
int _stdcall adt_inp_move_unit(int cardno, int inp_axis, int index, int axs_count, int

axs_list[], double pos_list[], int mode)			
指令描述			
基于脉冲当量编程模式的可缓存式多轴直线插补			
参数说明			
名称	**数据类型**	**范围**	**说明**
cardno	int	[0-9]	控制卡索引号
inp_axis	int	[60-63]	插补轴号
index	int	[0-∞]	索引信息，用于标识本次运动信息，一般设置为 0 即可
axs_count	int	[1-16]	参与直线插补轴数
axs_list	int*		参与直线插补的轴号列表，数组尺寸须等于 axs_count
pos_list	double*		参与直线插补的轴目标位置列表，数组尺寸须等于 axs_count，单位 mm
mode	int	[0, 1]	位置模式 0：相对位置 1：绝对位置
返回值			
0 表示执行成功，其他返回值表示执行失败，失败原因参考标准错误码			
指令分类	指令列表_插补运动		
相关章节	直线插补		
相关例程	例程 10-1 直线插补		

adt_inp_move_pulse

指令格式			
int _stdcall adt_inp_move_pulse(int cardno, int inp_axis, int index, int axs_count, int axs_list[], long pos_list[], int mode)			
指令描述			
基于脉冲编程模式的可缓存式多轴直线插补			
参数说明			
名称	**数据类型**	**范围**	**说明**
cardno	int	[0-9]	控制卡索引号
inp_axis	int	[60-63]	插补轴号

index	int	[0-∞]	索引信息，用于标识本次运动信息，一般设置为 0 即可
axs_count	int	[1-16]	参与直线插补轴数
axs_list	int*		参与直线插补的轴号列表，数组尺寸须大于等于 axs_count
pos_list	long *		参与直线插补的轴目标位置列表，数组尺寸须大于等于 axs_count，单位 pulse
mode	int	[0, 1]	轴位置模式 0：相对位置坐标 1：绝对位置坐标
返回值			
0 表示执行成功，其他返回值表示执行失败，失败原因参考标准错误码			
指令分类	指令列表_插补运动		
相关章节	直线插补		
相关例程			

adt_inp_arc2_unit

指令格式
int _stdcall adt_inp_arc2_unit(int cardno, int inp_axis, int index, int axs_list[2], double pos_list[2], double ctr_list[2], int dir, int mode)
指令描述
基于脉冲当量编程模式的可缓存式平面圆弧插补

参数说明			
名称	**数据类型**	**范围**	**说明**
cardno	int	[0-9]	控制卡索引号
inp_axis	int	[60-63]	插补轴号
index	int	[0-∞]	索引信息，用于标识本次运动信息，一般设置为 0 即可
axs_list	int*		参与平面圆弧插补的轴号列表，数组尺寸须等于 2
pos_list	double*		参与平面圆弧插补的轴目标位置列表，数组尺寸须等于 2，单位 mm

ctr_list	double *		参与平面圆弧插补的轴圆心位置列表，数组尺寸须等于 2，单位 mm
dir	int	[0, 1]	平面圆弧插补方向 0：逆时针 1：顺时针
mode	int	[0, 1]	位置模式 0：相对位置 1：绝对位置
返回值			
0 表示执行成功，其他返回值表示执行失败，失败原因参考标准错误码			
指令分类	指令列表_插补运动		
相关章节	平面圆弧插补		
相关例程	例程 10-2 平面圆弧插补		

adt_inp_arc3_unit

指令格式
int _stdcall adt_inp_arc3_unit(int cardno, int inp_axis, int index, int axs_list[3], double pos2_list[3], double pos3_list[3], int dir, int mode)
指令描述
基于脉冲当量编程模式的可缓存式球面圆弧插补
参数说明

名称	数据类型	范围	说明
cardno	int	[0-9]	控制卡索引号
inp_axis	int	[60-63]	插补轴号
index	int	[0-∞]	索引信息，用于标识本次运动信息，一般设置为 0 即可
axs_list	int*		参与球面圆弧插补的轴号列表，数组尺寸须等于 3
pos2_list	double*		参与球面圆弧插补的圆弧上第二点轴目标位置列表，数组尺寸须等于 3，单位 mm
pos3_list	double *		参与球面圆弧插补的圆弧上第三点轴目标位置列表，数组尺寸须等于 3，单位 mm
dir	int	[0, 1]	球面圆弧类型 0：半圆 1：整圆
mode	int	[0, 1]	位置模式 0：相对位置 1：绝对位置

返回值	
0 表示执行成功，其他返回值表示执行失败，失败原因参考标准错误码	
指令分类	指令列表_插补运动
相关章节	球面圆弧插补
相关例程	

adt_inp_helix2_unit

指令格式
int _stdcall adt_inp_helix2_unit(int cardno, int InpAxis, int index, int AxisList[4], double TargetPos[4], double CenterPos[2], int dir, int PosType)
指令描述
任意两轴圆弧插补，二轴跟随
参数说明

名称	数据类型	范围	说明
cardno	int	[0-9]	控制卡索引号
InpAxis	int	[60-63]	插补轴号
index	int	[0-∞]	索引信息，用于标识本次运动信息，一般设置为 0 即可
AxisList	int*		为平面圆弧插补坐标系内轴号
TargetPos	double*		为随动轴坐标系内轴号，如果为 0 则表示不跟随
CenterPos	double *		圆心坐标，单位:unit
dir	int	[0, 1]	圆弧方向：0，逆时针；1，顺时针；
PosType	int	[0, 1]	位置模式 0：相对位置 1：绝对位置

返回值	
0 表示执行成功，其他返回值表示执行失败，失败原因参考标准错误码	
指令分类	指令列表_插补运动
相关章节	
相关例程	

adt_inp_helix3_unit

指令格式

int _stdcall adt_inp_helix3_unit(int cardno, int InpAxis, int index, int AxisList[5], double Pos2List[3], double Pos3List[5], int ArcFlag, int PosType)			
指令描述			
任意三轴圆弧插补，二轴跟随			
参数说明			
名称	**数据类型**	**范围**	**说明**
cardno	int	[0-9]	控制卡索引号
InpAxis	int	[60-63]	插补轴号
index	int	0	索引信息，用于标识本次运动信息，一般设置为 0 即可
AxisList	int*		为空间圆弧插补坐标系内轴号
Pos2List	double		为空间圆弧的第二点坐标，单位:unit
Pos3List	double		为随动轴的目标点坐标，单位：unit
ArcFlag	int		0，圆弧；1，整圆
PosType	int	[0, 1]	位置模式 0：相对位置 1：绝对位置
返回值			
0 表示执行成功，其他返回值表示执行失败，失败原因参考标准错误码			
指令分类	指令列表_插补运动		
相关章节			
相关例程			

adt_nurbs_curve

指令格式			
int _stdcall adt_nurbs_curve(int cardno, int InpAxis, int PosType, int DataNum, int AxisList[3],double PosData[][3], int VectorMode, double StartVec[3], double EndVec[3], int InpMode, double InpPara[2])			
指令描述			
任意二轴或三轴样条曲线(nurbs)指令			
参数说明			
名称	**数据类型**	**范围**	**说明**
cardno	int	[0-9]	控制卡索引号

InpAxis	int	[60-63]	插补轴号
PosType	int	0	位置模式 0：相对位置 1：绝对位置
DataNum	int		用户输入数据点个数，即指明 PosData 数组中坐标点的个数
AxisList	int		轴号列表，若使用 2 轴平面插补，AxisList[0]、AxisList[1]设置轴号，第三轴设置为 0
PosData	double		用户输入拟合点坐标数组，单位 mm,数值范围[-10000,10000],3 列分别表示第一轴、第二轴和第三轴坐标值。
VectorMode	int		起点和终点切线确定模式
StartVec	double	[0, 1]	起点切线方向，是一个三维向量，模必须大于 0，可以是一般向量或单位方向向量。使用模式 1 和模式 4 时，必须指定起点和终点的切线方向
EndVec	double		终点切线方向，是一个三维向量，模必须大于 0，可以是一般向量或单位方向向量。使用模式 1 和模式 4 时，必须指定起点和终点的切线方向
InpMode	int		插补模式，设置 InpMode=3,按指定步长，插补轨迹不一定经过教导点；设置 InpMode=4，按指定步长插补，插补轨迹经过教导点。
InpPara	double		InpPara[0] 样条曲线插补的步长，单位 mm，需要设置合适的值（需大于 0.1mm），设置过大或过小有可能速度不顺滑，发生振动。 InpPara[1],保留参数，暂时未使用，赋值 0 即可。
返回值			
0 表示执行成功，其他返回值表示执行失败，失败原因参考标准错误码			

指令分类	指令列表_插补运动
相关章节	
相关例程	

18.9. PT

adt_set_pt_mode

指令格式			
int _stdcall adt_set_pt_mode (int cardno, int axis, int mode=0)			
指令描述			
设置 PT FIFO 模式			
参数说明			
名称	**数据类型**	**范围**	**说明**
cardno	int	[0-9]	控制卡索引号
axis	int	[1-MAXAXIS]	轴号
mode	int		0 静态，不清除 FIFO 数据 1 动态，执行后清除 FIFO 数据
返回值			
0 表示执行成功，其他返回值表示执行失败，失败原因参考标准错误码			
指令分类	指令列表_PT		
相关章节	点位模式		
相关例程			

adt_set_pt_data

指令格式			
int _stdcall adt_set_pt_data(int cardno, int axis, double pos, int time, int type)			
指令描述			
发送 PT 数据			
参数说明			
名称	**数据类型**	**范围**	**说明**
cardno	int	[0-9]	控制卡索引号
axis	int	[1-MAXAXIS]	轴号
pos	double*		位置，单位 mm,相对量，本段位置
time	int		时间，单位 ms，相对量，本段时间
type	int		0：普通段，两段直线首末速度相等，默认为该类型 1：匀速段

			2：减速段（速度减到 0）
返回值			
0 表示执行成功，其他返回值表示执行失败，失败原因参考标准错误码			
指令分类	指令列表_PT		
相关章节	点位模式		
相关例程			

adt_set_pt_table

指令格式			
int _stdcall adt_set_pt_table(int cardno, int axis, int count, double pos[], int time[], int type[])			
指令描述			
发送 PT 数据列表 需保证 PT 剩余缓存空间大于 count			
参数说明			
名称	**数据类型**	**范围**	**说明**
cardno	int	[0-9]	控制卡索引号
axis	int	[1-MAXAXIS]	轴号
count	double		PT 数据段个数，范围（1-1000）
pos	int*		位置，单位 mm,相对量，本段位置
time	int*		时间，单位 ms，相对量，本段时间
type	int*		0：普通段，两段直线首末速度相等，默认为该类型 1：匀速段 2：减速段（速度减到 0）
返回值			
0 表示执行成功，其他返回值表示执行失败，失败原因参考标准错误码			
指令分类	指令列表_PT		
相关章节	点位模式		
相关例程			

adt_clr_pt_data

指令格式

int _stdcall adt_clr_pt_data(int cardno, int axis)			
指令描述			
清空 PT 缓存			
参数说明			
名称	**数据类型**	**范围**	**说明**
cardno	int	[0-9]	控制卡索引号
axis	int	[1-MAXAXIS]	轴号
返回值			
0 表示执行成功，其他返回值表示执行失败，失败原因参考标准错误码			
指令分类	指令列表_PT		
相关章节	点位模式		
相关例程			

adt_get_pt_space

指令格式			
int _stdcall adt_get_pt_space(int cardno, int axis, int *space)			
指令描述			
获取 PT 缓冲区剩余段数			
参数说明			
名称	**数据类型**	**范围**	**说明**
cardno	int	[0-9]	控制卡索引号
axis	int	[1-MAXAXIS]	轴号
space	int*		PT 剩余缓存区大小 注意：PT 最大缓存 1000 段数据，下发 PT 数据前，注意查询缓存区大小
返回值			
0 表示执行成功，其他返回值表示执行失败，失败原因参考标准错误码			
指令分类	指令列表_PT		
相关章节	点位模式		
相关例程			

adt_get_pt_index

指令格式

int _stdcall adt_get_pt_index(int cardno, int axis, int *index)			
指令描述			
获取当前执行 PT 数据段索引			
参数说明			
名称	**数据类型**	**范围**	**说明**
cardno	int	[0-9]	控制卡索引号
axis	int	[1-MAXAXIS]	轴号
index	int*		当前执行 PT 数据段索引 注意：PT 最大缓存 1000 段数据，索引 1-1000
返回值			
0 表示执行成功，其他返回值表示执行失败，失败原因参考标准错误码			
指令分类	指令列表_PT		
相关章节	点位模式		
相关例程			

adt_pt_start

指令格式			
int _stdcall adt_pt_start(int cardno, int AxisNum, int AxisList[])			
指令描述			
启动 PT 运动			
参数说明			
名称	**数据类型**	**范围**	**说明**
cardno	int	[0-9]	控制卡索引号
AxisNum	int	[1-MAXAXIS]	启动的轴数
AxisList	int*		轴号列表,轴列表个数和 AxisNum 一致 注意：保证提前下发 PT 数据，PT 数据为空时，命令不执行
返回值			
0 表示执行成功，其他返回值表示执行失败，失败原因参考标准错误码			
指令分类	指令列表_PT		

相关章节	点位模式
相关例程	

adt_set_pt_loop

指令格式			
int _stdcall adt_set_pt_loop(int cardno, int axis, int loop)			
指令描述			
设置循环运行（只有 PT 静态模式有效）			
参数说明			
名称	数据类型	范围	说明
cardno	int	[0-9]	控制卡索引号
axis	int	[1-MAXAXIS]	启动的轴数
loop	int		循环次数，[0-100]，默认 0，即不循环，只运行一次 注意：PT 启动前，运动中，设置有效
返回值			
0 表示执行成功，其他返回值表示执行失败，失败原因参考标准错误码			
指令分类	指令列表_PT		
相关章节	点位模式		
相关例程			

adt_get_pt_loop

指令格式			
int _stdcall adt_get_pt_loop(int cardno, int axis, int*loop)			
指令描述			
获取循环运行执行次数			
参数说明			
名称	数据类型	范围	说明
cardno	int	[0-9]	控制卡索引号
axis	int	[1-MAXAXIS]	启动的轴数
loop	int*		获取循环运行执行次数
返回值			
0 表示执行成功，其他返回值表示执行失败，失败原因参考标准错误码			

指令分类	指令列表_PT
相关章节	点位模式
相关例程	

18.10. 回零控制类

adt_set_home_mode

指令格式			
int _stdcall adt_set_home_mode(int cardno, int axis, int mode, int stop0, int limit, int double back_range, double offset)			
指令描述			
DSP 回零模式设置			
参数说明			
名称	**数据类型**	**范围**	**说明**
cardno	int	[0-9]	控制卡索引号
axis	int	[1-MAXAXIS]	轴号
mode	int	[0-3]	回零模式，3 位二进制数 D0 – 回零方向 0:负向 1:正向 D1 – 原点位置 0:固定停留在原点信号靠近硬件正限位一侧 1:停留在首次触发原点信号的位置
stop0	int	[0, 1]	STOP0(机械原点)有效电平 0：低电平有效 1：高电平有效
limit	int	[0-7]	硬件限位模式，3 位二进制数 D0– 硬件限位有效电平 0:低电平 1:高电平 D1 – 硬件正限位模式 0:禁用 1:启用 D2 – 硬件负限位模式 0:禁用 1:启用
back_range	double		反退距离
offset	double		原点偏移
返回值			
0 表示执行成功，其他返回值表示执行失败，失败原因参考标准错误码			
指令分类	指令列表_回零控制类		
相关章节	DSP 回零		
相关例程	例程 12-1 DSP 回零		

adt_set_home_speed

指令格式			
int _stdcall adt_set_home_speed(int cardno, int axis, double startv, double searchv, double homev, double acc)			
指令描述			
DSP 回零速度设置			
参数说明			
名称	**数据类型**	**范围**	**说明**
cardno	int	[0-9]	控制卡索引号
axis	int	[1-MAXAXIS]	轴号
startv	double		回零起始速度
searchv	double		回零搜索高速
homev	double		回零搜索低速
acc	double		回零搜索加速度
返回值			
0 表示执行成功，其他返回值表示执行失败，失败原因参考标准错误码			
指令分类	指令列表_回零控制类		
相关章节	DSP 回零		
相关例程	例程 12-1 DSP 回零		

adt_set_home_process

指令格式			
int _stdcall adt_set_home_process(int cardno, int axis)			
指令描述			
单轴驱动回零，与 adt_get_home_status 组合完成单轴回零。			
参数说明			
名称	**数据类型**	**范围**	**说明**
cardno	int	[0-9]	控制卡索引号
axis	int	`	轴号
返回值			
0 表示执行成功，其他返回值表示执行失败，失败原因参考标准错误码			
指令分类	指令列表_回零控制类		

相关章节	DSP 回零
相关例程	例程 12-1 DSP 回零

adt_get_home_status

指令格式			
int _stdcall adt_get_home_status(int cardno, int axis)			
指令描述			
单轴外部回零状态查询，与 adt_set_home_process 组合完成单轴回零			
参数说明			
名称	数据类型	范围	说明
cardno	int	[0-9]	控制卡索引号
axis	int	[1-MAXAXIS]	轴号
返回值			
0:回零成功;-1 表示参数 1 错误;-2 表示参数 2 错误;-3 表示回零未启动; 1-8 表示执行的步骤,分别表示: 1 :快速接近原点,搜索 ORG 信号 2 :检查 ORG 信号是否找到 3 :反向退出原点 4 :检查反向退出原点是否完成 5 :低速接近原点,搜索原点信号 6 :检查 ORG 信号搜索是否完成 7 :原点偏移 8 :检查原点偏移 -100x:回零第 x 步出现异常,例如-1001 表示回零第 1 步出现异常			
指令分类	指令列表_回零控制类		
相关章节	DSP 回零		
相关例程	例程 12-1 DSP 回零		

adt_set_servo_home_para

指令格式
int _stdcall adt_set_servo_home_para(int cardno, int axis, int home_offset, int home_v1, int home_v2, int home_acc)
指令描述

伺服内部回零(SDO)参数设置			
参数说明			
名称	**数据类型**	**范围**	**说明**
cardno	int	[0-9]	控制卡索引号
axis	int	[1-MAXAXIS]	轴号
home_offset	int		回零原点偏移，0x607C，0x00
home_v1	int		回零第一速度，0x6099，0x01
home_v2	int		回零第二速度，0x6099，0x02
home_acc	int		回零加速度，0x609A，0x00
返回值			
0 表示执行成功，其他返回值表示执行失败，失败原因参考标准错误码			
指令分类	指令列表_回零控制类		
相关章节	总线伺服回零		
相关例程	例程 12-2 总线伺服回零		

adt_set_servo_home

指令格式			
int _stdcall adt_set_servo_home(int cardno, int axis, int home_mode)			
指令描述			
伺服内部回零(SDO)启动			
参数说明			
名称	**数据类型**	**范围**	**说明**
cardno	int	[0-9]	控制卡索引号
axis	int	[1-MAXAXIS]	轴号
home_mode	int	[0,35]	回零模式，具体含义参考任意总线伺服驱动器说明书，0x6098，0x00
返回值			
0 表示执行成功，其他返回值表示执行失败，失败原因参考标准错误码			
指令分类	指令列表_回零控制类		
相关章节	总线伺服回零		
相关例程	例程 12-2 总线伺服回零		

adt_get_servo_home_status

指令格式			
int _stdcall adt_get_servo_home_status(int cardno, int axis)			
指令描述			
伺服内部回零(SDO)状态检查			
参数说明			
名称	**数据类型**	**范围**	**说明**
cardno	int	[0-9]	控制卡索引号
axis	int	[1-MAXAXIS]	轴号
返回值			
0表示执行成功，其他返回值表示执行失败， 1：回零过程中　0：回零成功 -1：回零命令错误　-2：回零过程错误或超时　-3：回零过程中伺服报警			
指令分类	指令列表_回零控制类		
相关章节	总线伺服回零		
相关例程	例程 12-2 总线伺服回零		

adt_stop_servo_home

指令格式			
int _stdcall adt_stop_servo_home(int cardno, int axis)			
指令描述			
停止伺服回零			
参数说明			
名称	**数据类型**	**范围**	**说明**
cardno	int	[0-9]	控制卡索引号
axis	int	[1-MAXAXIS]	轴号
返回值			
0：正确；非零：失败；			
指令分类	指令列表_回零控制类		
相关章节	总线伺服回零		
相关例程	例程 12-2 总线伺服回零		

18.11. 总线 IO 控制类

adt_read_servo_inbit

指令格式			
int _stdcall adt_read_servo_inbit(int cardno, int axis, int index, int *state)			
指令描述			
读取伺服自带输入端口电平 输入端口的状态电平与从站常态有关，若从站的输入端口常态下是高电平，那么状态为 0 时为高电平，1 时为低电平			
参数说明			
名称	**数据类型**	**范围**	**说明**
cardno	int	[0-9]	控制卡索引号
axis	int	[1-MAXAXIS]	轴号
index	int	[0-15]	输入点索引，从 0 开始
state	int*	[0,1]	当前电平 0：常态　　1：非常态
返回值			
0 表示执行成功，其他返回值表示执行失败，失败原因参考标准错误码			
指令分类	指令列表_总线 IO 控制类		
相关章节	伺服 IO 控制		
相关例程			

adt_read_servo_inport

指令格式			
int _stdcall adt_read_servo_inport(int cardno, int axis, int group, int *inmap)			
指令描述			
按组读取伺服自带输入端口电平 输入端口的状态电平与从站常态有关，若从站的输入端口常态下是高电平，那么状态为 0 时为高电平，1 时为低电平			
参数说明			
名称	**数据类型**	**范围**	**说明**
cardno	int	[0-9]	控制卡索引号
axis	int	[1-MAXAXIS]	轴号

group	int		输入点组号，8 位一组，从 0 开始
inmap	int*	[0-255]	输入状态：bit0--bit7，按位映射 0：常态　　1：非常态
返回值			
0 表示执行成功，其他返回值表示执行失败，失败原因参考标准错误码			
指令分类	指令列表_总线 IO 控制类		
相关章节	伺服 IO 控制		
相关例程			

adt_write_servo_outbit

指令格式			
int _stdcall adt_write_servo_outbit(int cardno, int axis, int index, int value)			
指令描述			
操作总线伺服 IO 输出点			
参数说明			
名称	**数据类型**	**范围**	**说明**
cardno	int	[0-9]	控制卡索引号
axis	int	[1-MAXAXIS]	轴号
index	int		输出点索引，从 0 开始
value	int*	[0,1]	输出状态： 0：关闭　　1：打开
返回值			
0 表示执行成功，其他返回值表示执行失败，失败原因参考标准错误码			
指令分类	指令列表_总线 IO 控制类		
相关章节	伺服 IO 控制		
相关例程			

adt_read_servo_outbit

指令格式
int _stdcall adt_read_servo_outbit(int cardno, int axis, int index, int*value)
指令描述
读取总线伺服 IO 输出点状态
参数说明

名称	数据类型	范围	说明
cardno	int	[0-9]	控制卡索引号
axis	int	[1-MAXAXIS]	轴号
index	int		输出点索引，从 0 开始
value	int*	[0,1]	输出状态： 0：关闭　　1：打开
返回值			
0 表示执行成功，其他返回值表示执行失败，失败原因参考标准错误码			
指令分类	指令列表_总线 IO 控制类		
相关章节	伺服 IO 控制		
相关例程			

adt_read_busio_inbit

指令格式
int _stdcall adt_read_busio_inbit(int cardno, int node, int index, int*state)
指令描述
按节点号读取总线 IO 输入点状态 输入端口的状态电平与从站常态有关，若从站的输入端口常态下是高电平，那么状态为 0 时为高电平，1 时为低电平
参数说明

名称	数据类型	范围	说明
cardno	int	[0-9]	控制卡索引号
node	int	[1-32]	IO 节点号，从 1 开始
index	int		输入点索引，从 0 开始
state	int*	[0,1]	0：常态，1：非常态
返回值			
0 表示执行成功，其他返回值表示执行失败，失败原因参考标准错误码			
指令分类	指令列表_总线 IO 控制类		
相关章节	总线 IO 模块控制		
相关例程			

adt_read_busio_inport

指令格式

int _stdcall adt_read_busio_inport(int cardno, int node, int group, int *inmap)			
指令描述			
按节点号读取总线 IO 输入点状态 输入端口的状态电平与从站常态有关，若从站的输入端口常态下是高电平，那么状态为 0 时为高电平，1 时为低电平			
参数说明			
名称	**数据类型**	**范围**	**说明**
cardno	int	[0-9]	控制卡索引号
node	int	[1-32]	节点号
group	int		输入点索引，从 0 开始
inmap	int*	[0-255]	输入状态：bit0--bit7，按位映射 0：常态　　1：非常态
返回值			
0 表示执行成功，其他返回值表示执行失败，失败原因参考标准错误码			
指令分类	指令列表_总线 IO 控制类		
相关章节	总线 IO 模块控制		
相关例程			

adt_write_busio_outbit

指令格式			
int _stdcall adt_write_busio_outbit(int cardno, int node, int index, int value)			
指令描述			
按节点号操作总线 IO 输出点			
参数说明			
名称	**数据类型**	**范围**	**说明**
cardno	int	[0-9]	控制卡索引号
node	int	[1-32]	IO 节点号，从 1 开始
index	int		输入点索引，从 0 开始
value	int	[0,1]	0：关闭，1：打开
返回值			
0 表示执行成功，其他返回值表示执行失败，失败原因参考标准错误码			
指令分类	指令列表_总线 IO 控制类		

相关章节	总线 IO 模块控制
相关例程	

adt_write_busio_outport

指令格式			
int _stdcall adt_write_busio_outport(int cardno, int node, int group, int outmap)			
指令描述			
按组操作总线 IO 输出点			
参数说明			
名称	数据类型	范围	说明
cardno	int	[0-9]	控制卡索引号
node	int	[1-32]	IO 节点号，从 1 开始
group	int		输出点组号，8 位一组，从 0 开始
outmap	int	[0-255]	输出点，bit0-bit7 有效，按位映射 0：关闭，1：打开
返回值			
0 表示执行成功，其他返回值表示执行失败，失败原因参考标准错误码			
指令分类	指令列表_总线 IO 控制类		
相关章节	总线 IO 模块控制		
相关例程			

adt_read_busio_outbit

指令格式			
int _stdcall adt_read_busio_outbit(int cardno, int node, int index, int*value)			
指令描述			
按读节点号读取总线 IO 输出点状态			
参数说明			
名称	数据类型	范围	说明
cardno	int	[0-9]	控制卡索引号
node	int	[1-32]	IO 节点号，从 1 开始
index	int		输入点索引，从 0 开始
value	int	[0,1]	0：关闭，1：打开
返回值			

0 表示执行成功，其他返回值表示执行失败，失败原因参考标准错误码	
指令分类	指令列表_总线 IO 控制类
相关章节	总线 IO 模块控制
相关例程	

adt_read_busio_outport

指令格式			
int _stdcall adt_read_busio_outport(int cardno, int node, int group, int*outmap)			
指令描述			
按组操作总线 IO 输出点			
参数说明			
名称	**数据类型**	**范围**	**说明**
cardno	int	[0-9]	控制卡索引号
node	int	[1-32]	IO 节点号，从 1 开始
group	int		输出点组号，8 位一组，从 0 开始
outmap	int	[0-255]	输出点，bit0-bit7 有效，按位映射 0：关闭，1：打开
返回值			
0 表示执行成功，其他返回值表示执行失败，失败原因参考标准错误码			
指令分类	指令列表_总线 IO 控制类		
相关章节	总线 IO 模块控制		
相关例程			

18.12. 缓存事件控制类

adt_get_fifo_event_len

指令格式			
int _stdcall adt_get_fifo_event_len(int cardno, int InpAxis, int *len)			
指令描述			
获取缓存事件的剩余段数。			
参数说明			
名称	**数据类型**	**范围**	**说明**
cardno	int	[0-9]	控制卡索引号
InpAxis	int	[60-63]	插补轴号
len	int *		获取回来的剩余段数
返回值			
0 表示执行成功，其他返回值表示执行失败，失败原因参考标准错误码			
指令分类	指令列表_缓存事件控制类		
相关章节	缓存事件余量		
相关例程	例程 14-1 缓存事件控制		

adt_get_fifo_event_index

指令格式			
int _stdcall adt_get_fifo_event_index(int cardno, int InpAxis, int *index)			
指令描述			
获取最新触发缓存事件的索引号。			
参数说明			
名称	**数据类型**	**范围**	**说明**
cardno	int	[0-9]	控制卡索引号
InpAxis	int	[60-63]	插补轴号
index	int *		当前执行缓存事件的索引
返回值			
0 表示执行成功，其他返回值表示执行失败，失败原因参考标准错误码			
指令分类	指令列表_缓存事件控制类		
相关章节			
相关例程			

adt_set_fifo_adjust_time

指令格式			
int _stdcall adt_set_fifo_adjust_time(int cardno, int InpAxis, double mstime)			
指令描述			
设置提前或滞后触发时间			
参数说明			
名称	**数据类型**	**范围**	**说明**
cardno	int	[0-9]	控制卡索引号
InpAxis	int	[60-63]	插补轴号
mstime	double		时间：单位（ms）,大于 0 滞后触发，小于 0 提前触发。
返回值			
0 表示执行成功，其他返回值表示执行失败，失败原因参考标准错误码			
指令分类	指令列表_缓存事件控制类		
相关章节			
相关例程			

adt_set_fifo_adjust_pos

指令格式			
int _stdcall adt_set_fifo_adjust_pos(int cardno, int InpAxis, double pos)			
指令描述			
设置提前或滞后触发位置			
参数说明			
名称	**数据类型**	**范围**	**说明**
cardno	int	[0-9]	控制卡索引号
InpAxis	int	[60-63]	插补轴号
pos	double		插补合成位置，大于 0 滞后触发，小于 0 提前触发。
返回值			
0 表示执行成功，其他返回值表示执行失败，失败原因参考标准错误码			
指令分类	指令列表_缓存事件控制类		
相关章节			

相关例程	

adt_set_fifo_outbit

指令格式			
int _stdcall adt_set_fifo_outbit(int cardno, int InpAxis, int index, int IoNode, int IoIdx, int value, double speed=-1)			
指令描述			
设置缓存输出事件			
参数说明			
名称	**数据类型**	**范围**	**说明**
cardno	int	[0-9]	控制卡索引号
InpAxis	int	[60-63]	插补轴号
index	int	[0,999]	缓存事件索引
IoNode	int	[1-MAXAXIS]	缓存控制总线 IO 模块节点号
IoIdx	int	[0-15]	缓存控制端口号
value	int	[0, 1]	缓存控制电平 0:关闭 1：打开
speed	double		默认-1，IO 动作前不作速度约束，其它：范围（0.0-5000mm/sec）
返回值			
0 表示执行成功，其他返回值表示执行失败，失败原因参考标准错误码			
指令分类	指令列表_缓存事件控制类		
相关章节	缓存输出控制		
相关例程			

adt_set_fifo_multi_io

指令格式			
int _stdcall adt_set_fifo_multi_io(int cardno, int InpAxis, int index, int IoNode, int IoGp, int EnMap, int OutMap, double speed=-1)			
指令描述			
多路缓存输出控制			
参数说明			
名称	**数据类型**	**范围**	**说明**

cardno	int	[0-9]	控制卡索引号
InpAxis	int	[60-63]	插补轴号
index	int	[0,999]	缓存事件索引
IoNode	int	[1-MAXAXIS]	输出 IO 节点号，[1-MaxNode]
IoGp	int		输出点组号，8 位一组，从 0 开始
EnMap	int		按位（bit0-bit7）指定要操作的输出点： 位值为 1 则对相应的输出点进行操作； 位值为 0 则输出点不收影响；
OutMap	int		该组输出点的状态设置，按位映射： 0：关闭，1：打开
speed	double		默认为-1，IO 动作前不作速度约束，其他： 范围（0.0-5000mm/sec）

返回值
0 表示执行成功，其他返回值表示执行失败，失败原因参考标准错误码

指令分类	指令列表_缓存事件控制类
相关章节	缓存输出控制
相关例程	例程 14-1 缓存事件控制

adt_set_fifo_delay

指令格式
int _stdcall adt_set_fifo_delay(int cardno, int InpAxis, int index, int millisecond)
指令描述
设置缓存运动暂停事件
参数说明

名称	数据类型	范围	说明
cardno	int	[0-9]	控制卡索引号
InpAxis	int	[60-63]	插补轴号
index	int	[0-999]	缓存事件索引，用于标识本次缓存信息，一般设置为 0 即可。
millisecond	int		暂停时间，单位：ms

返回值	
0 表示执行成功，其他返回值表示执行失败，失败原因参考标准错误码	
指令分类	指令列表_缓存事件控制类
相关章节	缓存延时控制
相关例程	例程 14-1 缓存事件控制

adt_set_fifo_wait_inbit

指令格式
int _stdcall adt_set_fifo_wait_inbit (int cardno, int InpAxis, int index, int node, int IoIdx, int level, double timeout = 0)
指令描述
设置插补缓存运动暂停等待输入信号

参数说明

名称	数据类型	范围	说明
cardno	int	[0-9]	控制卡索引号
InpAxis	int	[60-63]	插补轴号
index	int	[0-999]	缓存事件索引，用于标识本次缓存信息，一般设置为 0 即可。
node	int	[1-MaxNode]	IO 板索引号
IoIdx	int	[0-MaxIdx]	输出点索引
level	int	[0-1]	输入信号有效电平 0：低电平 1：高电平
timeout	double	[0-65535]	等待超时时间，等于 0 为一直等待，单位：ms

返回值	
0 表示执行成功，其他返回值表示执行失败，失败原因参考标准错误码	
指令分类	指令列表_缓存事件控制类
相关章节	缓存运动暂停等待输入
相关例程	

adt_clear_fifo_event

指令格式
int _stdcall adt_clear_fifo_event(int cardno, int InpAxis)
指令描述
缓存事件清空

参数说明			
名称	**数据类型**	**范围**	**说明**
cardno	int	[0-9]	控制卡索引号
inp_axis	int	[60-63]	插补轴号
返回值			
0 表示执行成功，其他返回值表示执行失败，失败原因参考标准错误码			
指令分类	指令列表_缓存事件控制类		
相关章节			
相关例程			

18.13. 从站仿真控制类

adt_set_axis_resource

指令格式			
int _stdcall adt_set_axis_resource(int cardno, int AxisNum)			
指令描述			
设置规划轴资源数			
参数说明			
名称	**数据类型**	**范围**	**说明**
cardno	int	[0-9]	控制卡索引号
AxisNum	int	[0-MAXAXIS]	运动规划轴资源数
返回值			
0 表示执行成功，其他返回值表示执行失败，失败原因参考标准错误码			
指令分类	指令列表_从站仿真控制类		
相关章节			
相关例程			

adt_get_axis_resource

指令格式			
int _stdcall adt_get_axis_resource(int cardno, int*AxisNum)			
指令描述			
获取规划轴资源数			
参数说明			
名称	**数据类型**	**范围**	**说明**
cardno	int	[0-9]	控制卡索引号
AxisNum	int*		运动规划轴资源数
返回值			
0 表示执行成功，其他返回值表示执行失败，失败原因参考标准错误码			
指令分类	指令列表_从站仿真控制类		
相关章节			
相关例程			

adt_set_axis_bound

指令格式

int _stdcall adt_set_axis_bound(int cardno, int axis, int type, int channel)			
指令描述			
设置规划轴资源绑定输出通道			
参数说明			
名称	**数据类型**	**范围**	**说明**
cardno	int	[0-9]	控制卡索引号
axis	int	[1-MAXAXIS]	运动规划轴号
type	int	[0-2]	轴类型：0，虚拟轴；1，总线轴输出；2，脉冲轴输出
channel	int		输出通道号，虚拟轴参数无意义，总线轴[1-MAXAXIS]
返回值			
0 表示执行成功，其他返回值表示执行失败，失败原因参考标准错误码			
指令分类	指令列表_从站仿真控制类		
相关章节			
相关例程			

adt_get_axis_bound

int _stdcall adt_get_axis_bound(int cardno, int axis, int*type, int*channel)			
指令描述			
获取规划轴资源绑定输出通道			
参数说明			
名称	**数据类型**	**范围**	**说明**
cardno	int	[0-9]	控制卡索引号
axis	int	[1-MAXAXIS]	运动规划轴号
type	int	[0-2]	轴类型：0，虚拟轴；1，总线轴输出；2，脉冲轴输出
channel	int		输出通道号，虚拟轴参数无意义，总线轴[1-MAXAXIS]
返回值			
0 表示执行成功，其他返回值表示执行失败，失败原因参考标准错误码			
指令分类	指令列表_从站仿真控制类		

相关章节	
相关例程	

adt_reset_axis_bound

指令格式			
int _stdcall adt_reset_axis_bound(int cardno)			
指令描述			
重置规划轴资源绑定输出通道			
参数说明			
名称	数据类型	范围	说明
cardno	int	[0-9]	控制卡索引号
返回值			
0 表示执行成功，其他返回值表示执行失败，失败原因参考标准错误码			
指令分类	指令列表_从站仿真控制类		
相关章节			
相关例程			

adt_set_virtual_io_num

指令格式			
int _stdcall adt_set_virtual_io_num(int cardno, int ChannelNum)			
指令描述			
配置虚拟 IO 资源通道			
参数说明			
名称	数据类型	范围	说明
cardno	int	[0-9]	控制卡索引号
ChannelNum	int	[0-4]	虚拟 IO 资源通道数
返回值			
0 表示执行成功，其他返回值表示执行失败，失败原因参考标准错误码			
指令分类	指令列表_从站仿真控制类		
相关章节			
相关例程			

adt_get_virtual_io_num

指令格式

int _stdcall adt_get_virtual_io_num(int cardno, int*ChannelNum)			
指令描述			
获取虚拟 IO 资源通道			
参数说明			
名称	**数据类型**	**范围**	**说明**
cardno	int	[0-9]	控制卡索引号
ChannelNum	int*		虚拟 IO 资源通道数
返回值			
0 表示执行成功，其他返回值表示执行失败，失败原因参考标准错误码			
指令分类	指令列表_从站仿真控制类		
相关章节			
相关例程			

adt_write_busio_inbit

指令格式			
int _stdcall adt_write_busio_inbit(int cardno, int node, int index, int state)			
指令描述			
按节点号设置总线 IO 输入点状态			
参数说明			
名称	**数据类型**	**范围**	**说明**
cardno	int	[0-9]	控制卡索引号
node	int	[1-32]	IO 节点号
index	int		输入点索引，从 0 开始
state	int	[0,1]	0，常态；1，非常态
返回值			
0 表示执行成功，其他返回值表示执行失败，失败原因参考标准错误码			
指令分类	指令列表_从站仿真控制类		
相关章节			
相关例程			

adt_write_busio_inport

指令格式
int _stdcall adt_write_busio_inport(int cardno, int node, int group, int inmap)

指令描述			
按节点号设置总线 IO 输入点状态			
参数说明			
名称	**数据类型**	**范围**	**说明**
cardno	int	[0-9]	控制卡索引号
node	int	[1-32]	IO 节点号
group	int		输入点组号，8 位一组，从 0 开始
inmap	int	[0-255]	输入状态，bit0-bit7 有效，按位映射
返回值			
0 表示执行成功，其他返回值表示执行失败，失败原因参考标准错误码			
指令分类	指令列表_从站仿真控制类		
相关章节			
相关例程			

18.14. SDO/PDO 类

adt_ecat_sdo_read

指令格式			
int _stdcall adt_ecat_sdo_read(int cardno, int node, unsigned int index, unsigned int subIndex, unsigned char *pData, unsigned int datalen)			
指令描述			
Ethercat SDO 数据读操作			
参数说明			
名称	**数据类型**	**范围**	**说明**
cardno	int	[0-9]	控制卡索引号
node	int	[1-32]	从站节点号
index	unsigned int	[0,1]	对象字典索引
subIndex	unsigned int		对象字典子索引
pData	unsigned char*		数据指针，这里使用 U8 类型指针操作数据，其他长度数据，定义对应长度的 U8 数据即可
datalen	unsigned int	[1,2,4]	数据长度
返回值			
0 表示执行成功，120 失败，失败原因对象字典数据从站不支持，或者数据长度有误，其他返回值表示执行失败，失败原因参考标准错误码			
指令分类	指令列表_SDO/PDO		
相关章节	SDO/PDO		
相关例程			

adt_ecat_sdo_write

指令格式
int _stdcall adt_ecat_sdo_write(int cardno, int node, unsigned int index, unsigned int subIndex, unsigned char *pData, unsigned int datalen)
指令描述
Ethercat SDO 数据写操作

参数说明			
名称	**数据类型**	**范围**	**说明**
cardno	int	[0-9]	控制卡索引号
node	int	[1-32]	从站节点号
index	unsigned int	[0,1]	对象字典索引
subIndex	unsigned int		对象字典子索引
pData	unsigned char*		数据指针，这里使用 U8 类型指针操作数据，其他长度数据，定义对应长度的 U8 数据即可
datalen	unsigned int	[1,2,4]	数据长度
返回值			
0 表示执行成功，120 失败，失败原因对象字典数据从站不支持，或者数据长度有误，其他返回值表示执行失败，失败原因参考标准错误码			
指令分类	指令列表_SDO/PDO		
相关章节	SDO/PDO		
相关例程			

adt_ecat_pdo_read

指令格式
int _stdcall adt_ecat_pdo_read(int cardno, unsigned int addr, unsigned char *pData, unsigned int datalen)
指令描述
Ethercat 过程数据读操作

参数说明			
名称	**数据类型**	**范围**	**说明**
cardno	int	[0-9]	控制卡索引号
addr	unsigned int		数据地址，为配置工具中设置的地址
pData	unsigned char*		数据指针，这里使用 U8 类型指针操作数据，其他长度数据，定义对应长度的 U8 数据即可

datalen	unsigned int		数据长度，只能是 1,2,4
返回值			
0 表示执行成功，120 失败，失败原因对象字典数据从站不支持，或者数据长度有误，其他返回值表示执行失败，失败原因参考标准错误码			
指令分类	指令列表_SDO/PDO		
相关章节	SDO/PDO		
相关例程			

adt_ecat_pdo_write

指令格式			
int _stdcall adt_ecat_pdo_write(int cardno, unsigned int addr, unsigned char *pData, unsigned int datalen)			
指令描述			
Ethercat 过程数据写操作			
参数说明			
名称	**数据类型**	**范围**	**说明**
cardno	int	[0-9]	控制卡索引号
addr	unsigned int		数据地址，为配置工具中设置的地址
pData	unsigned char*		数据指针，这里使用 U8 类型指针操作数据，其他长度数据，定义对应长度的 U8 数据即可
datalen	unsigned int		数据长度，只能是 1,2,4
返回值			
0 表示执行成功，120 失败，失败原因对象字典数据从站不支持，或者数据长度有误，其他返回值表示执行失败，失败原因参考标准错误码			
指令分类	指令列表_SDO/PDO		
相关章节	SDO/PDO		
相关例程			

18.15. 17.14 用户控制类

adt_set_user_password

指令格式			
int _stdcall adt_set_user_password(int cardno, const char* oldpw, const char* newpw)			
指令描述			
修改/设置用户密码			
参数说明			
名称	**数据类型**	**范围**	**说明**
cardno	int	[0-9]	控制卡索引号
oldpw	const char*		旧密码校验，首次使用该功能时不会校验该密码
newpw	const char*		新密码设置
返回值			
0 表示执行成功，其他返回值表示执行失败，失败原因参考标准错误码			
指令分类	指令列表_用户控制类		
相关章节	用户控制		
相关例程	例程 16-1 用户控制		

adt_check_password

指令格式			
int _stdcall adt_check_password(int cardno, const char* password)			
指令描述			
校验用户密码			
参数说明			
名称	**数据类型**	**范围**	**说明**
cardno	int	[0-9]	控制卡索引号
password	const char*		校验密码
返回值			
0 表示执行成功，其他返回值表示执行失败，失败原因参考标准错误码			
指令分类	指令列表_用户控制类		
相关章节	用户控制		
相关例程	例程 16-1 用户控制		

18.16. ADT-9188 控制

adt_write_outbit

指令格式			
int _stdcall adt_write_outbit(int cardno, int index, int value)			
指令描述			
9188 本地输出点操作			
参数说明			
名称	**数据类型**	**范围**	**说明**
cardno	int	[0-9]	控制卡索引号
index	int	[1-7]	输出点索引号
value	int	[0,1]	0，关闭；1，打开
返回值			
0 表示执行成功，其他返回值表示执行失败，失败原因参考标准错误码			
指令分类	指令列表_ADT9188 扩展板控制		
相关章节	高速 IO 端口		
相关例程			

adt_read_outbit

指令格式			
int _stdcall adt_read_outbit(int cardno, int index, int*status)			
指令描述			
9188 本地输出点操作			
参数说明			
名称	**数据类型**	**范围**	**说明**
cardno	int	[0-9]	控制卡索引号
index	int	[1-7]	输出点索引号
status	int*	[0,1]	0，关闭；1，打开
返回值			
0 表示执行成功，其他返回值表示执行失败，失败原因参考标准错误码			
指令分类	指令列表_ADT9188 扩展板控制		
相关章节	高速 IO 端口		
相关例程			

adt_read_inport

指令格式			
int _stdcall adt_read_inport(int cardno, int*inmap)			
指令描述			
9188 本地输入按组获取状态			
参数说明			
名称	**数据类型**	**范围**	**说明**
cardno	int	[0-9]	控制卡索引号
inmap	int*	[0-255]	输入状态，bit0-bit7 有效，按位映射
返回值			
0 表示执行成功，其他返回值表示执行失败，失败原因参考标准错误码			
指令分类	指令列表_ADT9188 扩展板控制		
相关章节	高速 IO 端口		
相关例程			

adt_read_inbit

指令格式			
int _stdcall adt_read_inbit(int cardno, int index, int *status)			
指令描述			
9188 本地输入点状态获取			
参数说明			
名称	**数据类型**	**范围**	**说明**
cardno	int	[0-9]	控制卡索引号
index	int	[0-7]	输入点索引号
status	int*	[0,1]	0，关闭；1，打开
返回值			
0 表示执行成功，其他返回值表示执行失败，失败原因参考标准错误码			
指令分类	指令列表_ADT9188 扩展板控制		
相关章节	高速 IO 端口		
相关例程			

adt_set_input_filter

指令格式

int _stdcall adt_set_input_filter(int cardno, int msec)			
指令描述			
设置 9188 输入信号滤波时间			
参数说明			
名称	**数据类型**	**范围**	**说明**
cardno	int	[0-9]	控制卡索引号
msec	int	[0-15]	滤波时间 0-15ms，默认 1ms
返回值			
0 表示执行成功，其他返回值表示执行失败，失败原因参考标准错误码			
指令分类	指令列表_ADT9188 扩展板控制		
相关章节	高速 IO 端口		
相关例程			

adt_get_compare_len

指令格式			
int _stdcall adt_get_compare_len(int cardno, int cmp, int *len)			
指令描述			
获取位置比较器余量			
参数说明			
名称	**数据类型**	**范围**	**说明**
cardno	int	[0-9]	控制卡索引号
cmp	int	[0-3]	对应 OUT0-OUT3 4 个高速输出端口
len	int*		剩余未完成位置比较的点数
返回值			
0 表示执行成功，其他返回值表示执行失败，失败原因参考标准错误码			
指令分类	指令列表_ADT9188 扩展板控制		
相关章节	位置比较		
相关例程	例程 17-1 2D 位置比较		

adt_get_compare_status

指令格式
int _stdcall adt_get_compare_status(int cardno, int cmp, int *count)
指令描述

获取比较触发状态			
参数说明			
名称	**数据类型**	**范围**	**说明**
cardno	int	[0-9]	控制卡索引号
cmp	int	[0-3]	对应 OUT0-OUT3 4 个高速输出端口
count	int*		已经完成位置比较的点数量
返回值			
0 表示执行成功，其他返回值表示执行失败，失败原因参考标准错误码			
指令分类	指令列表_ADT9188 扩展板控制		
相关章节	位置比较		
相关例程	例程 17-1 2D 位置比较		

adt_clear_compare_point

指令格式			
int _stdcall adt_clear_compare_point(int cardno, int cmp)			
指令描述			
清除位置比较器的所有比较点，复位比较器			
参数说明			
名称	**数据类型**	**范围**	**说明**
cardno	int	[0-9]	控制卡索引号
cmp	int	[0-3]	对应 OUT0-OUT3 4 个高速输出端口
返回值			
0 表示执行成功，其他返回值表示执行失败，失败原因参考标准错误码			
指令分类	指令列表_ADT9188 扩展板控制		
相关章节	位置比较		
相关例程	例程 17-1 2D 位置比较		

adt_reset_compare

指令格式
int _stdcall adt_reset_compare(int cardno, int cmp)
指令描述
关闭位置比较器并清空状态
参数说明

名称	数据类型	范围	说明
cardno	int	[0-9]	控制卡索引号
cmp	int	[0-3]	对应 OUT0-OUT3 4 个高速输出端口
返回值			
0 表示执行成功，其他返回值表示执行失败，失败原因参考标准错误码			
指令分类	指令列表_ADT9188 扩展板控制		
相关章节	位置比较		
相关例程	例程 17-1 2D 位置比较		

adt_set_compare_xd_mode

指令格式
int _stdcall adt_set_compare_xd_mode(int cardno, int cmp, int enable, int xd, int axis[], int source, int mode, int level)
指令描述
设置位置比较器模式

参数说明			
名称	数据类型	范围	说明
cardno	int	[0-9]	控制卡索引号
cmp	int	[0-3]	对应 OUT0-OUT3 4 个高速输出端口
enable	int	[0,1]	1，打开位置比较；0，关闭位置比较
xd	int	[1-3]	位置比较的维数
Axis[]	int*		轴列表
source	int	[0,1]	比较源： 0，逻辑位置（暂未开放）； 1，实际位置（编码器位置）
mode	int*	[0-2]	IO 输出模式： 0，反转（比较器触发一次，电平反转一次） 1，脉宽输出（输出状态持续指定时间后翻转） 2，PWM 脉冲输出，可设定每个比较点输出的 PWM 频率
level	int	[0,1]	输出点常态：0，关闭；1，打开
返回值			
0 表示执行成功，其他返回值表示执行失败，失败原因参考标准错误码			

指令分类	指令列表_ADT9188 扩展板控制
相关章节	位置比较
相关例程	例程 17-1 2D 位置比较

adt_get_compare_xd_mode

指令格式
int _stdcall adt_get_compare_xd_mode(int cardno, int cmp, int *enable, int *xd, int axis[], int *source, int *mode, int *level)
指令描述
获取位置比较器模式
参数说明

名称	数据类型	范围	说明
cardno	int	[0-9]	控制卡索引号
cmp	int	[0-3]	对应输出端口 0-3
enable	int*		1，打开位置比较；0，关闭位置比较
xd	int*		位置比较的维数
axis[]	int		轴列表
source	int*		比较源：0，逻辑位置；1，实际位置（编码器位置）
mode	int*		IO 输出模式： 0，反转（比较器触发一次，电平反转一次） 1，脉宽输出（输出状态持续指定时间后翻转） 2，PWM 脉冲输出，可设定每个比较点输出的 PWM 频率
level	int*		输出点常态：0，关闭；1，打开

返回值
0 表示执行成功，其他返回值表示执行失败，失败原因参考标准错误码

指令分类	指令列表_ADT9188 扩展板控制
相关章节	位置比较
相关例程	例程 17-1 2D 位置比较

adt_set_compare_xd_area

指令格式
int _stdcall adt_set_compare_xd_area(int cardno, int cmp, long area[])
指令描述

设定 2/3D 触发区域范围			
参数说明			
名称	**数据类型**	**范围**	**说明**
cardno	int	[0-9]	控制卡索引号
cmp	int	[0-3]	对应 OUT0-OUT3 4 个高速输出端口
area	long*		比较区域范围，单位：脉冲
返回值			
0 表示执行成功，其他返回值表示执行失败，失败原因参考标准错误码			
指令分类	指令列表_ADT9188 扩展板控制		
相关章节	位置比较		
相关例程	例程 17-1 2D 位置比较		

adt_add_compare_xd_point

指令格式			
int _stdcall adt_add_compare_xd_point(int cardno, int cmp, long pos[], double time_h, double time_l)			
指令描述			
添加位置比较位置点			
参数说明			
名称	**数据类型**	**范围**	**说明**
cardno	int	[0-9]	控制卡索引号
cmp	int	[0-3]	对应 OUT0-OUT3 4 个高速输出端口
pos	long*		比较位置点，下标 0,1,2 和模式设置的轴号映射，位置模式为绝对位置，单位：pulse
time_h	double		PWM 脉冲输出，OUT 打开时间，单位：ms
time_l	double		PWM 脉冲输出，OUT 打开时间，单位：ms, 只有在 PWM 输出模式下，此参数有效
返回值			
0 表示执行成功，其他返回值表示执行失败，失败原因参考标准错误码			
指令分类	指令列表_ADT9188 扩展板控制		
相关章节	位置比较		
相关例程	例程 17-1 2D 位置比较		

adt_set_encoder_mode

指令格式			
int _stdcall adt_set_encoder_mode(int cardno, int encd, int type, int dir)			
指令描述			
设置 9188 外部编码器工作模式			
参数说明			
名称	**数据类型**	**范围**	**说明**
cardno	int	[0-9]	控制卡索引号
encd	int	[1-8]	编码器接收端口号
type	int	[0,1]	0：A/B 脉冲输入；1：上/下（PPIN/PMIN）脉冲输入
dir	int		0：正逻辑方向；1：负逻辑方向
返回值			
0 表示执行成功，其他返回值表示执行失败，失败原因参考标准错误码			
指令分类	指令列表_ADT9188 扩展板控制		
相关章节			
相关例程			

adt_get_endoder_pos

指令格式			
int _stdcall adt_get_encoder_pos(int cardno, int encd, long *pos)			
指令描述			
获取 9180 扩展编码器位置			
参数说明			
名称	**数据类型**	**范围**	**说明**
cardno	int	[0-9]	控制卡索引号
encd	int	[1-8]	编码器接收端口号
pos	long*		位置，单位：pulse
返回值			
0 表示执行成功，其他返回值表示执行失败，失败原因参考标准错误码			
指令分类	指令列表_ADT9188 扩展板控制		
相关章节			
相关例程			

adt_set_endoder_pos

指令格式			
int _stdcall adt_set_encoder_pos(int cardno, int encd, long pos)			
指令描述			
设置 9180 扩展编码器位置			
参数说明			
名称	**数据类型**	**范围**	**说明**
cardno	int	[0-9]	控制卡索引号
encd	int	[1-8]	编码器接收端口号
pos	long*		位置，单位：pulse
返回值			
0 表示执行成功，其他返回值表示执行失败，失败原因参考标准错误码			
指令分类	指令列表_ADT9188 扩展板控制		
相关章节			
相关例程			

18.17. 辅助功能

adt_get_bit

指令格式			
int _stdcall adt_get_bit(int value, int bit)			
指令描述			
读取整型数的指定位的值			
参数说明			
名称	**数据类型**	**范围**	**说明**
value	int		目标数
bit	int		指定位
返回值			
整型数的指定位的值			
指令分类	指令列表_辅助功能		
相关章节			
相关例程			

adt_set_bit

指令格式			
void _stdcall adt_set_bit(int *value, int bit)			
指令描述			
指定位写 1			
参数说明			
名称	**数据类型**	**范围**	**说明**
value	int*		目标数（并得到设定好的数）
bit	int		指定位
返回值			
指令分类			
相关章节			
相关例程			

adt_clr_bit

指令格式

void _stdcall adt_clr_bit(int *value, int bit)			
指令描述			
指定位写 0			
参数说明			
名称	**数据类型**	**范围**	**说明**
value	int*		目标数（并得到设定好的数）
bit	int		指定位
返回值			
指令分类	指令列表_辅助功能		
相关章节			
相关例程			

adt_cpl_bit

指令格式			
void _stdcall adt_cpl_bit(int *value, int bit)			
指令描述			
指定位值取反			
参数说明			
名称	**数据类型**	**范围**	**说明**
value	int*		目标数（并得到设定好的数）
bit	int		指定位
返回值			
0 表示执行成功，其他返回值表示执行失败，失败原因参考标准错误码			
指令分类	指令列表_辅助功能		
相关章节			
相关例程			

第十九章 附录

19.1. 例程索引

例程 4-1 基础控制与版本信息获取
例程 5-1 从 RAM/ROM 中加载从站配置，解析配置，获取轴资源信息
例程 5-2 从本地文件导入从站配置
例程 6-1 总线 IO 端口映射
例程 6-2 配置轴基本参数
例程 7-1 轴状态监控
例程 7-2 停止信息解释示例
例程 9-1 定量驱动
例程 9-2 连续驱动
例程 9-6 龙门双驱
例程 10-1 直线插补
例程 10-2 平面圆弧插补
例程 10-3 缓存插补
例程 12-1 DSP 回零
例程 12-2 总线伺服回零
例程 14-1 缓存事件控制
例程 16-1 用户控制
例程 17-1 2D 位置比较

19.2. 例程附加接口与定义

DoEvent – 操作系统控制权转让

DoEvent 会将控制权传给操作系统。当操作系统处理完队列中的事件，并且在 SendKeys 队列中的所有键也都已送出之后，返回控制权。

DoEvent 对于简化诸如允许用户取消一个已启动的过程 — 例如搜寻一个文件 — 特别有用。对于长时间过程，放弃控制权最好使用定时器或通过委派任务给 ActiveX EXE 部件来完成。以后，任务还是完全独立于应用程序，多任务及时间片由操作系统来处理。

若系统对实时性要求很高，无暇顾及操作系统事件队列，可考虑将工艺流程放入线程，且不要使用 DoEvent。

```
void DoEvent()
{
	MSG msg;
	if (::PeekMessage(&msg, NULL, 0, 0, PM_REMOVE))
	{
		::TranslateMessage(&msg);
		::DispatchMessage(&msg);
	}
}
```

VERIFY_RETURN –执行错误则跳出主函数返回错误码

```
#define VERIFY_RETURN(func)	\
{	\
	int retn = func;	\
	if(retn)	\
	return retn;\
}
```

adt_till_stop – 等待指定轴驱动停止并检查停止信息

```
int adt_till_stop(int card, int axis)
{
	int stt = 0;
	while(true)
	{
```

```
        VERIFY_RETURN(adt_get_axis_status(card, axis, &stt));
        if(0 == stt)
            break;
    }
    if(INPA_AXIS==axis || INPB_AXIS==axis)
        return ERR_NONE;
    VERIFY_RETURN(adt_get_stopdata(card, axis, &stt));
    char msg[256] = {0};
    DecodeStopData(msg, 256, card, axis, stt);
    printf_s(msg);
    return ERR_NONE;
}
```

VERIFY_RETURN_MSG – 执行错误则解析打印并跳出主函数返回指定值

```
#define VERIFY_RETURN_MSG(func, itf, val)    \
{    \
    int retn = func;    \
    if(retn)    \
    {    \
    char msg[256] = {0};    \
    DecodeErrCode(msg, 256, retn, itf);\
    printf(msg);\
    system("pause"); \
    return val;  \
    }    \
}
```

CONFIRM_DRIVE — 驱动执行确认

```
#define CONFIRM_DRIVE(val)              {    \
    printf("参数已设置完成!\n请输入: 1--执行驱动 其他--不执行驱动并退出\n");    \
    int flag = 0;cin>>flag;  if(1 != flag) return val;    printf("执行驱动!\n");\
}
```

19.3. EtherCAT CIA 对象组常用 6000h 分配

索引	子索引	说明	访问	数据类型	PDO 映射
0x603F	0x00	错误代码	RO	UINT16	N
0x6040	0x00	控制字	RW	UINT16	Y
0x6041	0x00	状态字	RO	UINT16	Y
0x6060	0x00	操作模式	RW	INT8	Y
0x6061	0x00	操作模式显示	RO	INT8	Y
0x6064	0x00	反馈位置	RO	DINT	Y
0x606C	0x00	反馈速度	RO	DINT	Y
0x6071	0x00	目标转矩	RW	INT16	Y
0x6077	0x00	反馈转矩	RO	INT16	Y
0x607A	0x00	目标位置	RW	DINT	Y
0x607C	0x00	回零偏移量	RW	DINT	N
0x607D	0x01	反转限位值	RW	DINT	N
0x607D	0x02	正转限位值	RW	DINT	N
0x607E	0x00	极性控制	RW	UINT8	N
0x607F	0x00	最大轮廓速度	RW	UDINT	N
0x6081	0x00	轮廓速度	RW	UDINT	N
0x6083	0x00	轮廓加速度	RW	UDINT	N
0x6084	0x00	轮廓减速度	RW	UDINT	N
0x6087	0x00	转矩斜坡率	RW	UDINT	N
0x6091	0x01	主编码器齿轮比分子	RW	UDINT	N
0x6091	0x02	主编码器齿轮比分母	RW	UDINT	N
0x6091	0x03	辅编码器齿轮比分子	RW	UDINT	N
0x6091	0x04	辅编码器齿轮比分母	RW	UDINT	N
0x6098	0x00	回零类型	RW	UINT8	N
0x6099	0x01	第一回零速度	RW	UDINT	N
0x6099	0x02	第二回零速度	RW	UDINT	N
0x609A	0x00	回零加速度	RW	UDINT	N
0x60B0	0x00	位置偏移量	RW	DINT	N
0x60B1	0x00	速度偏移量	RW	DINT	N

0x60B2	0x00	转矩偏移量	RW	DINT	N
0x60FF	0x00	轮廓目标速度	RW	DINT	N

19.4. 文档版本

版本号	修订日期
1.0	2017/9/30
2.0	2020/4/27
3.0	2021/1/4
4.0	2023/3/30